



Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**



Grado en Ingeniería Informática

Trabajo Fin de Grado

**Implementación del Algoritmo para el
Cálculo de la Ecuación Diferencial
Matricial de Riccati con Procesamiento
Paralelo mediante OpenMP**

Autor: Borja Pérez-Villacastín Palacín
Tutor: José Antonio Mateo Cortés

Madrid, Enero 2025

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado
Grado en Ingeniería Informática

Título: Implementación del Algoritmo para el Cálculo de la Ecuación
Diferencial Matricial de Riccati con Procesamiento Paralelo me-
diante OpenMP

Enero 2025

Autor: Borja Pérez-Villacastín Palacín

Tutor: José Antonio Mateo Cortés

Departamento de Arquitectura y Tecnología de Sistemas Infor-
máticos

Escuela Técnica Superior de Ingenieros Informáticos

Universidad Politécnica de Madrid

Tabla de contenidos

1. Introducción	1
2. Estado del Arte	3
2.1. Métodos de Diferenciación Hacia Atras (BDF)	3
2.2. Algoritmo BDF	5
2.2.1. Introducción al problema	5
2.2.2. Estructura general del código	5
2.2.3. Flujo de ejecución general	10
2.2.4. Conclusión	10
2.3. OpenMP	11
3. Implementación	16
3.1. Matrix Subroutines	16
3.1.1. SolveLS	16
3.1.2. Mmul	20
3.1.3. Mcpy	23
3.1.4. Mset	24
3.1.5. Mnorm	26
3.1.6. Mmulc	29
3.1.7. Msub	30
3.2. Multi-Processed Matrix Subroutines	31
3.2.1. SolveLS	31
3.2.2. Mmul	34
3.2.3. Mcpy	35
3.2.4. Mset	36
3.2.5. Mnorm	37
3.2.6. Mmulc	39
3.2.7. Msub	39
4. Experimentación y Resultados	40
4.1. Entorno 1	44
4.2. Entorno 2	55
4.3. Entorno 3	65
4.4. Limonero DATSI	75
5. Conclusiones	86

TABLA DE CONTENIDOS

6. Análisis de impacto	88
Bibliografía	91
Anexos	93
A. Diseño de Pruebas	93
B. Procesado de Resultados MATLAB	100

Capítulo 1

Introducción

Jacopo Francesco Riccati [1], matemático italiano, dedicó parte de su vida al estudio de ecuaciones diferenciales, lo cual lo llevó a descubrir una solución particular para un tipo específico de ecuación, conocida hoy como la *Ecuación Diferencial de Riccati* (DRE). En un primer momento, esta ecuación tenía como objetivo el análisis de problemas hidrodinámicos; sin embargo, hoy en día tiene múltiples aplicaciones en diversas áreas de ciencia e ingeniería, como la mecánica cuántica, la física matemática y las finanzas, destacando entre ellas su aplicación en los problemas de control óptimo, pertenecientes al área de la teoría de control.

Los problemas de control óptimo suscitan un gran interés, ya que tienen un amplio rango de usos en distintas áreas. A grandes rasgos, buscan optimizar el comportamiento de un sistema bajo ciertas condiciones, utilizando una función que minimice (o maximice) un criterio de desempeño o costo, respetando además unas restricciones impuestas por el sistema. Nos pueden servir para cosas como gestionar la batería de vehículos autónomos, optimizar la trayectoria de cohetes o incluso para tratamientos médicos optimizados que mejoren la regulación de dosis de medicamento. En definitiva, tienen muchas aplicaciones y son de gran utilidad; no obstante, surge un verdadero problema a la hora de tratar de resolverlos. Su resolución pasa por la discretización de ecuaciones diferenciales parciales que generan DREs de gran escala, involucrando así matrices de gran tamaño. Además, muchos de los problemas de control óptimo tienen modos rápidos y lentos; es decir, algunas variables o componentes del sistema cambian de valor con rapidez, mientras que otras de manera más pausada, lo que genera que las DREs asociadas sean rígidas y se complique aún más su resolución numéricamente. Todo esto produce que sus métodos de resolución tengan una complejidad computacional muy elevada, que aumenta considerablemente de forma rápida. Es aquí donde se quiere poner el foco, en la complejidad exponencial derivada de estos problemas, que fuerza a buscar nuevas técnicas o formas más eficientes de procesar los algoritmos.

En un contexto donde las aplicaciones requieren resolver problemas cada vez más complejos y manejar grandes volúmenes de datos, la computación secuencial no puede seguir escalando al ritmo necesario. Eso se debe principalmente

Capítulo 1. Introducción

a las limitaciones físicas en la frecuencia de los procesadores y al alto consumo energético que conlleva el aumento de velocidad en un solo núcleo. El procesamiento paralelo surge como respuesta a estos problemas y permite abordar las limitaciones distribuyendo el trabajo entre varios núcleos que trabajan de forma simultánea en una misma tarea, aprovechando así los recursos de manera más inteligente. Existen muchas herramientas y librerías que permiten implementar programación paralela en el código, pero una de ellas nos llama especialmente la atención: OpenMP.

Llegados a este punto, el objetivo de este trabajo de fin de grado consistirá en implementar un algoritmo para la resolución de la Ecuación Diferencial Matricial de Riccati mediante procesamiento paralelo con OpenMP. El uso de este método permitirá explorar la eficiencia y escalabilidad del algoritmo comparando su rendimiento en términos de tiempo de ejecución y la optimización de recursos. Con este enfoque no solo se busca resolver problemas de gran escala con mayor rapidez, sino también contribuir al desarrollo de técnicas optimizadas para problemas rígidos, típicos en el contexto de las DMREs.

En cuanto a la estructura de esta memoria, comenzaremos con un capítulo dedicado al estado del arte del trabajo a realizar. Este capítulo contendrá un resumen de los diferentes métodos de resolución de la Ecuación Diferencial de Riccati, un análisis y muestra del algoritmo que implementaremos, y terminará tratando OpenMP junto con las funcionalidades que aporta. Por otro lado, continuará con un capítulo dedicado a la implementación del algoritmo, en el que se desarrollarán dos librerías que jugarán un papel fundamental en este. Para continuar, el siguiente capítulo se centrará en exponer las diferentes pruebas realizadas sobre ambas librerías en diferentes entornos de experimentación, junto con sus resultados y un análisis específico para cada entorno. Posteriormente, se reflejarán las conclusiones finales de este trabajo de fin de grado en otro capítulo, terminando con un último en el que se analizará el posible impacto de este en diferentes áreas. También cabe destacar que contará con varios anexos cuyo contenido creemos que no era oportuno incluir en ningún capítulo debido al foco de este trabajo de fin de grado, pero que aun así, creemos que han sido relevantes para su desarrollo. En ellos se podrá encontrar el diseño de las pruebas realizadas o diversos scripts destinados a procesar y graficar la gran cantidad de resultados obtenidos en las pruebas.

Capítulo 2

Estado del Arte

2.1. Métodos de Diferenciación Hacia Atras (BDF)

La implementación de algoritmos para resolver la ecuación diferencial de Riccati (DRE) ha cobrado relevancia en los últimos años debido a su aplicación en diversas áreas, anteriormente mencionadas. El desarrollo de métodos numéricos eficientes es esencial para el manejo de problemas de gran escala y rigidez matemática, por lo que haremos un breve repaso por los distintos tipos de métodos o enfoques que existen para la resolución de DREs:

- **Vectorización:** Como su nombre indica, se basan en vectorizar la DRE. Desenrollan las matrices en vectores que generan un sistema formado por n^2 ecuaciones diferenciales, que se resuelve utilizando cualquier esquema de integración numérica. Lo podemos considerar un enfoque un tanto ingenuo, no es adecuado para problemas de gran escala, ya que para métodos implícitos habría que resolver sistemas no lineales de n^2 incógnitas en cada paso de tiempo.
- **Linealización:** Buscan transformar la DRE en un sistema de ecuaciones diferenciales matriciales lineales de primer orden, para posteriormente resolver este. Aunque pueden ser métodos eficientes para algunos problemas, al igual que la vectorización, nos resultan poco prácticos ante problemas de gran escala debido al cálculo necesario de la exponencial de matrices ($2n \times 2n$).
- **Método de Chandrasekhar:** Este método transforma la DRE en dos sistemas acoplados de ecuaciones diferenciales no lineales, pero su aplicación es limitada por dificultades numéricas y es inestable en general, incluso si es adecuado para problemas de gran escala.
- **Métodos de Superposición:** Estos métodos se basan en la propiedad de superposición de soluciones de Riccati, con la que, conociendo una solución particular de la ecuación, permite construir la solución general. Aunque útiles teóricamente, implican integrar varias veces la DRE con diferentes condiciones iniciales, lo que resulta en una complejidad computacional

alta, especialmente para problemas de gran escala.

- **Versiones matriciales de métodos estándar para ODEs:** Estos métodos utilizan algoritmos matriciales basados en algoritmos estándar para resolver Ecuaciones Diferenciales Ordinarias (ODEs). Aquí se presta especial atención a los métodos que, al expresarse en forma matricial, generan ecuaciones algebraicas de Riccati (AREs) como el sistema no lineal que debe resolverse en cada paso. Para esto hay muchas formas, pero en este contexto quizás la más atractiva sea los métodos de Diferencias Hacia Atrás (BDF)[2]. Estos métodos transforman la ecuación diferencial en una secuencia de problemas algebraicos que se resuelven iterativamente, permitiendo así una implementación eficiente para los problemas de gran escala, y además, siendo particularmente adecuados para ODEs rígidas.

Tras analizar los distintos tipos de métodos, se llega fácilmente a la conclusión de que el más atractivo para el contexto en el que nos encontramos son las versiones matriciales con BDF. Ya no solo porque nos sirven para ODEs rígidas y tienen buen rendimiento para problemas de gran escala, sino porque el uso de matrices permite explotar fácilmente la programación paralela durante las operaciones con ellas.

La implementación de los métodos BDF ya se ha probado con anterioridad en sistemas paralelos, usando plataformas tanto de CPU[2] como GPU[3], mostrando mejoras significativas en rendimiento. La paralelización permite descomponer operaciones de álgebra lineal en múltiples procesadores, reduciendo así el tiempo de ejecución. En particular, el uso de GPUs a través de bibliotecas como CUBLAS[4] ha sido eficaz para operaciones de gran tamaño, superando el rendimiento de CPUs multinúcleo en operaciones específicas de álgebra lineal. Sin embargo, el manejo de la memoria y la optimización energética siguen siendo áreas críticas en el desarrollo de implementaciones paralelas.

En conclusión, el método BDF es una de las estrategias más prometedoras para la resolución de DMREs debido a su adaptabilidad a problemas rígidos y de gran escala. Combinado junto con el procesamiento paralelo, nos permite abordar problemas de alta complejidad, abriendo oportunidades para una mayor optimización y eficiencia en el cálculo de soluciones en aplicaciones avanzadas.

2.2. Algoritmo BDF

Para el desarrollo de este trabajo de fin de grado se parte de una versión secuencial del algoritmo a implementar en MATLAB, la cual nos ayudará a tener un mejor entendimiento del algoritmo y nos permitirá desarrollar una versión paralela de este mismo de una manera más eficiente. Este capítulo se centrará únicamente en las partes y el funcionamiento de este algoritmo, enseñando partes de su código y explicándolas en detalle.

2.2.1. Introducción al problema

El conjunto de funciones en MATLAB que ha sido entregado, implementa el método mencionado anteriormente de diferenciación hacia atrás (Backward Differentiation Formula, BDF) para resolver ecuaciones diferenciales matriciales de Riccati (DMRE). Estas ecuaciones tienen la forma general:

$$\dot{X}(t) = A_{21}(t) + A_{22}(t)X(t) - X(t)A_{11}(t) - X(t)A_{12}(t)X(t), \quad (2.1)$$

con una condición inicial $X(0) = X_0$. El objetivo del código es calcular una aproximación numérica de la solución $X(t)$ en un intervalo de tiempo $[0, t_f]$. Debido a la complejidad de estas ecuaciones, el método BDF convierte el problema diferencial en una secuencia de problemas algebraicos (ARE, Algebraic Riccati Equations), los cuales se resuelven iterativamente.

2.2.2. Estructura general del código

El flujo general del código se divide en tres componentes principales:

1. **Configuración y supervisión (`pridr_bdf`):** Esta función organiza los parámetros del problema y llama a las funciones auxiliares para resolver la ecuación diferencial.
2. **Método numérico BDF (`dgeirdbdf`):** Se encarga de discretizar la ecuación diferencial y resolver las AREs en cada paso temporal.
3. **Resolución de AREs (`dgearrefpo`):** En cada paso del método BDF, esta función resuelve las ecuaciones algebraicas subyacentes utilizando un método iterativo de punto fijo.

A continuación, se explica el propósito y funcionamiento de cada una de estas funciones en detalle.

Función principal: `pridr_bdf`

La función `pridr_bdf` (Listing 2.1) actúa como el controlador principal del proceso. Configura los parámetros del problema, selecciona los datos apropiados y coordina la ejecución del método BDF. Según el valor del parámetro k , el código selecciona diferentes funciones para generar las matrices del sistema y las condiciones iniciales del problema.

Capítulo 2. Estado del Arte

Dentro de esta función, se iteran diferentes configuraciones de paso de tiempo (H) y tiempo final (T_f), llamando a la función `dgeidrdbdf` para resolver la ecuación Riccati en cada caso. Una vez obtenida la solución, `pridr_bdf` evalúa la precisión calculando el error relativo entre la solución numérica y una solución analítica proporcionada por otra función (`datasc2`). Además, registra el tiempo de ejecución y la precisión de los resultados.

Esta función permite explorar y evaluar el rendimiento del método BDF bajo distintas configuraciones, siendo esencial para la validación del algoritmo.

```
1 function pridr_bdf(k,pn,pr,H,Tf,r,tol,maxiter)
2 if k==1
3     f= 'datac1';
4     f0='data0c1';
5     fs='datasc1';
6 elseif k==2
7     f= 'datac2';
8     f0='data0c2';
9     fs='datasc2';
10 else
11     f= 'datac3';
12     f0='data0c3';
13     fs='datasc3';
14 end
15
16 fprintf('Invariant DRE %d: Pn=%d Pr=%g\n',k,pn,pr)
17
18 fprintf('BDF: r=%d tol=%e MaxIter=%d\n',r,tol,maxiter)
19 for i=1:length(H)
20     h=H(i);
21     for j=1:length(Tf)
22         tf=Tf(j);
23         fprintf('\n')
24         tic;
25         [X0,t,k]=dgeidrdbdf(pn,pr,f,f0,tf,h,r,'dgearefpo',tol,maxiter);
26         fl=toc;X=feval(fs,pn,pr,t);
27         normX=norm(X);
28         if isnan(X0)
29             e=-1;
30         elseif normX>0
31             e=norm(X-X0)/normX;
32         else
33             e=norm(X-X0);
34         end
35         fprintf('dgeidrdbdf(%s) tf=%d h=%e: k=%6d Te=%10f e=%e\n','dgearefpo',tf,
36             ,h,k,fl,e)
37     end
38 end
39 X0
```

Listing 2.1: Implementación de `pridrdbdf` en MATLAB

Implementación del método BDF: dgeirdbdf

La función `dgeirdbdf` (Listing 2.2) implementa el método BDF para aproximar la solución de la DMRE. El método BDF convierte la ecuación diferencial en una serie de ecuaciones algebraicas mediante discretización temporal. En los primeros pasos ($r-1$, donde r es el orden del método BDF), se generan soluciones preliminares ($X_1, X_2, \dots, X_{r-1}$) usando una versión simplificada de la ecuación. Para los pasos posteriores ($X_r, X_{r+1}, \dots$), se resuelven AREs completas.

Cada paso temporal requiere ajustar las matrices del sistema utilizando coeficientes predefinidos del método BDF (α y β). Luego, se llama a una función auxiliar, generalmente `dgearefpo`, para resolver el sistema algebraico correspondiente. Este proceso se repite iterativamente hasta alcanzar el tiempo final t_f o hasta que el método detecte un error de convergencia.

En resumen, `dgeirdbdf` es el núcleo del cálculo numérico, que organiza las iteraciones necesarias y asegura que la solución se calcule paso a paso de manera precisa.

```

1  function [X0,t1,niter]=dgeirdbdf(pn,pr,datat,data0,tf,h,r,met,tol,maxiter)
2
3  beta=[ 1      2/3      6/11      12/25      60/137      60/147];
4  alfa=[ 1      0      0      0      0      0;
5         4/3     -1/3      0      0      0      0;
6         18/11   -9/11     2/11     0      0      0;
7         48/25   -36/25    16/25    -3/25    0      0;
8         300/137 -300/137  200/137  -75/137  12/137  0;
9         360/147 -450/147  400/147  -225/147  72/147  -10/147];
10
11  % Obtaining the initial conditions
12
13  [t0,X0]=feval(data0,pn,pr);
14  [m,n]=size(X0);
15  Im=eye(m);
16  % The following loop computes the solutions in the first r-1 steps (X1, X2,...,
    Xr-1)
17  niter=0;
18  t=t0;
19  [A11,A12,A21,A22]=feval(datat,pn,pr,t);
20
21
22  for j=1:r-1
23      % Updating the matrices Xi
24      for k=j:-1:2
25          X(1:m,1:n,k)=X(1:m,1:n,k-1);
26      end
27      X(1:m,1:n,1)=X0;
28      t=t+h;
29      % BDF step
30      hb=h*beta(j);
31      A21m=hb*A21;
32      for k=1:j
33          A21m=A21m+alfa(j,k)*X(1:m,1:n,k);
34      end
35      A22m=hb*A22-Im;
36      A11m=hb*A11;

```

```
37     A12m=hb*A12;
38     [X0,k]=feval(met,A11m,A12m,A21m,A22m,X0,tol,maxiter);
39     t,k,X0
40     t1=t;
41     if k== -1
42         niter=-1;
43         return
44     end
45     niter=niter+k;
46 end
47
48 % The following loop computes the solutions for the following steps (Xr, Xr
49 % +1,...)
50
51 hb=h*beta(r);
52
53 while t<t1+h
54     % Updating the matrices Xi
55     for k=r:-1:2
56         X(1:m,1:n,k)=X(1:m,1:n,k-1);
57     end
58     X(1:m,1:n,1)=X0;
59     t=t+h;
60     % BDF step
61     A21m=hb*A21;
62     for k=1:r
63         %hb, alfa(r,k), A21
64         %X(1:m,1:n,k)
65         A21m=A21m+alfa(r,k)*X(1:m,1:n,k);
66     end
67     A22m=hb*A22-Im;
68     A11m=hb*A11;
69     A12m=hb*A12;
70     [X0,k]=feval(met,A11m,A12m,A21m,A22m,X0,tol,maxiter);
71     t1=t;
72     if k== -1
73         niter=-1;
74         return
75     end
76     niter=niter+k;
77 end
```

Listing 2.2: Implementación de dgeirdbdf en MATLAB

Resolución de ecuaciones algebraicas: dgearefpo

La función `dgearefpo` (Listing 2.3) resuelve las ecuaciones algebraicas de Riccati que surgen en cada paso BDF. Estas ecuaciones tienen la forma:

$$A_{21} + A_{22}X - XA_{11} - XA_{12}X = 0. \quad (2.2)$$

El método utilizado es iterativo y combina el método de Newton con un enfoque de punto fijo. En cada iteración, `dgearefpo` realiza los siguientes pasos:

1. Calcula las matrices intermedias:

$$C_{22} = A_{22} - XA_{12}, \quad C_{21} = -A_{21} + XA_{11}.$$

2. Resuelve el sistema lineal $C_{22}X = C_{21}$ para actualizar X .
3. Calcula el error usando la norma de Frobenius para comparar la solución actual con la anterior.
4. Si el error es menor que la tolerancia especificada (`tol`), se considera que la solución ha convergido. De lo contrario, continúa iterando hasta alcanzar el número máximo de iteraciones (`maxiter`).

Si el método no converge dentro de las iteraciones permitidas, se devuelve un error, lo que indica que el problema no pudo resolverse con los parámetros dados.

```

1 function [X0,k]=dgearefpo(A11,A12,A21,A22,X0,tol,maxiter)
2 k=0;
3 while k<maxiter
4     k=k+1;
5     C22=A22-X0*A12;
6     C21=-A21+X0*A11;
7     C21=C22\C21;
8     % norma=norm(C21-X0,inf);
9     norma= norm(X0-C21,'fro'); % We change the norm to compare with CUDA
10    X0=C21;
11    if norma<tol
12        break
13    end
14 end
15 if k==maxiter
16     k=-1;
17 end

```

Listing 2.3: Implementación de `dgearefpo` en MATLAB

2.2.3. Flujo de ejecución general

El flujo de ejecución completo es el siguiente:

1. **Inicialización (pridr_bdf):**

- Se definen los parámetros del problema y se seleccionan las funciones adecuadas para las matrices y condiciones iniciales.

2. **Discretización temporal (dgeirdbdf):**

- Se transforma la ecuación diferencial en pasos discretos mediante el método BDF.
- Se resuelven los problemas algebraicos en cada paso llamando a dgearefpo.

3. **Resolución iterativa (dgearefpo):**

- En cada paso, se resuelve una ARE hasta que la solución converge o se alcanzan las iteraciones máximas.

4. **Evaluación de resultados (pridr_bdf):**

- Se calcula el error relativo entre la solución obtenida y una referencia analítica.
- Se registra el tiempo de ejecución y otros detalles para analizar el rendimiento.

2.2.4. Conclusión

El conjunto de funciones MATLAB implementa de manera modular y eficiente un método numérico basado en BDF para resolver ecuaciones diferenciales matriciales de Riccati. La división en módulos (pridr_bdf, dgeirdbdf, dgearefpo) facilita la adaptabilidad y el análisis del algoritmo, permitiendo su evaluación en diferentes configuraciones y escenarios. Este diseño estructurado será crucial para trasladar el algoritmo a un entorno paralelo en C, optimizando su ejecución en sistemas de alto rendimiento.

2.3. OpenMP

OpenMP (Open Multi-Processing)[5] es considerado un estándar "de facto" para la paralelización en sistemas de memoria compartida. Ofrece un modelo basado en directivas que facilita la creación y sincronización de hilos, permitiendo especificar qué partes del código deben ejecutarse en paralelo, cómo se distribuyen las tareas entre los hilos y cómo se sincronizan las operaciones. Su objetivo principal es facilitar la implementación de paralelismo sin requerir la reestructuración completa del programa, por ello sus directivas nos permiten dividir automáticamente las iteraciones de bucles entre los diferentes núcleos del procesador, maximizando el aprovechamiento de los recursos disponibles y reduciendo significativamente el tiempo de cálculo. Además, es importante destacar en el contexto en el que nos encontramos que es especialmente útil en el procesamiento de matrices, ya que las operaciones matriciales suelen implicar múltiples bucles anidados en los que cada iteración realiza cálculos independientes del resto que pueden ser paralelizados de manera eficiente.

Como se ha mencionado anteriormente, OpenMP se basa en el uso de memoria compartida (Figura 2.1) durante la creación y sincronización de hilos. Esto significa que, de base, todos los hilos tienen acceso al mismo espacio de memoria global, por lo que los datos almacenados en variables compartidas son visibles y modificables por cualquier hilo. Este enfoque permite que múltiples hilos trabajen sobre los mismos datos en paralelo, simplificando la programación y evitando la necesidad de comunicación explícita entre procesos, como ocurre en los modelos de memoria distribuida. También hay que destacar que, aunque de base las variables sean compartidas, OpenMP incluye directivas para la creación de variables privadas a cada hilo, ya que no siempre se quiere que todos compartan la misma información.

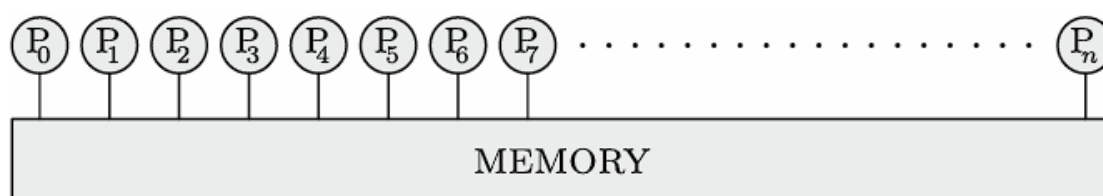


Figura 2.1: Multiprocesador con memoria compartida [5]

En cuanto a la arquitectura subyacente de OpenMP, sus simples directivas ocultan un enfoque complejo basado en hilos. El hilo maestro es el encargado de iniciar el programa y cuando llega a una región paralela en el código, este hilo maestro crea un conjunto de hilos secundarios, que ejecutan partes del trabajo de manera concurrente. Al final de la región paralela, los hilos secundarios se sincronizan y terminan, dejando que el hilo maestro continúe con la ejecución secuencial.

A continuación, con el fin de ilustrar mejor su funcionamiento y facilitar su entendimiento, se van a mostrar varios ejemplos de programas sencillos con

Capítulo 2. Estado del Arte

OpenMP.

El primero de ellos es un "Hola mundo". Como se puede observar en el Listing 2.4, se ha incluido la directiva `#pragma omp parallel` que indica que llega una región paralela. Durante la ejecución, al alcanzar dicha región, se crean hilos que ejecutan el contenido de su interior.

```
1 ...  
2 #pragma omp parallel  
3 {  
4     printf("Hola mundo. Hilo: %i", omp_get_thread_num());  
5 }  
6 ...
```

Listing 2.4: Hola mundo, OpenMP

```
Hola mundo. Hilo: 2  
Hola mundo. Hilo: 4  
Hola mundo. Hilo: 3  
Hola mundo. Hilo: 5  
Hola mundo. Hilo: 0  
Hola mundo. Hilo: 1
```

Listing 2.5: Resultados Hola Mundo con OpenMP

Como se puede apreciar en el Listing 2.5, la ejecución de los hilos no sigue un orden, esto se debe a que los hilos ejecutan sus tareas de manera concurrente y el sistema operativo no garantiza un orden específico en su ejecución. Cada hilo compite por los recursos del procesador y puede comenzar o terminar su tarea en un momento diferente, dependiendo de factores como la carga del sistema. También dependerá de la programación del sistema operativo, ya que es el encargado de asignar tiempo de CPU a los hilos de acuerdo con su propio planificador, que prioriza tareas en función de factores internos. Todo esto provoca que no se pueda saber de manera exacta el orden de ejecución de los hilos, por lo que para determinadas tareas se tendrán que buscar formas de sincronizarlos.

Para el siguiente ejemplo, mostrado en el Listing 2.6, se va a paralelizar un bucle `for` mediante la directiva `#pragma omp parallel for`. Con esto se consigue que OpenMP se encargue de dividir automáticamente las iteraciones del bucle y repartirlas entre los hilos para que las ejecuten de forma paralela.

```
1 ...  
2 #pragma omp parallel for  
3 for(int i=0; i<8; i++){  
4     printf("Iteracion: %i, Hilo: %i \n", i, omp_get_thread_num());  
5 }  
6 ...
```

Listing 2.6: Bucle `for` paralelizado con OpenMP

```
Iteracion: 0, Hilo: 0  
Iteracion: 1, Hilo: 0  
Iteracion: 4, Hilo: 2  
Iteracion: 5, Hilo: 2  
Iteracion: 6, Hilo: 3
```



```
Iteracion: 7, Hilo: 3
Iteracion: 2, Hilo: 1
Iteracion: 3, Hilo: 1
```

Listing 2.7: Resultados paralelizacion bucle for con OpenMP

Como se puede observar en los resultados mostrados en el Listing 2.7, OpenMP divide en partes iguales las iteraciones del bucle entre los diferentes hilos. Esto es muy útil para casos sencillos; sin embargo, para paralelizar de manera eficiente, se tendrá que tener en cuenta siempre el número de iteraciones de los bucles y, sobre todo, la carga de trabajo que conlleva cada una de ellas. Si se distribuyen las iteraciones a partes iguales entre los hilos y algunas de ellas requiriesen más carga de trabajo que las otras, la carga de trabajo total entre los hilos quedaría desbalanceada y se obtendría un tiempo de ejecución mayor. Para conseguir la mayor eficiencia, se va a hacer uso de las cláusulas `schedule(static, N)` y `schedule(dynamic, N)`.

De base, la paralelización de bucles for es siempre static con $N = \text{iteraciones} / N^{\circ}\text{Threads}$, a no ser que se especifique lo contrario. Esta cláusula hace que las iteraciones se dividan al comienzo de la ejecución del bucle en grupos de N iteraciones entre los hilos. Es útil para bucles cuyas iteraciones tengan cargas de trabajo similares; sin embargo, en muchos bucles esto no es así, por lo que a continuación se va a mostrar en el Listing 2.8 un sencillo ejemplo con la cláusula `schedule(dynamic, N)`.

```
1 ...
2 #pragma omp parallel for schedule(dynamic, 1)
3 for(int i=0; i<8; i++){
4     if(i==0 || i==4) sleep(1);
5     printf("Iteracion: %i, Hilo: %i \n", i, omp_get_thread_num());
6 }
7 ...
```

Listing 2.8: Bucle for paralelizado dinamicamente con OpenMP

```
Iteracion: 2, Hilo: 0
Iteracion: 1, Hilo: 2
Iteracion: 5, Hilo: 2
Iteracion: 6, Hilo: 2
Iteracion: 7, Hilo: 2
Iteracion: 3, Hilo: 3
Iteracion: 4, Hilo: 0
Iteracion: 0, Hilo: 1
```

Listing 2.9: Resultados paralelizacion dinámica bucle for con OpenMP

En los resultados de este caso, mostrados en el Listing 2.9, la cláusula `schedule(dynamic, 1)` se encarga de dividir las iteraciones de una en una dinámicamente entre los hilos, es decir, cuando un hilo termina de ejecutar su iteración asignada se le asigna otra de las que hay libres. De esta forma, se puede ver que el hilo número 2 ejecuta tres iteraciones, ya que los hilos 0 y 1 están ocupados ejecutando iteraciones que llevan más tiempo.

Capítulo 2. Estado del Arte

Ahora que se ha desarrollado una pequeña base de OpenMP, se va a plantear un problema a la hora de paralelizar el código, el cual se muestra en el Listing 2.10.

```
1 ...
2 int sum=0;
3 #pragma omp parallel for
4 for(int i=1; i<=100; i++){
5     sum=sum+i;
6 }
7 printf("Sumatorio = %i", sum);
8 ...
```

Listing 2.10: Sumatorio de 1 al 100 paralelizado (Incorrecto)

```
Sumatorio = 190
```

Listing 2.11: Resultado paralelizacion sumatorio (Incorrecto)

A primera vista, puede parecer que el código es correcto; sin embargo, al mirar su resultado, mostrado en el Listing 2.11, queda evidente que algo no funciona como debería. El resultado correcto es 5050, pero se obtiene 190, una cifra bastante alejada del resultado esperado. Como se ha mencionado anteriormente, los hilos comparten memoria y el valor de las variables, lo cual nos ayuda en muchos casos, pero en muchos otros, como este, se debe tener en cuenta. Cuando los diferentes hilos acceden simultáneamente a la variable `sum`, ya sea para leerla o para almacenar el resultado de la operación, se superponen los unos con los otros, causando fallos en la integridad de la memoria. Esto se llama *Condicion de Carrera*, debido a que los hilos están compitiendo por el uso de la variable. Para solucionarlo, se deben buscar formas de que el programa realice el mismo trabajo, pero sincronizando los hilos y coordinando los accesos a la variable que comparten. A continuación, se va a mostrar en el Listing 2.12 un ejemplo de una solución posible y en el Listing 2.13 su resultado.

```
1 ...
2 omp_set_num_threads(5);
3 int sum=0; int sumThrd=0; int iteraciones=0;
4 #pragma omp parallel for firstprivate(iteraciones, sumThrd) schedule(static,
5     20)
6 for(int i=1; i<=100; i++){
7     sumThrd=sumThrd+i;
8     iteraciones++;
9     if(iteraciones == 20){
10         #pragma omp critical
11         sum=sum+sumThrd;
12     }
13 }
14 printf("Sumatorio = %i\n", sum);
15 ...
```

Listing 2.12: Sumatorio de 1 al 100 paralelizado (Poco eficiente)

```
Sumatorio = 5050
```

Listing 2.13: Resultado paralelizacion sumatorio (Poco Eficiente)

Para lograr la solución, se ha tenido que crear una copia privada para cada hilo de las variables `iteraciones` y `sumThr` mediante la cláusula `firstprivate` (`iteraciones, sumThr`) (existen otras parecidas como `private(x)` o `lastprivate(x)` pero en este caso se necesita que se inicialicen todas con el valor de la variable al llegar a la zona paralela). Estas variables servirán para almacenar la suma de cada hilo y el número de iteraciones que ha sumado, para posteriormente, cuando hayan sumado todas sus iteraciones, sumar al total, el cual está protegido por la cláusula `#pragma omp critical` que garantiza que solo un hilo pueda ejecutar la sección a la vez. Finalmente, se consigue la solución correcta, pero de una forma engorrosa y poco eficiente. OpenMP tiene integrada una cláusula que solventa fácilmente este problema, la cláusula `reduction(op:var)`.

```

1 ...
2 int sum=0;
3 #pragma omp parallel for reduction(+:sum)
4 for(int i=0; i<=100; i++){
5     sum=sum+i;
6 }
7 printf("Sumatorio = %i\n", sum);
8 ...

```

Listing 2.14: Sumatorio de 1 al 100 paralelizado (Correcto)

```
Sumatorio = 5050
```

Listing 2.15: Resultado paralelizacion sumatorio (Correcto)

Como se puede apreciar en el Listing 2.14, la solución idónea requiere muchas menos líneas de código y modificaciones que la anterior. Se consigue el mismo resultado (Listing 2.15) con una sola cláusula, la cual realiza el mismo trabajo que el código anterior. En la Figura 2.2 se puede apreciar una sencilla ilustración del funcionamiento de esta cláusula.

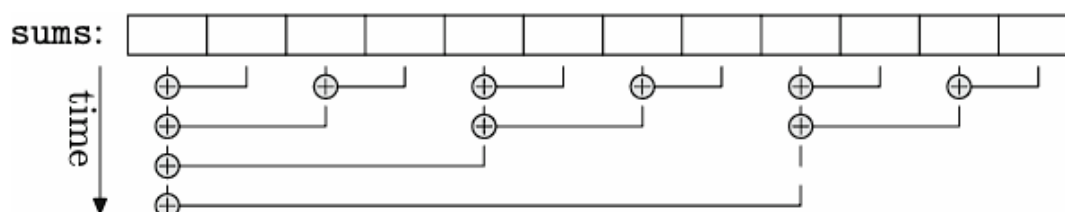


Figura 2.2: Funcionamiento `reduction(+:sum)` [5]

Esta cláusula es una pequeña muestra de la potencia y sencillez de OpenMP. Es una amplia herramienta que cuenta con muchas funcionalidades, pero aquí solo se pretende hacer un breve repaso de las bases, funcionalidades y desafíos de programar con OpenMP, con el fin de facilitar la comprensión de la paralelización del algoritmo a tratar en este trabajo de fin de grado.

Capítulo 3

Implementacion

Tras el análisis del algoritmo, se llega a la conclusión de que donde más se puede explotar las ventajas de la programación paralela es en las constantes operaciones matriciales que hay dentro de este, puesto que en todas ellas es necesario realizar una gran cantidad de cálculos totalmente independientes entre sí, permitiendo así dividirlos entre los distintos núcleos del procesador. Además, todos los cálculos con matrices en C suelen conllevar múltiples bucles "for" que se podrán paralelizar sin mucha complejidad con pragmas de OpenMP.

Todo esto ha llevado a la creación de dos librerías; la primera de ellas contendrá nuestra propia implementación de algunas funciones con matrices que se utilizan de manera repetida en el algoritmo, mientras que la segunda consistirá en la unión de la implementación anterior junto con directivas de OpenMP que permitirán implementar procesamiento paralelo en el código. De esta forma, se busca no solo poder presentarlo de manera más clara, sino principalmente permitirnos realizar pruebas más eficientes en las que se estudie cada parte paralelizable del código con el fin de conseguir el mayor rendimiento.

3.1. Matrix Subroutines

La librería ha sido nombrada "Matrix Subroutines"(MS) ya que, como se ha dicho antes, únicamente implementa operaciones de matrices. A continuación, se van a tratar las diferentes funciones que incluye, explicando tanto su propósito como los parámetros de entrada y de salida y su funcionamiento. Para ello, se añadirán fragmentos del código de la librería que se cree que ayudarán a mejorar su entendimiento y comprensión.

3.1.1. SolveLS

Propósito La función `solveLS` (Listing 3.10) resuelve un sistema de ecuaciones lineales de la forma $AX = B$, donde A es una matriz cuadrada de dimensiones $N \times N$, B es una matriz de términos independientes de dimensiones $N \times NRHS$, y X es la solución del sistema. La solución se obtiene utilizando la factorización LU con pivoteo parcial, seguida de la resolución de sistemas triangulares.

Parámetros

- *N*: Puntero a un entero que indica el tamaño *N* de la matriz cuadrada *A*.
- *NRHS*: Puntero a un entero que indica el número de columnas en la matriz *B* (es decir, el número de sistemas a resolver simultáneamente).
- *A*: Puntero a un arreglo de números en punto flotante que representa la matriz *A*. Esta matriz será transformada durante la factorización LU.
- *LDA*: Puntero a un entero que indica la longitud de la primera dimensión de la matriz *A* en memoria.
- *IPIV*: Puntero a un arreglo de enteros que almacena los índices de pivote utilizados durante la factorización.
- *B*: Puntero a un arreglo de números en punto flotante que representa la matriz *B*. Al final de la ejecución, contendrá la solución *X*.
- *LDB*: Puntero a un entero que indica la longitud de la primera dimensión de la matriz *B* en memoria.
- *INFO*: Puntero a un entero que almacena información sobre el estado de la operación. Si es distinto de cero, indica que ocurrió un error durante la factorización.

Funcionamiento

1. **Factorización LU:** Se llama a la función auxiliar `mlufact`, que realiza la factorización LU de la matriz *A* con pivoteo parcial.
 - Si ocurre un error durante esta etapa (por ejemplo, si *A* es singular), el valor de `INFO` se actualizará y la ejecución de `solve`s se detendrá.
2. **Resolución del sistema triangular:** Si la factorización LU es exitosa, se llama a la función auxiliar `tsolve` para resolver los sistemas triangulares generados por la factorización, obteniendo la solución *X*.

```

1 void solve_s(int *N, int *NRHS, double *A, int *LDA, int *IPIV, double *B, int
  *LDB, int *INFO) {
2     // Factorización LU con pivoteo parcial
3     mlufact(N, A, LDA, IPIV, INFO);
4
5     // Si hubo un error en la factorización, salir
6     if (*INFO != 0) {
7         return;
8     }
9
10    // Resolver el sistema triangular usando la factorización LU
11    tsolve(N, NRHS, A, LDA, B, LDB, IPIV);
12 }

```

Listing 3.1: Implementacion de `solve_s` en C

Mlufact

Propósito La función `mlufact` realiza la factorización LU con pivoteo parcial de la matriz A . Esta descompone A en dos matrices triangulares: L (triangular inferior) y U (triangular superior), donde $PA = LU$ y P es una matriz de permutación que refleja los pivotes utilizados.

Parámetros

- N, A, LDA : Como descrito anteriormente.
- $IPIV$: Arreglo que almacena los índices de las filas permutadas.
- $INFO$: Indicador de error. Si es distinto de cero, indica que la matriz A es singular.

Funcionamiento

1. **Búsqueda del pivote:** Para cada columna, se busca el elemento de mayor valor absoluto en la columna activa y se registra su posición.
2. **Intercambio de filas:** Si el pivote no está en la posición diagonal actual, se intercambian las filas correspondientes.
3. **Factorización LU:**
 - Cada elemento por debajo del pivote se divide por el valor del pivote.
 - Los valores en la matriz se actualizan restando el producto del elemento en L con los correspondientes en U .

```
1 void mlufact(int *N, double *A, int *LDA, int *IPIV, int *INFO) {
2     *INFO = 0;
3     int n=*N; int lda=*LDA;
4     for (int i = 0; i < n; i++) {
5         // Buscar el pivote máximo en la columna actual
6         int max_index = i;
7         double max_value = fabs(A[i + i * lda]);
8         for (int k = i + 1; k < n; k++) {
9             double abs_val = fabs(A[k + i * lda]);
10            if (abs_val > max_value) {
11                max_value = abs_val;
12                max_index = k;
13            }
14        }
15
16        // Verificar si el pivote es cero (matriz singular)
17        if (max_value == 0.0) {
18            *INFO = i + 1;
19            return;
20        }
21
22        // Guardar el índice de pivote
23        IPIV[i] = max_index;
24
25        // Intercambiar filas si es necesario
26        if (max_index != i) {
27            for (int j = 0; j < n; j++) {
```

```

28         double temp = A[i + j * lda];
29         A[i + j * lda] = A[max_index + j * lda];
30         A[max_index + j * lda] = temp;
31     }
32 }
33
34 // Factorización LU
35 for (int j = i + 1; j < n; j++) {
36     A[j + i * lda] /= A[i + i * lda];
37     for (int k = i + 1; k < n; k++) {
38         A[j + k * lda] -= A[j + i * lda] * A[i + k * lda];
39     }
40 }
41 }
42 }

```

Listing 3.2: Factorización LU con pivoteo parcial

Tsolve

Propósito La función `tsolve` utiliza la factorización LU generada por `mlufact` para resolver el sistema de ecuaciones. Esto se logra mediante sustituciones hacia adelante y hacia atrás.

Parámetros

- $N, NRHS, A, LDA, B, LDB, IPIV$: Como descrito anteriormente.

Funcionamiento

1. **Aplicar permutaciones:** Se reordenan las filas de la matriz B según los índices de pivote almacenados en $IPIV$.
2. **Sustitución hacia adelante:** Resuelve $L \cdot Y = B$, donde L es triangular inferior. Esto se realiza iterando desde la primera fila hasta la última.
3. **Sustitución hacia atrás:** Resuelve $U \cdot X = Y$, donde U es triangular superior. Esto se realiza iterando desde la última fila hacia la primera.

```

1 void tsolve(int *N, int *NRHS, double *A, int *LDA, double *B, int *LDB, int *
  IPIV) {
2     int n=*N; int lda=*LDA; int nrhs=*NRHS; int ldb=*LDB;
3     // Aplicar las permutaciones a B
4     for (int i = 0; i < n; i++) {
5         if (IPIV[i] != i) {
6             for (int j = 0; j < nrhs; j++) {
7                 double temp = B[i + j * ldb];
8                 B[i + j * ldb] = B[IPIV[i] + j * ldb];
9                 B[IPIV[i] + j * ldb] = temp;
10            }
11        }
12    }
13
14    // Sustitución hacia adelante para resolver L * Y = B
15    for (int j = 0; j < nrhs; j++) {
16        for (int i = 0; i < n; i++) {
17            for (int k = 0; k < i; k++) {

```

```
18         B[i + j * ldb] -= A[i + k * lda] * B[k + j * ldb];
19     }
20 }
21 }
22
23 // Sustitución hacia atrás para resolver U * X = Y
24 for (int j = 0; j < nrhs; j++) {
25     for (int i = n - 1; i >= 0; i--) {
26         for (int k = i + 1; k < n; k++) {
27             B[i + j * ldb] -= A[i + k * lda] * B[k + j * ldb];
28         }
29         B[i + j * ldb] /= A[i + i * lda];
30     }
31 }
32 }
```

Listing 3.3: Resolución del sistema triangular

3.1.2. Mmul

Propósito La función `mmul` realiza la multiplicación de matrices genérica con opción de transposición y escalado. Calcula la operación:

$$C = \alpha \cdot \text{op}(A) \cdot \text{op}(B) + \beta \cdot C$$

donde $\text{op}(A)$ y $\text{op}(B)$ pueden ser la matriz original o su transpuesta, dependiendo de los valores de los parámetros `TRANSA` y `TRANSB`.

Parámetros

- `TRANSA`: Puntero a un carácter que indica si la matriz A debe ser transpuesta:
 - `'N'` o `'n'`: No transponer A .
 - `'T'` o `'t'`: Transponer A .
- `TRANSB`: Puntero a un carácter que indica si la matriz B debe ser transpuesta:
 - `'N'` o `'n'`: No transponer B .
 - `'T'` o `'t'`: Transponer B .
- M : Puntero a un entero que representa el número de filas de la matriz C y A (si no está transpuesta).
- N : Puntero a un entero que representa el número de columnas de la matriz C y B (si no está transpuesta).
- K : Puntero a un entero que indica la dimensión común entre A y B para la multiplicación.
- `ALPHA`: Puntero a un escalar multiplicador para $\text{op}(A) \cdot \text{op}(B)$.
- A : Puntero a un arreglo que representa la matriz A en memoria.

- LDA: Puntero a un entero que indica la longitud de la primera dimensión de la matriz A en memoria.
- B: Puntero a un arreglo que representa la matriz B en memoria.
- LDB: Puntero a un entero que indica la longitud de la primera dimensión de la matriz B en memoria.
- BETA: Puntero a un escalar multiplicador para la matriz C .
- C: Puntero a un arreglo que representa la matriz C en memoria. Inicialmente contiene los valores de C , pero al finalizar, contiene el resultado.
- LDC: Puntero a un entero que indica la longitud de la primera dimensión de la matriz C en memoria.

Funcionamiento

1. Configuración inicial:

- Se determinan las opciones de transposición para las matrices A y B según los valores de `TRANSA` y `TRANSB`.
- La variable `trans_a` se establece en 1 si A debe ser transpuesta, y 0 en caso contrario. De manera similar, `trans_b` se establece para B .

2. Escalado de C por β :

- Cada elemento de la matriz C se multiplica por el escalar β . Esto asegura que los valores originales de C estén correctamente ajustados antes de sumar el nuevo resultado.

3. Multiplicación en bloques:

- La multiplicación de matrices se realiza utilizando un enfoque basado en bloques para mejorar la localidad de memoria y aprovechar mejor las cachés del procesador.
- Se define un tamaño de bloque (`BLOCK_SIZE`), ajustable según la arquitectura del hardware.
- Se procesan bloques de las matrices A , B , y C en iteraciones anidadas. Esto reduce los accesos repetidos a la memoria principal.

4. Cálculo de la suma parcial:

- Para cada bloque de $C[i, j]$, se calcula la suma de productos escalares entre las filas de A y las columnas de B , ajustando las operaciones según si A o B están transpuestas.
- El valor resultante se escala por α y se suma al valor actual de $C[i, j]$.

5. Almacenamiento del resultado:

- El resultado final de cada elemento se almacena en la posición correspondiente de la matriz C .

Capítulo 3. Implementacion

```
1 void mmul(char *TRANSA, char *TRANSB, int *M, int *N, int *K,  
2         double *ALPHA, double *A, int *LDA, double *B, int *LDB,  
3         double *BETA, double *C, int *LDC) {  
4  
5     int trans_a = (*TRANSA == 'T' || *TRANSA == 't');  
6     int trans_b = (*TRANSB == 'T' || *TRANSB == 't');  
7  
8     // Escalar matriz C por BETA  
9     for (int j = 0; j < *N; j++) {  
10         for (int i = 0; i < *M; i++) {  
11             C[i + j * (*LDC)] *= *BETA;  
12         }  
13     }  
14  
15     // Bloques para mejorar la localidad de memoria  
16     const int BLOCK_SIZE = 64; // Ajustar según la arquitectura  
17  
18     for (int jj = 0; jj < *N; jj += BLOCK_SIZE) {  
19         for (int ii = 0; ii < *M; ii += BLOCK_SIZE) {  
20             for (int ll = 0; ll < *K; ll += BLOCK_SIZE) {  
21                 for (int j = jj; j < jj + BLOCK_SIZE && j < *N; j++) {  
22                     for (int i = ii; i < ii + BLOCK_SIZE && i < *M; i++) {  
23                         double sum = 0.0;  
24                         for (int l = ll; l < ll + BLOCK_SIZE && l < *K; l++) {  
25                             double a_val = trans_a ? A[l + i * (*LDA)] : A[i +  
26                                 l * (*LDA)];  
27                             double b_val = trans_b ? B[j + l * (*LDB)] : B[l +  
28                                 j * (*LDB)];  
29                             sum += a_val * b_val;  
30                         }  
31                         C[i + j * (*LDC)] += *ALPHA * sum;  
32                     }  
33                 }  
34             }  
35     }
```

Listing 3.4: Implementacion de mmul en C

3.1.3. Mcpy

Propósito La función `mcpy` copia elementos de una matriz A a otra matriz B , permitiendo opciones específicas para copiar la matriz completa o solo su parte triangular superior o inferior. Esto es útil en aplicaciones donde las matrices están organizadas de manera triangular o cuando se necesita replicar una matriz en otra ubicación en memoria.

Parámetros

- **UPLO**: Puntero a un carácter que especifica qué parte de la matriz A será copiada:
 - `'A'` o `'a'`: Copiar toda la matriz.
 - `'U'` o `'u'`: Copiar solo la parte triangular superior ($i \leq j$).
 - `'L'` o `'l'`: Copiar solo la parte triangular inferior ($i \geq j$).
- **M**: Puntero a un entero que indica el número de filas de la matriz A y B .
- **N**: Puntero a un entero que indica el número de columnas de la matriz A y B .
- **A**: Puntero a un arreglo que representa la matriz fuente A .
- **LDA**: Puntero a un entero que indica la longitud de la primera dimensión de la matriz A en memoria.
- **B**: Puntero a un arreglo que representa la matriz destino B .
- **LDB**: Puntero a un entero que indica la longitud de la primera dimensión de la matriz B en memoria.

Funcionamiento

1. Interpretación del parámetro **UPLO**:

- Dependiendo del valor de **UPLO**, la función determina qué parte de la matriz A será copiada a B :
 - `'A'` o `'a'`: Copia todos los elementos de A .
 - `'U'` o `'u'`: Copia únicamente los elementos en o por encima de la diagonal principal ($i \leq j$).
 - `'L'` o `'l'`: Copia únicamente los elementos en o por debajo de la diagonal principal ($i \geq j$).

2. Copia de la matriz completa (**UPLO** = `'A'`):

- Itera sobre cada fila (i) y columna (j) de la matriz A , copiando los valores directamente en B respetando las dimensiones proporcionadas.

3. Copia de la parte triangular superior (**UPLO** = `'U'`):

Capítulo 3. Implementacion

- Para cada columna j , itera únicamente sobre las filas i donde $i \leq j$. Esto asegura que solo se copian los elementos en o por encima de la diagonal principal.

4. Copia de la parte triangular inferior (UPLO = 'L'):

- Para cada columna j , itera únicamente sobre las filas i donde $i \geq j$. Esto asegura que solo se copian los elementos en o por debajo de la diagonal principal.

5. Estructura de memoria:

- Se utiliza un acceso indexado para matrices almacenadas en memoria contigua con dimensiones ajustadas por LDA y LDB.

```
1 void mcpy(char *UPLO, int *M, int *N, double *A, int *LDA, double *B, int *LDB)
2 {
3     int lda=*LDA; int ldb=*LDB;
4     if (*UPLO == 'A' || *UPLO == 'a') { // Copiar toda la matriz
5         for (int j = 0; j < *N; j++) {
6             for (int i = 0; i < *M; i++) {
7                 B[i + j * ldb] = A[i + j * lda];
8             }
9         }
10    } else if (*UPLO == 'U' || *UPLO == 'u') { // Copiar solo la parte
11        triangular superior
12        for (int j = 0; j < *N; j++) {
13            for (int i = 0; i <= j && i < *M; i++) {
14                B[i + j * ldb] = A[i + j * lda];
15            }
16        }
17    } else if (*UPLO == 'L' || *UPLO == 'l') { // Copiar solo la parte
18        triangular inferior
19        for (int j = 0; j < *N; j++) {
20            for (int i = j; i < *M; i++) {
21                B[i + j * ldb] = A[i + j * lda];
22            }
23        }
24    }
25 }
```

Listing 3.5: Implementacion de mcpy en C

3.1.4. Mset

Propósito La función `mset` inicializa los elementos de una matriz A de dimensiones $M \times N$ con valores específicos, permitiendo diferenciar entre la diagonal principal y el resto de los elementos. Además, ofrece la posibilidad de inicializar toda la matriz o restringirse a las partes triangulares superior o inferior.

Parámetros

- UPLO: Puntero a un carácter que especifica qué parte de la matriz se inicializará:
 - 'A' o 'a': Inicializar toda la matriz.

- 'U' o 'u': Inicializar solo la parte triangular superior ($i \leq j$).
- 'L' o 'l': Inicializar solo la parte triangular inferior ($i \geq j$).
- *M*: Puntero a un entero que representa el número de filas de la matriz *A*.
- *N*: Puntero a un entero que representa el número de columnas de la matriz *A*.
- ALPHA: Puntero a un número en punto flotante que define el valor para los elementos fuera de la diagonal principal.
- BETA: Puntero a un número en punto flotante que define el valor para los elementos en la diagonal principal.
- *A*: Puntero a un arreglo que representa la matriz *A* en memoria. Al finalizar la ejecución, contendrá los valores inicializados.
- LDA: Puntero a un entero que indica la longitud de la primera dimensión de la matriz *A* en memoria.

Funcionamiento

1. Configuración del comportamiento según UPLO:

- 'A' o 'a': Todos los elementos de la matriz son inicializados. Los valores en la diagonal se establecen a β , mientras que los demás se establecen a α .
- 'U' o 'u': Solo los elementos en la parte triangular superior ($i \leq j$) se inicializan. Los elementos de la diagonal se establecen a β y los demás a α .
- 'L' o 'l': Solo los elementos en la parte triangular inferior ($i \geq j$) se inicializan. Los elementos de la diagonal se establecen a β y los demás a α .

2. Inicialización de la matriz:

- La función recorre las filas (*i*) y columnas (*j*) de *A* utilizando bucles anidados.
- Dependiendo del valor de UPLO:
 - Se evalúa si el índice (*i, j*) está en la diagonal ($i == j$) para asignar β .
 - Si no está en la diagonal, se asigna el valor α .
- Los índices respetan la disposición de la matriz en memoria, controlada por el valor de LDA.

3. Estructura de acceso:

- Se accede a los elementos de la matriz *A* utilizando la fórmula $A[i + j * lda]$, que considera la disposición columnar de los datos.

```
1 void mset(char *UPLO, int *M, int *N, double *ALPHA, double *BETA, double *A,  
   int *LDA) {  
2     int lda = *LDA;  
3     if (*UPLO == 'A' || *UPLO == 'a') { // Inicializar toda la matriz  
4         for (int j = 0; j < *N; j++) {  
5             for (int i = 0; i < *M; i++) {  
6                 A[i + j * lda] = (i == j) ? *BETA : *ALPHA;  
7             }  
8         }  
9     } else if (*UPLO == 'U' || *UPLO == 'u') { // Inicializar solo la parte  
   triangular superior  
10        for (int j = 0; j < *N; j++) {  
11            for (int i = 0; i <= j && i < *M; i++) {  
12                A[i + j * lda] = (i == j) ? *BETA : *ALPHA;  
13            }  
14        }  
15    } else if (*UPLO == 'L' || *UPLO == 'l') { // Inicializar solo la parte  
   triangular inferior  
16        for (int j = 0; j < *N; j++) {  
17            for (int i = j; i < *M; i++) {  
18                A[i + j * lda] = (i == j) ? *BETA : *ALPHA;  
19            }  
20        }  
21    }  
22 }
```

Listing 3.6: Implementacion de mset en C

3.1.5. Mnorm

Propósito La función `mnorm` calcula distintas normas para una matriz A de dimensiones $M \times N$. Las normas permiten medir diferentes propiedades de la matriz y son fundamentales en análisis numérico y álgebra lineal.

Parámetros

- **NORM:** Puntero a un carácter que especifica el tipo de norma a calcular:
 - 'M' o 'm': Norma máximo absoluto (valor máximo absoluto en la matriz).
 - '1' o 'O' o 'o': Norma uno (suma máxima de los valores absolutos en las columnas).
 - 'I' o 'i': Norma infinito (suma máxima de los valores absolutos en las filas).
 - 'F' o 'f' o 'E' o 'e': Norma de Frobenius (raíz cuadrada de la suma de los cuadrados de todos los elementos).
- **M:** Puntero a un entero que indica el número de filas de la matriz A .
- **N:** Puntero a un entero que indica el número de columnas de la matriz A .
- **A:** Puntero a un arreglo que representa la matriz A en memoria.

- **LDA:** Puntero a un entero que indica la longitud de la primera dimensión de la matriz A en memoria.
- **WORK:** Puntero a un arreglo de trabajo utilizado para calcular la norma infinito. Debe ser al menos de tamaño M .

Funcionamiento

1. Norma máximo absoluto (NORM = 'M'):

- Itera sobre todos los elementos de A , calcula su valor absoluto, y guarda el máximo encontrado.
- Esta norma mide el elemento con mayor magnitud en la matriz.

2. Norma uno (NORM = '1'):

- Calcula la suma de los valores absolutos de cada columna.
- Encuentra y devuelve la suma máxima entre todas las columnas.
- Representa el mayor "peso"columnar de la matriz.

3. Norma infinito (NORM = 'I'):

- Inicializa el espacio de trabajo **WORK** a cero.
- Para cada fila, acumula la suma de los valores absolutos de los elementos en esa fila.
- Encuentra y devuelve la suma máxima entre todas las filas.
- Representa el mayor "peso"fila de la matriz.

4. Norma de Frobenius (NORM = 'F' o NORM = 'E'):

- Calcula la suma de los cuadrados de todos los elementos en la matriz.
- Devuelve la raíz cuadrada de esta suma.
- Mide la magnitud global de la matriz.

5. Resultado:

- La función retorna el valor de la norma calculada.

Capítulo 3. Implementacion

```
1 double mnorm(char *NORM, int *M, int *N, double *A, int *LDA, double *WORK) {
2     double norm = 0.0;
3     // Norma Máximo Absoluto ('M')
4     if (*NORM == 'M' || *NORM == 'm') {
5         for (int j = 0; j < *N; j++) {
6             for (int i = 0; i < *M; i++) {
7                 double abs_val = fabs(A[i + j * (*LDA)]);
8                 if (abs_val > norm) {
9                     norm = abs_val;
10                }
11            }
12        }
13    }
14    // Norma Uno ('1')
15    else if (*NORM == '1' || *NORM == 'O' || *NORM == 'o') {
16        for (int j = 0; j < *N; j++) {
17            double col_sum = 0.0;
18            for (int i = 0; i < *M; i++) {
19                col_sum += fabs(A[i + j * (*LDA)]);
20            }
21            if (col_sum > norm) {
22                norm = col_sum;
23            }
24        }
25    }
26    // Norma Infinito ('I')
27    else if (*NORM == 'I' || *NORM == 'i') {
28        for (int i = 0; i < *M; i++) {
29            WORK[i] = 0.0; // Inicializar el espacio de trabajo
30        }
31        for (int j = 0; j < *N; j++) {
32            for (int i = 0; i < *M; i++) {
33                WORK[i] += fabs(A[i + j * (*LDA)]);
34            }
35        }
36        // Encontrar el valor máximo en WORK (la suma máxima de filas)
37        for (int i = 0; i < *M; i++) {
38            if (WORK[i] > norm) {
39                norm = WORK[i];
40            }
41        }
42    }
43    // Norma Frobenius ('F' o 'E')
44    else if (*NORM == 'F' || *NORM == 'f' || *NORM == 'E' || *NORM == 'e') {
45        double sum = 0.0; // Para acumular la suma de cuadrados
46        for (int j = 0; j < *N; j++) {
47            for (int i = 0; i < *M; i++) {
48                double val = A[i + j * (*LDA)];
49                sum += val * val; // Acumula el cuadrado de cada elemento
50            }
51        }
52        norm = sqrt(sum); // Calcula la raíz cuadrada de la suma
53    }
54    return norm; }
```

Listing 3.7: Implementacion de mnorm en C

3.1.6. Mmulc

Propósito La función `mmulc` realiza una operación de escalado elemento a elemento sobre una matriz cuadrada A de tamaño $t \times t$, multiplicando cada elemento por un escalar c y almacenando el resultado en otra matriz D . Este tipo de operación es útil en transformaciones lineales, normalización o ajustes de magnitud de una matriz.

Parámetros

- D : Puntero a un arreglo de números en punto flotante donde se almacenará el resultado. Debe tener capacidad para al menos $t \times t$ elementos.
- t : Entero que indica el tamaño de la matriz cuadrada $t \times t$.
- A : Puntero a un arreglo de números en punto flotante que representa la matriz fuente A .
- c : Escalar en punto flotante que se utiliza para multiplicar cada elemento de la matriz A .

Funcionamiento

1. Recorrido de la matriz:

- Se utiliza un bucle anidado para recorrer los elementos de la matriz A .
- El bucle externo itera sobre las filas i , mientras que el bucle interno itera sobre las columnas j .

2. Cálculo del índice:

- Dado que las matrices están almacenadas en una disposición lineal en memoria (formato contiguo por filas), el índice para un elemento (i, j) se calcula como:

$$\text{índice} = i \times t + j$$

- Este índice se utiliza tanto para acceder a los elementos de A como para asignar los valores en D .

3. Operación de escalado:

- Para cada elemento de A , se multiplica por el escalar c y el resultado se almacena en la posición correspondiente de D .

4. Almacenamiento del resultado:

- Los resultados de la operación se almacenan directamente en la matriz D , preservando el formato cuadrado y la disposición de filas y columnas.

```

1 void mmulc(double *D, int t, double *A, double c) {
2     for (int i = 0; i < t; i++) {
3         for (int j = 0; j < t; j++) {
4             D[(i*t)+j] = c*A[(i*t)+j]; } } }
```

Listing 3.8: Implementación de `mmulc` en C

3.1.7. Msub

Propósito La función `msub` calcula la diferencia elemento a elemento entre dos matrices cuadradas A y B de tamaño $t \times t$. El resultado de la operación $D = A - B$ se almacena en otra matriz D . Este tipo de operación es fundamental en álgebra matricial y tiene aplicaciones en análisis numérico, gráficos y resolución de sistemas de ecuaciones.

Parámetros

- D : Puntero a un arreglo de números en punto flotante donde se almacenará el resultado de la operación. Debe tener capacidad para al menos $t \times t$ elementos.
- t : Entero que indica el tamaño de las matrices cuadradas $t \times t$.
- A : Puntero a un arreglo de números en punto flotante que representa la matriz minuendo A .
- B : Puntero a un arreglo de números en punto flotante que representa la matriz sustraendo B .

Funcionamiento

1. Recorrido de las matrices:

- La función utiliza dos bucles anidados:
 - El bucle externo itera sobre las filas i de las matrices.
 - El bucle interno itera sobre las columnas j de las matrices.

2. Cálculo del índice:

- Dado que las matrices están almacenadas en un formato lineal en memoria (por filas), el índice para el elemento (i, j) se calcula como:

$$\text{índice} = i \times t + j$$

- Este índice se utiliza para acceder a los elementos correspondientes de A , B , y para almacenar el resultado en D .

3. Cálculo de la diferencia:

- Para cada elemento (i, j) , se realiza la operación:

$$D[i, j] = A[i, j] - B[i, j]$$

- El resultado se asigna directamente en la posición correspondiente de D .

4. Almacenamiento del resultado:

- La matriz D se completa con los resultados de la resta, preservando el formato cuadrado y la disposición fila-columna.

3.2. Multi-Processed Matrix Subroutines

```
1 void msub(double *D, int t, double *A, double *B) {
2     for (int i = 0; i < t; i++) {
3         for (int j = 0; j < t; j++) {
4             D[(i*t)+j]=A[(i*t)+j]-B[(i*t)+j];
5         }
6     }
7 }
```

Listing 3.9: Implementacion de msub en C

3.2. Multi-Processed Matrix Subroutines

Una vez se tiene ya la librería Matrix Subroutines funcionando de manera óptima, se crea una nueva con el fin de implementar en ella el procesamiento paralelo. La nueva librería se llama "Multi-Processed Matrix Subroutines"(MPMS) y consiste en la unión de las funciones de MS junto con directivas de OpenMP que permitan procesar las funciones de forma paralela.

Durante su implementación, se ha buscado siempre la configuración más óptima, que permita los menores tiempos de ejecución con matrices de gran escala. Para ello, se ha intentado balancear la carga lo máximo posible entre los núcleos disponibles y aprovechar así todo lo posible las partes paralelizables de las funciones. A continuación, se mostrará cómo se han paralelizado las funciones de esta librería y se comentarán brevemente las decisiones que se han tomado sobre la paralelización.

3.2.1. SolveLS

Para comenzar, la función `solveLS`, presente en el Listing 3.10, no sufrirá ningún cambio respecto a MS. Esto se debe a que, principalmente, son las funciones auxiliares que esta función llama las que realmente desempeñan su cometido, por lo que, para conseguir que `solveLS` se procese en paralelo, se paralelizarán `mlufact` y `tsolve`.

```
1 void solveLS(int *N, int *NRHS, double *A, int *LDA, int *IPIV, double *B, int
   *LDB, int *INFO) {
2     // Factorización LU con pivoteo parcial
3     mlufact(N, A, LDA, IPIV, INFO);
4
5     // Si hubo un error en la factorización, salir
6     if (*INFO != 0) {
7         return;
8     }
9
10    // Resolver el sistema triangular usando la factorización LU
11    tsolve(N, NRHS, A, LDA, B, LDB, IPIV);
12 }
```

Listing 3.10: Implementacion de solveLS en C con procesamiento paralelo (OpenMP)

Mlufact

Para lograr un procesamiento paralelo eficiente en `mlufact` (Listing 3.11) se ha paralelizado su bucle principal, el cual itera sobre las filas de la matriz A, utilizando la directiva de OpenMP `#pragma omp parallel for schedule(dynamic)`. De esta forma se aprovecharán los múltiples núcleos del procesador, dividiendo entre estos las iteraciones del bucle que realiza operaciones independientes como la búsqueda de pivotes, el intercambio de filas y la factorización LU.

Si se distribuyesen las iteraciones a partes iguales entre los núcleos, se comprobaría que la carga de trabajo no se distribuye de igual manera. Esto se debe a que las iteraciones del bucle tienen distinta complejidad computacional, disminuyendo esta a medida que avanzan las iteraciones. Por ello, la cláusula `schedule(dynamic)` es la manera más óptima, ya que permite distribuir las iteraciones entre los hilos de manera dinámica, asignando nuevas iteraciones a los núcleos a medida que terminan con las anteriores. Así se conseguirá un mayor balance de la carga, evitando que algunos hilos queden inactivos mientras otros procesan iteraciones más complejas, lo cual se traduce en mejor rendimiento y tiempos de ejecución de la función.

Como opción alternativa, se consideró paralelizar los bucles internos en sustitución del externo; sin embargo, esto no es posible ya que presentan dependencias entre cálculos en cada iteración.

```
1 void mlufact(int *N, double *A, int *LDA, int *IPIV, int *INFO) {
2     *INFO = 0;
3     int n=*N; int lda=*LDA;
4     #pragma omp parallel for schedule(dynamic)
5     for (int i = 0; i < n; i++) {
6         // Buscar el pivote máximo en la columna actual
7         int max_index = i;
8         double max_value = fabs(A[i + i * lda]);
9         for (int k = i + 1; k < n; k++) {
10             double abs_val = fabs(A[k + i * lda]);
11             if (abs_val > max_value) {
12                 max_value = abs_val;
13                 max_index = k; } }
14         // Verificar si el pivote es cero (matriz singular)
15         if (max_value == 0.0) *INFO = i + 1;
16         // Guardar el índice de pivote
17         IPIV[i] = max_index;
18         // Intercambiar filas si es necesario
19         if (max_index != i) {
20             for (int j = 0; j < n; j++) {
21                 double temp = A[i + j * lda];
22                 A[i + j * lda] = A[max_index + j * lda];
23                 A[max_index + j * lda] = temp; } }
24         // Factorización LU
25         for (int j = i + 1; j < n; j++) {
26             A[j + i * lda] /= A[i + i * lda];
27             for (int k = i + 1; k < n; k++) {
28                 A[j + k * lda] -= A[j + i * lda] * A[i + k * lda]; } } }
```

Listing 3.11: Factorización LU con pivoteo parcial con procesamiento paralelo (OpenMP)

Tsolve

Para la paralelización de la función `tsolve` (Listing 3.12), se han paralelizado tres secciones principales del código: los tres bucles externos que se realizan en este.

Primero, mediante la directiva `#pragma omp parallel for` se paralelizará el bucle que aplica las permutaciones a la matriz de A. Cada hilo puede procesar de forma independiente diferentes filas de B ya que no existen dependencias entre las iteraciones, mejorando así la eficiencia.

A continuación, se utiliza también la directiva `#pragma omp parallel for` para paralelizar la sección encargada de realizar la sustitución hacia adelante para resolver $L \cdot Y = B$. Aunque este cálculo haga uso de bucles anidados, las iteraciones sobre las columnas del bucle externo son independientes, permitiendo que cada hilo pueda procesar columnas diferentes.

Por último, para la sección que realiza la sustitución hacia atrás que resuelve $U \cdot X = Y$ se ha paralelizado mediante `#pragma omp parallel for collapse(2)`. Mediante la cláusula `collapse(2)` se consigue combinar el bucle externo (sobre las columnas) junto con uno interno (sobre las filas) permitiendo una mejor distribución del trabajo entre los hilos. Esta cláusula resulta particularmente eficiente cuando ambos bucles tienen un gran número de iteraciones.

```

1 void tsolve(int *N, int *NRHS, double *A, int *LDA, double *B, int *LDB, int *
  IPIV) {
2   int n=*N; int lda=*LDA; int nrhs=*NRHS; int ldb=*LDB;
3   // Aplicar las permutaciones a B
4   #pragma omp parallel for
5   for (int i = 0; i < n; i++) {
6     if (IPIV[i] != i) {
7       for (int j = 0; j < nrhs; j++) {
8         double temp = B[i + j * ldb];
9         B[i + j * ldb] = B[IPIV[i] + j * ldb];
10        B[IPIV[i] + j * ldb] = temp;
11      } } }
12   // Sustitución hacia adelante para resolver L * Y = B
13   #pragma omp parallel for
14   for (int j = 0; j < nrhs; j++) {
15     for (int i = 0; i < n; i++) {
16       for (int k = 0; k < i; k++) {
17         B[i + j * ldb] -= A[i + k * lda] * B[k + j * ldb];
18       } } }
19   // Sustitución hacia atrás para resolver U * X = Y
20   #pragma omp parallel for collapse(2)
21   for (int j = 0; j < nrhs; j++) {
22     for (int i = n - 1; i >= 0; i--) {
23       for (int k = i + 1; k < n; k++) {
24         B[i + j * ldb] -= A[i + k * lda] * B[k + j * ldb];
25       }
26       B[i + j * ldb] /= A[i + i * lda]; } } }

```

Listing 3.12: Resolución del sistema triangular con procesamiento paralelo (OpenMP)

3.2.2. Mmul

En cuanto a la paralelización de la función `mmul`, presente en el Listing 3.11, se ha paralelizado uniendo los tres bucles principales, responsables del recorrido por bloques, mediante la directiva `#pragma omp parallel for collapse(3)`. Esta estrategia permite que los bloques de matrices A y B se procesen simultáneamente en diferentes hilos, asegurando una mayor granularidad en la división del trabajo.

Por otro lado, aunque no forme parte de la paralelización, se quiere destacar el uso de bloques de tamaño fijo (`BLOCK_SIZE`), ya que estos contribuyen a mejorar la localidad de la memoria, minimizando los fallos en la caché al limitar los accesos a submatrices más pequeñas que caben en los niveles superiores de la memoria caché. Como resultado, se obtiene una mayor optimización en arquitecturas modernas, donde la jerarquía de la memoria es un aspecto crucial para cálculos complejos.

```
1 void mmul(char *TRANSA, char *TRANSB, int *M, int *N, int *K,  
2         double *ALPHA, double *A, int *LDA, double *B, int *LDB,  
3         double *BETA, double *C, int *LDC) {  
4  
5     int trans_a = (*TRANSA == 'T' || *TRANSA == 't');  
6     int trans_b = (*TRANSB == 'T' || *TRANSB == 't');  
7  
8     // Escalar matriz C por BETA  
9     for (int j = 0; j < *N; j++) {  
10         for (int i = 0; i < *M; i++) {  
11             C[i + j * (*LDC)] *= *BETA;  
12         }  
13     }  
14  
15     // Bloques para mejorar la localidad de memoria  
16     const int BLOCK_SIZE = 64; // Ajustar según la arquitectura  
17     #pragma omp parallel for collapse(3)  
18     for (int jj = 0; jj < *N; jj += BLOCK_SIZE) {  
19         for (int ii = 0; ii < *M; ii += BLOCK_SIZE) {  
20             for (int ll = 0; ll < *K; ll += BLOCK_SIZE) {  
21                 for (int j = jj; j < jj + BLOCK_SIZE && j < *N; j++) {  
22                     for (int i = ii; i < ii + BLOCK_SIZE && i < *M; i++) {  
23                         double sum = 0.0;  
24                         for (int l = ll; l < ll + BLOCK_SIZE && l < *K; l++) {  
25                             double a_val = trans_a ? A[l + i * (*LDA)] : A[i +  
26                                 l * (*LDA)];  
27                             double b_val = trans_b ? B[j + l * (*LDB)] : B[l +  
28                                 j * (*LDB)];  
29                             sum += a_val * b_val;  
30                         }  
31                         C[i + j * (*LDC)] += *ALPHA * sum;  
32                     }  
33                 }  
34             }  
35         }  
36     }  
37 }
```

Listing 3.13: Implementacion de `mmmul` en C con procesamiento paralelo (OpenMP)

3.2.3. Mcpy

Para paralelizar `mcpy`, representada en el Listing 3.14, se paralelizará cada uno de los casos en los que se puede llamar a esta función. En primer lugar, para el caso de copiar toda la matriz se utilizará la directiva `#pragma omp parallel for collapse(2)` para unir tanto el bucle que recorre las filas como el de las columnas, consiguiendo una mayor granularidad del trabajo. Por otro lado, ambos casos de copiar una parte triangular de la matriz se paralelizan con directivas `#pragma omp parallel for`. En ambos casos no es posible combinar ambos bucles mediante la cláusula `collapse(2)` por la forma en la que están contruidos estos.

```

1 void mcpy(char *UPLO, int *M, int *N, double *A, int *LDA, double *B, int *LDB)
2 {
3     int lda=*LDA;
4     int ldb=*LDB;
5     if (*UPLO == 'A' || *UPLO == 'a') { // Copiar toda la matriz
6         #pragma omp parallel for collapse(2)
7         for (int j = 0; j < *N; j++) {
8             for (int i = 0; i < *M; i++) {
9                 B[i + j * ldb] = A[i + j * lda];
10            }
11        }
12    } else if (*UPLO == 'U' || *UPLO == 'u') { // Copiar solo la parte
13        triangular superior
14        #pragma omp parallel for
15        for (int j = 0; j < *N; j++) {
16            for (int i = 0; i <= j && i < *M; i++) { // Solo copiar si i <= j
17                B[i + j * ldb] = A[i + j * lda];
18            }
19        }
20    } else if (*UPLO == 'L' || *UPLO == 'l') { // Copiar solo la parte
21        triangular inferior
22        #pragma omp parallel for
23        for (int j = 0; j < *N; j++) {
24            for (int i = j; i < *M; i++) { // Solo copiar si i >= j
25                B[i + j * ldb] = A[i + j * lda];
26            }
27        }
28    }
29 }

```

Listing 3.14: Implementacion de `mcpy` en C con procesamiento paralelo (OpenMP)

3.2.4. Mset

En el caso de la función `mset` (Listing 3.15), su paralelización es muy similar a la de `mcpy`, ya que se tendrá que paralelizar las diferentes partes de la función por separado debido a que cada una corresponde a un tipo caso de uso de la función. En primer lugar, en la sección encargada de inicializar toda la matriz se utiliza la directiva `#pragma omp parallel for collapse(2)` para unir tanto el bucle que recorre las filas como el de las columnas, consiguiendo una distribución más fina del trabajo entre los hilos. Por otro lado, ambos casos de inicializar una parte triangular de la matriz se paralelizan con directivas `#pragma omp parallel for`. En ambos casos no es posible combinar ambos bucles mediante la cláusula `collapse(2)` por la forma en la que están contruidos estos.

```
1 void mset(char *UPLO, int *M, int *N, double *ALPHA, double *BETA, double *A,
2         int *LDA) {
3     if (*UPLO == 'A' || *UPLO == 'a') { // Inicializar toda la matriz
4         #pragma omp parallel for collapse(2)
5         for (int j = 0; j < *N; j++) {
6             for (int i = 0; i < *M; i++) {
7                 if (i == j) {
8                     A[i + j * (*LDA)] = *BETA; // Diagonal
9                 } else {
10                    A[i + j * (*LDA)] = *ALPHA; // Fuera de la diagonal
11                }
12            }
13        }
14    } else if (*UPLO == 'U' || *UPLO == 'u') { // Inicializar solo la parte
15        triangular superior
16        #pragma omp parallel for
17        for (int j = 0; j < *N; j++) {
18            for (int i = 0; i <= j && i < *M; i++) {
19                if (i == j) {
20                    A[i + j * (*LDA)] = *BETA; // Diagonal
21                } else {
22                    A[i + j * (*LDA)] = *ALPHA; // Fuera de la diagonal
23                }
24            }
25        }
26    } else if (*UPLO == 'L' || *UPLO == 'l') { // Inicializar solo la parte
27        triangular inferior
28        #pragma omp parallel for
29        for (int j = 0; j < *N; j++) {
30            for (int i = j; i < *M; i++) {
31                if (i == j) {
32                    A[i + j * (*LDA)] = *BETA; // Diagonal
33                } else {
34                    A[i + j * (*LDA)] = *ALPHA; // Fuera de la diagonal
35                }
36            }
37        }
38    }
39 }
```

Listing 3.15: Implementacion de `mset` en C con procesamiento paralelo (OpenMP)

3.2.5. Mnorm

Para la función `mnorm` se ha paralelizado el cálculo de cada una de las normas por separado:

- **Norma Máximo Absoluto ('M'):** se paralelizó el doble bucle que recorre los elementos de la matriz con `#pragma omp parallel for collapse(2)`. Esto combina los bucles anidados en un solo rango de iteraciones.
- **Norma Uno ('1'):** Las iteraciones del bucle externo que itera sobre las columnas de la matriz son independientes entre sí, por lo que se paralelizará utilizando `#pragma omp parallel for`, distribuyendo eficientemente el trabajo entre los hilos.
- **Norma Infinito ('I'):** se paraleliza la acumulación de valores en el array de trabajo `WORK` utilizando `#pragma omp parallel for collapse(2)`, ya que las operaciones son independientes para cada elemento.
- **Norma Frobenius ('F'):** se utilizó la directiva `#pragma omp parallel for collapse(2) reduction(+:sum)`. Este enfoque asegura que la acumulación de los cuadrados de los elementos se realice de manera segura y eficiente, sumando los resultados parciales mediante la cláusula `reduction`.

Su paralelización se puede apreciar al completo en el Listing 3.16.

```

1 double mnorm(char *NORM, int *M, int *N, double *A, int *LDA, double *WORK) {
2     double norm = 0.0;
3
4     // Norma Máximo Absoluto ('M')
5     if (*NORM == 'M' || *NORM == 'm') {
6         #pragma omp parallel for collapse(2)
7         for (int j = 0; j < *N; j++) {
8             for (int i = 0; i < *M; i++) {
9                 double abs_val = fabs(A[i + j * (*LDA)]);
10                if (abs_val > norm) {
11                    norm = abs_val;
12                }
13            }
14        }
15    }
16    // Norma Uno ('1')
17    else if (*NORM == '1' || *NORM == 'O' || *NORM == 'o') {
18        #pragma omp parallel for
19        for (int j = 0; j < *N; j++) {
20            double col_sum = 0.0;
21            for (int i = 0; i < *M; i++) {
22                col_sum += fabs(A[i + j * (*LDA)]);
23            }
24            if (col_sum > norm) {
25                norm = col_sum;
26            }
27        }
28    }
29    // Norma Infinito ('I')
30    else if (*NORM == 'I' || *NORM == 'i') {
31        for (int i = 0; i < *M; i++) {
32            WORK[i] = 0.0; // Inicializar el espacio de trabajo

```

Capítulo 3. Implementacion

```
33     }
34     #pragma omp parallel for collapse(2)
35     for (int j = 0; j < *N; j++) {
36         for (int i = 0; i < *M; i++) {
37             WORK[i] += fabs(A[i + j * (*LDA)]);
38         }
39     }
40     // Encontrar el valor máximo en WORK (la suma máxima de filas)
41     for (int i = 0; i < *M; i++) {
42         if (WORK[i] > norm) {
43             norm = WORK[i];
44         }
45     }
46 }
47 // Norma Frobenius ('F' o 'E')
48 else if (*NORM == 'F' || *NORM == 'f' || *NORM == 'E' || *NORM == 'e') {
49     double sum = 0.0; // Para acumular la suma de cuadrados
50     #pragma omp parallel for collapse(2) reduction(+:sum)
51     for (int j = 0; j < *N; j++) {
52         for (int i = 0; i < *M; i++) {
53             double val = A[i + j * (*LDA)];
54             sum += val * val; // Acumula el cuadrado de cada elemento
55         }
56     }
57     norm = sqrt(sum); // Calcula la raíz cuadrada de la suma
58 }
59
60 return norm;
61 }
```

Listing 3.16: Implementacion de `mnorm` en C con procesamiento paralelo (OpenMP)

3.2.6. Mmulc

La paralelización de la función `mmulc`, mostrada en el Listing 3.17, resulta bastante sencilla, ya que únicamente consiste en la paralelización del bucle que recorre las filas mediante la directiva `#pragma omp parallel for`. A la hora de paralelizar esta función, podría parecer que el uso de la cláusula `collapse(2)` mejoraría el rendimiento; sin embargo, tras realizar algunas pruebas, se ha concluido que es más eficiente sin ella.

```
1 void mmulc(double *D, int t, double *A, double c){
2     #pragma omp parallel for
3     for (int i=0; i<t; i++){
4         for (int j=0; j<t; j++){
5             D[(i*t)+j]=c*A[(i*t)+j];
6         }
7     }
8 }
```

Listing 3.17: Implementacion de `mmulc` en C con procesamiento paralelo (OpenMP)

3.2.7. Msub

Por último, la paralelización de la función `msub`, presente en el Listing 3.18, es muy similar a la de la función `mmulc`, ya que ambas funciones realizan tareas similares para las cuales hace falta recorrer matrices mediante dos bucles. Se ha utilizado de nuevo la directiva `#pragma omp parallel for` para paralelizar el bucle exterior encargado de recorrer las filas.

```
1 void msub(double *D, int t, double *A, double *B){
2     #pragma omp parallel for
3     for (int i = 0; i < t; i++){
4         for (int j = 0; j < t; j++){
5             D[(i*t)+j]=A[(i*t)+j]-B[(i*t)+j];
6         }
7     }
8 }
```

Listing 3.18: Implementacion de `msub` en C con procesamiento paralelo (OpenMP)

Capítulo 4

Experimentación y Resultados

En el presente capítulo se van a realizar una serie de pruebas para comparar el funcionamiento tanto de MS como de MPMS, además del tiempo de ejecución del algoritmo utilizando cada una de estas bibliotecas. Para ello, se ha desarrollado una serie de programas y scripts que se encargan de lanzar los diferentes tests y almacenar sus resultados. Sin embargo, en este apartado se centrará en sus resultados en distintos entornos y no en cómo están programadas o diseñadas las pruebas, por lo que todas estas serán tratadas en mayor profundidad durante el primer anexo. Por el momento, lo esencial es saber que se van a probar tanto las diferentes funciones de las librerías como el algoritmo utilizando ambas con múltiples tamaños de problema (N). Todas las pruebas se llevarán a cabo en diferentes entornos de experimentación, permitiéndonos comparar su funcionamiento en diferentes arquitecturas y evaluar de mejor forma los resultados obtenidos.

A continuación, se van a mostrar todas las pruebas y test realizados en los cuatro entornos de experimentación. Pero antes, es necesario aclarar algunos conceptos o mostrar el funcionamiento de algunas tecnologías para poder analizar y comprender mejor tanto las pruebas realizadas como sus resultados obtenidos. En concreto, se tratará el mecanismo de Simultaneous Multi-Threading (SMT) que permite "duplicar" los núcleos del procesador, el funcionamiento de las memorias caché durante los accesos a memoria y los conceptos de ganancia y eficiencia.

Simultaneous Multi-Threading (SMT)

SMT es un mecanismo que se basa en la capacidad del procesador para compartir sus recursos internos entre múltiples hilos, como la ALU y las cachés. Cada núcleo físico se divide entre varios núcleos lógicos que comparten los recursos del núcleo, cada uno con su propio conjunto de registros y su estado de contexto. De esta forma, si un hilo está esperando el acceso a memoria o a otra operación, otro hilo puede utilizar las unidades disponibles, minimizando los tiempos de inactividad del procesador.

Para simplificar, SMT busca reducir los ciclos inactivos del procesador provo-

cados por latencias de memoria o dependencias en el flujo de instrucciones, mediante el intercalado de instrucciones de diferentes hilos en un único núcleo físico. Este enfoque no pretende duplicar los recursos completos del núcleo, sino que optimiza el uso de los existentes.

El concepto de SMT quizás pueda parecer muy similar al de paralelismo a nivel de instrucción (Instruction-Level Parallelism, ILP). Esto se debe a que ILP y SMT son conceptos relacionados en el contexto de la arquitectura de procesadores, ya que ambos buscan aumentar el rendimiento del hardware al ejecutar múltiples tareas de forma concurrente; sin embargo, lo hacen en niveles diferentes y con enfoques complementarios.

Como ilustra la Figura 4.1, ILP ejecuta múltiples instrucciones independientes al mismo tiempo en un único núcleo con el fin de extraer paralelismo dentro de un único hilo, mientras que, como se ilustra en la Figura 4.2, SMT permite que varios hilos lógicos se ejecuten simultáneamente en un mismo núcleo físico, aprovechando los recursos del núcleo cuando ILP no puede extraer suficiente paralelismo (saturación ILP).

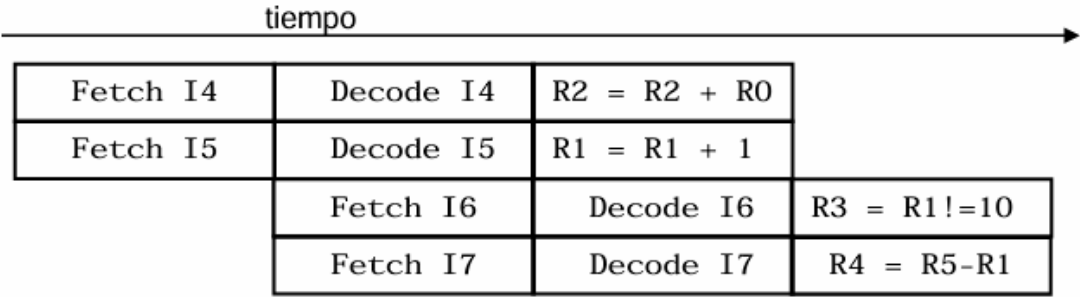


Figura 4.1: Representación pipeline de instrucciones con ILP [6]

ILP maximiza el uso de las unidades del núcleo a nivel de hilo, mientras que SMT mejora aún más la utilización al introducir varios hilos lógicos en un solo núcleo físico. Por ello, SMT se beneficia del ILP, porque un núcleo con alta capacidad de paralelismo a nivel de instrucciones (como múltiples ALUs, FPUs y pipelines) tiene más recursos disponibles para compartir entre hilos lógicos.

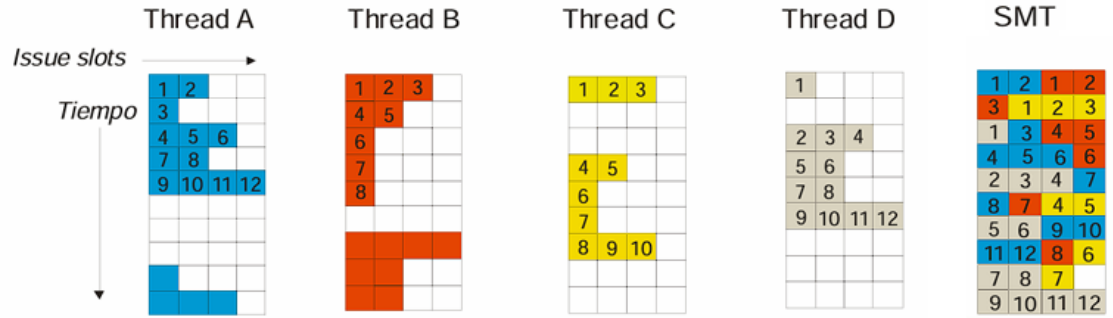


Figura 4.2: Ejemplo de SMT [6]

En resumen, este mecanismo puede conseguir maximizar el rendimiento al permitir que cada núcleo físico maneje múltiples hilos simultáneamente. Sin embargo, su efectividad depende de la propia carga de trabajo y de cómo se gestionan los recursos compartidos dentro del núcleo. Pueden resultar escenarios bien paralelizables y con baja contención en los que ofrezca un notable aumento en el rendimiento, mientras que en otras aplicaciones altamente demandantes de recursos, el beneficio puede ser limitado o incluso perjudicial.

Memoria Caché

Todos los procesadores contienen una pequeña sección de memoria muy rápida llamada registros; sin embargo, esta no es suficiente para albergar todos los datos que necesitan, por lo que es necesario acceder a la memoria principal frecuentemente. El problema aquí es que la velocidad de la memoria principal no puede seguir el ritmo al procesador y los accesos a esta son muy lentos en comparación. Esto crea la necesidad de una memoria intermedia mucho más rápida para contener datos recientes o que se acceden frecuentemente: la memoria caché.

La memoria cache surge con el propósito de reducir los tiempos de acceso a los datos más frecuentemente utilizados, optimizando el rendimiento del sistema; para ello, almacena copias de los datos e instrucciones más utilizados por el procesador. Se organiza en niveles jerárquicos, los cuales permiten que el procesador acceda a los datos de manera eficiente, minimizando las costosas operaciones de lectura y escritura en la RAM.

El funcionamiento básico de una memoria caché se puede describir en los siguientes pasos:

1. **Consulta en la caché:** Cuando el procesador necesita un dato, primero consulta en la memoria caché si este se encuentra almacenado. Este proceso se conoce como una *búsqueda en la caché*.
2. **Acierto de caché (cache hit):** Si el dato está en la caché, se denomina "acierto de caché". En este caso, el procesador obtiene el dato directamente de la caché, lo que reduce significativamente el tiempo de acceso.
3. **Fallo de caché (cache miss):** Si el dato no se encuentra en la caché, se produce un "fallo de caché". En este caso:
 - El procesador accede a la memoria principal para obtener el dato.
 - Una copia del dato se almacena en la caché para futuras consultas, siguiendo ciertas *políticas de reemplazo* para determinar qué datos deben mantenerse o eliminarse de la caché.

La memoria caché puede ser determinante en la ejecución de programas paralelos, principalmente por su arquitectura y la política de reemplazo que sigue a la hora de desalojar datos de ella. Dependiendo de estos dos factores, dos hilos que trabajan en conjunto pueden experimentar un fenómeno conocido como *cache thrashing*.

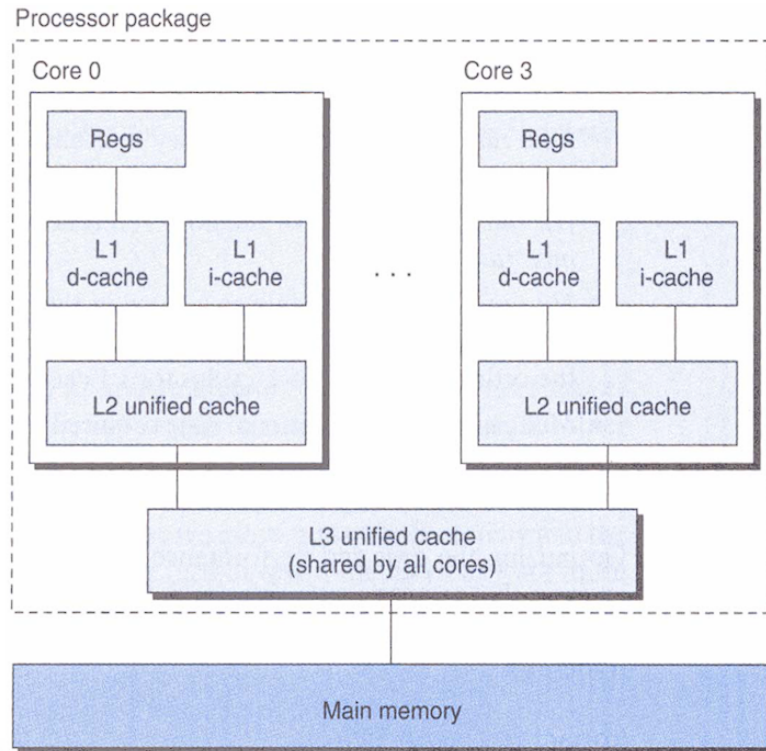


Figura 4.3: Ejemplo memoria caché Intel Core i7-6700 (Skylake) [7]

El *cache thrashing* se produce cuando dos hilos que acceden frecuentemente a conjuntos de datos de memoria diferentes y ocupan la misma sección en la memoria caché, se reemplazan mutuamente los datos del otro de forma repetida. Esto fuerza a que se realicen múltiples accesos a la memoria principal que degradan en gran medida el rendimiento, por lo que cobra especial relevancia el diseño de programas que minimicen los accesos conflictivos a la caché.

Ganancia (SpeedUP) y Eficiencia

A la hora de analizar los resultados de las diferentes pruebas, se van a utilizar dos indicadores clave para juzgar si estos han obtenido resultados más o menos favorables. Estos dos conceptos son la *ganancia (SpeedUP)* y la *eficiencia*.

Empezando por la ganancia, es una métrica utilizada en sistemas paralelos para medir cuánto se ha acelerado un programa al ejecutarse con varios hilos o procesadores en comparación con su ejecución secuencial. Es un indicador de la mejora en el rendimiento gracias a la paralelización. Para calcularla, se divide el tiempo secuencial entre el tiempo paralelo de la siguiente forma:

$$SpeedUp = \frac{T_s}{T_p}$$

Por otro lado, la eficiencia mide qué tan bien se utilizan los recursos disponibles (hilos o procesadores) en una implementación paralela. Es la relación entre la ganancia obtenida y el número de hilos/procesadores utilizados.

$$Eficiencia = \frac{SpeedUp}{NHilos}$$

Una ganancia alta indica que el programa se ha acelerado significativamente gracias a la paralelización. Por otro lado, una eficiencia cercana a 1 indica un uso óptimo de los recursos, mientras que valores bajos reflejan problemas como sobrecarga, comunicación excesiva o poca paralelización efectiva. Ambas métricas serán cruciales para evaluar el rendimiento del algoritmo paralelizado.

4.1. Entorno 1

El primer entorno consta de un procesador AMD Ryzen 7 PRO 8840HS de versión portátil con 8 núcleos a 3.3GHz. A continuación, se van a mostrar su rendimiento en las pruebas de las diferentes funciones de la librería. Todas estas pruebas se realizarán utilizando diferentes números de hilos con el fin de explorar las múltiples opciones y encontrar la más óptima. En concreto, se utilizarán 16, 32, 8 y 4 hilos. Estos números no son al azar, sino que parten de los 8 núcleos físicos que tiene el procesador, que a su vez se convierten en 16 (8 núcleos físicos y 8 lógicos) mediante Simultaneous Multithreading (SMT). Por ello, se probarán primero con 16 hilos, ya que teóricamente es la manera que mejor rendimiento puede conseguir; luego con 8 para comprobar si el SMT está siendo eficiente y finalmente se experimentará con el doble (32) y la mitad (4) de los hilos.

Rendimiento de las Funciones

Función: solvels

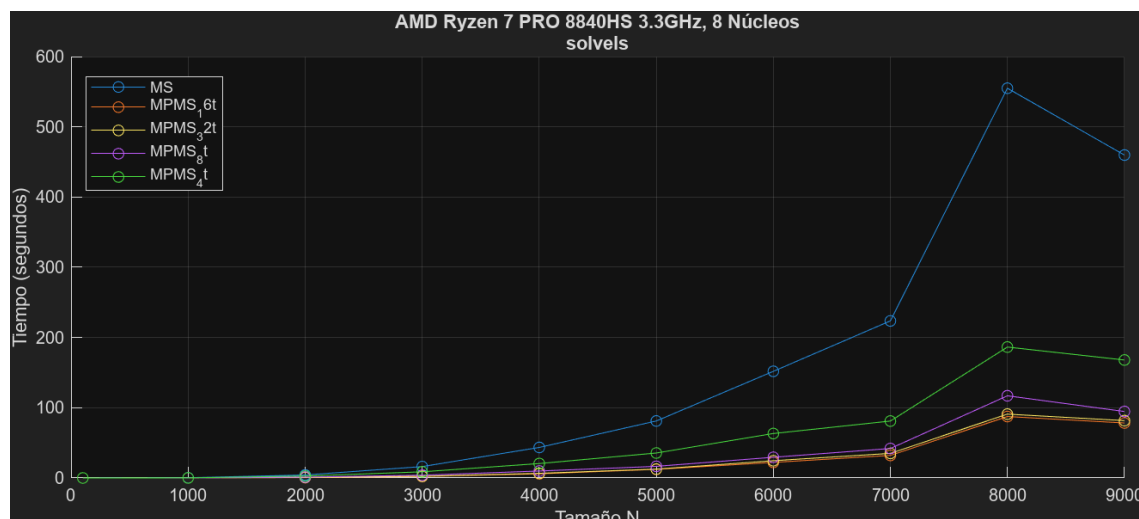


Figura 4.4: Tiempos de ejecución de `solvels` en un AMD Ryzen 7 PRO 8840HS

En la Figura 4.4 se muestran los tiempos de ejecución de MS y MPMS con distintos hilos en `solvels`. Como se puede observar, MPMS consigue tiempos de ejecución bastante menores; en concreto, su versión con 16 hilos parece ser la más rápida de todas.

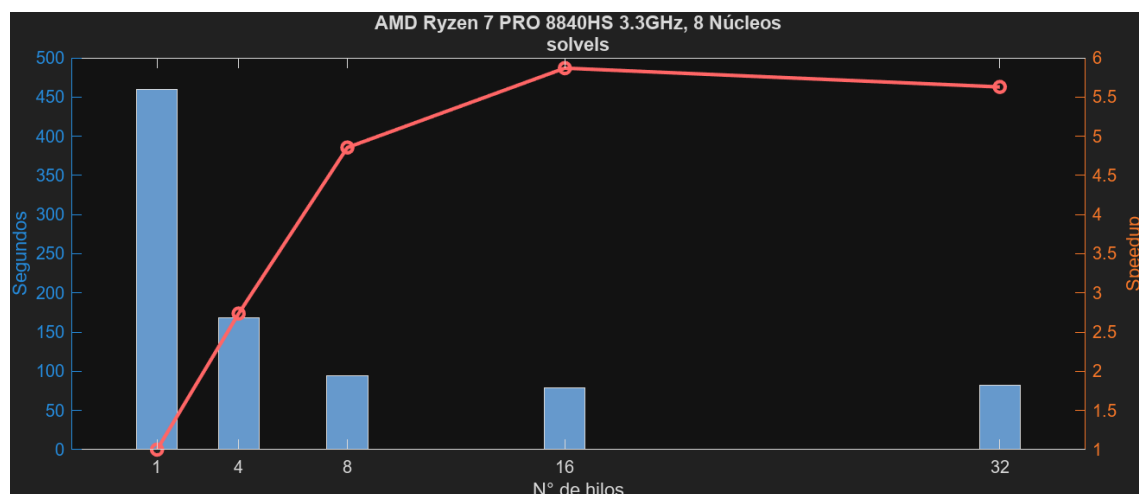


Figura 4.5: Tiempos y ganancias de `solvels` en un AMD Ryzen 7 PRO 8840HS

En cuanto a la Figura 4.5, muestra los tiempos finales de ejecución para cada versión, esta vez también añadiendo sus correspondientes SpeedUps. Se puede apreciar que la ganancia aumenta de forma progresiva hasta los 16 hilos; a partir de estos se reduce.

En cuanto a la presente tabla, muestra las ganancias y eficiencias por hilo de

Capítulo 4. Experimentación y Resultados

Nº Hilos	SpeedUp	Eficiencia
4	2.74	0.68
8	4.86	0.61
16	5.87	0.37
32	5.63	0.18

cada versión. Como se puede apreciar, la mayor eficiencia por hilo se consigue con únicamente cuatro hilos, muy seguido por 8 hilos y decreciendo a medida que aumenta el número de hilos.

Función: `mmul`

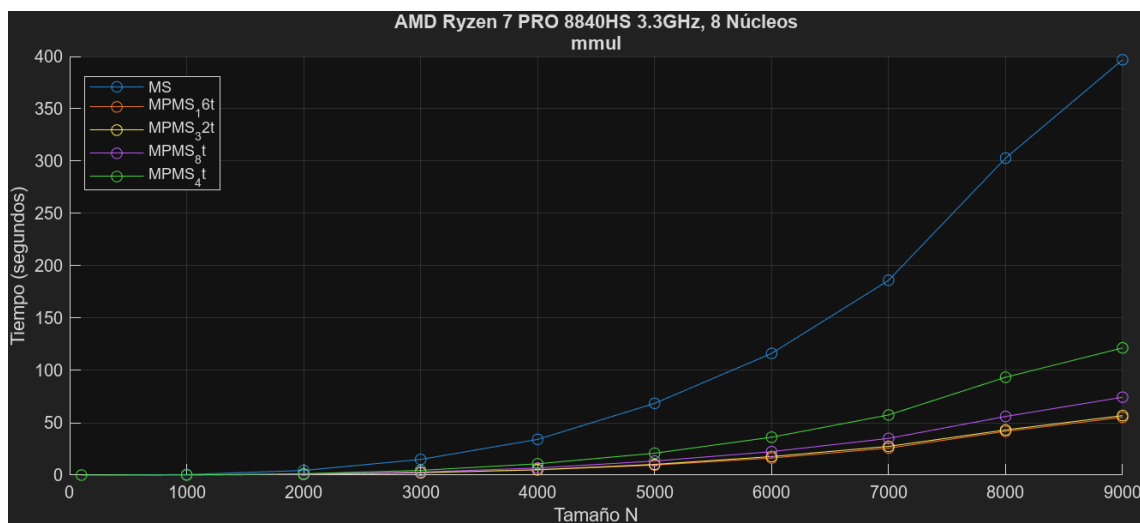


Figura 4.6: Tiempos de ejecución de `mmul` en un AMD Ryzen 7 PRO 8840HS

En la Figura 4.6, se puede apreciar una gran diferencia entre tiempos de ejecución de `mmul`, siendo de nuevo la versión de 16 hilos la que mejor tiempo consigue, muy seguida de la de 32 hilos, luego 8 hilos y, finalmente, 4 hilos.

En la Figura 4.7 se nos muestra que las ganancias de cada versión, al ser la versión de 16 hilos la más rápida, su ganancia es la mayor, muy seguida de la versión de 32 hilos. De nuevo, aumentando de forma gradual la ganancia, estancándose en 16 hilos y luego decreciendo.

Nº Hilos	SpeedUp	Eficiencia
4	3.27	0.82
8	5.33	0.67
16	7.19	0.45
32	6.98	0.22

Como se ha mencionado antes, con 16 hilos se consigue la mayor ganancia(7.19) sin embargo, la mayor eficiencia por hilo se produce con 4 hilos (0.81), disminuyendo esta a medida que aumentan los hilos.

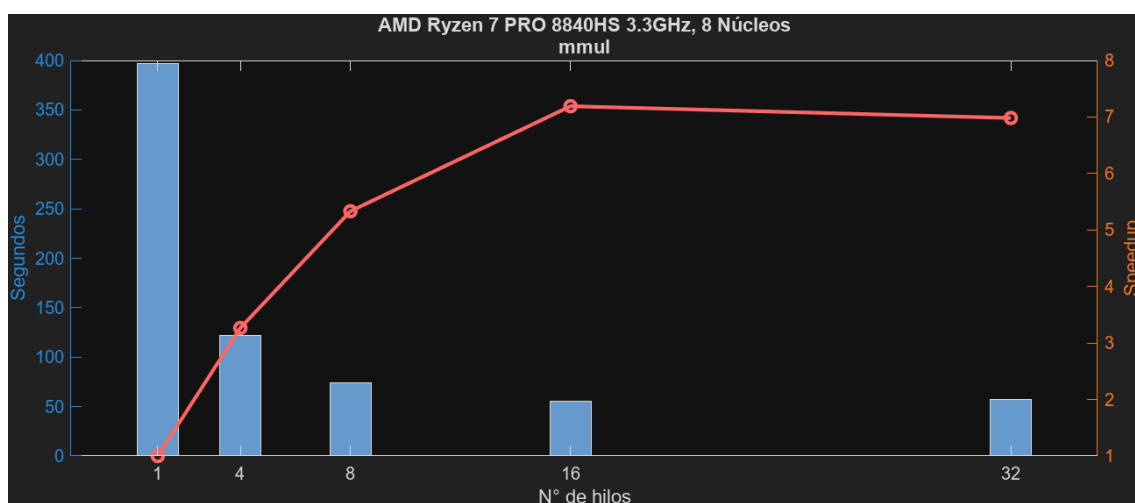


Figura 4.7: Tiempos y ganancias de `mmul` en un AMD Ryzen 7 PRO 8840HS

Función: `mset`

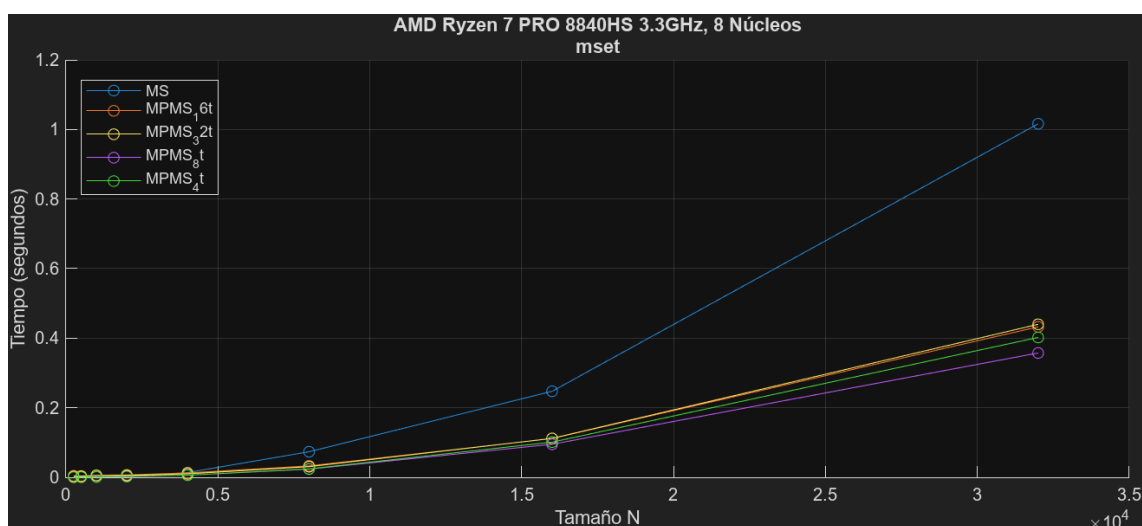


Figura 4.8: Tiempos de ejecución de `mset` en un AMD Ryzen 7 PRO 8840HS

En cuanto a la función `mset`, la Figura 4.8 muestra que los tiempos de ejecución no son mucho mejores en su versión paralela que en la secuencial, obteniendo un rendimiento similar en las versiones paralelas independientemente del número de hilos utilizados.

En cuanto a sus ganancias, en la Figura 4.9 se observa la mayor con 8 hilos, siendo este el número a partir del cual comienza a decrecer la ganancia al aumentar los hilos.

De nuevo, la mayor eficiencia se consigue con 4 hilos, siendo esta de 0.63 por hilo.

Capítulo 4. Experimentación y Resultados

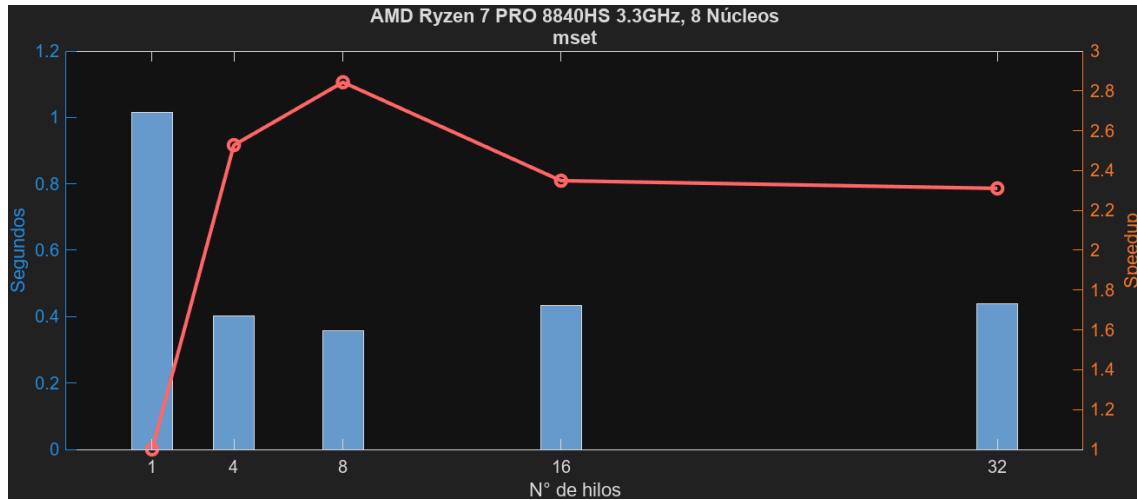


Figura 4.9: Tiempos y ganancias de `mset` en un AMD Ryzen 7 PRO 8840HS

Nº Hilos	SpeedUp	Eficiencia
4	2.53	0.63
8	2.84	0.36
16	2.35	0.15
32	2.31	0.07

Función: `mnorm`

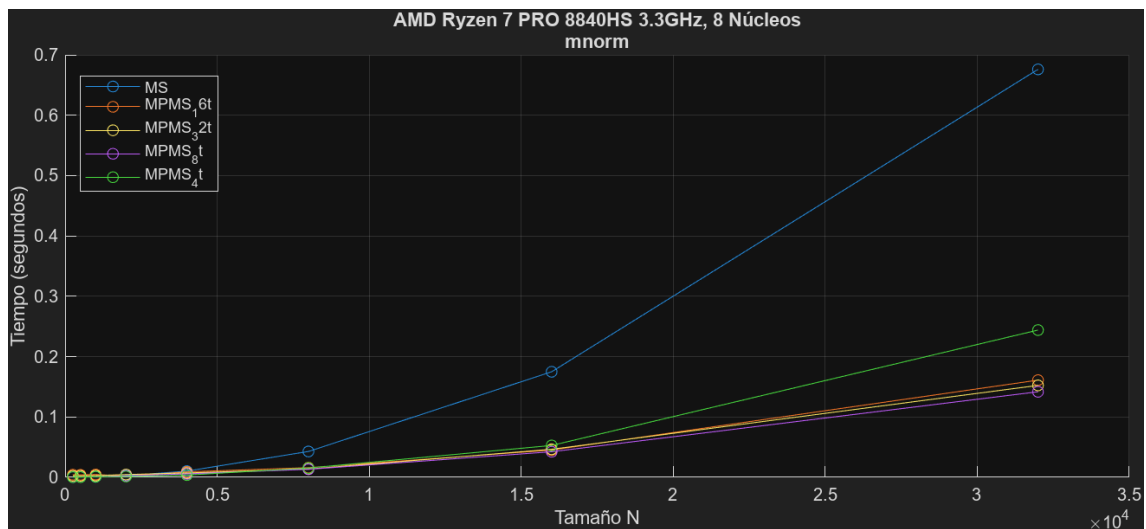


Figura 4.10: Tiempos de ejecución de `mnorm` en un AMD Ryzen 7 PRO 8840HS

En cuanto a la función `mnorm`, sus tiempos de ejecución representados en la Figura 4.10 muestran una gran diferencia entre las versiones paralelas y la secuencial. Sin embargo, las versiones paralelas no muestran grandes diferencias entre ellas, siendo la de 8 hilos la más rápida y la de 4 hilos la que más se aleja

del resto.

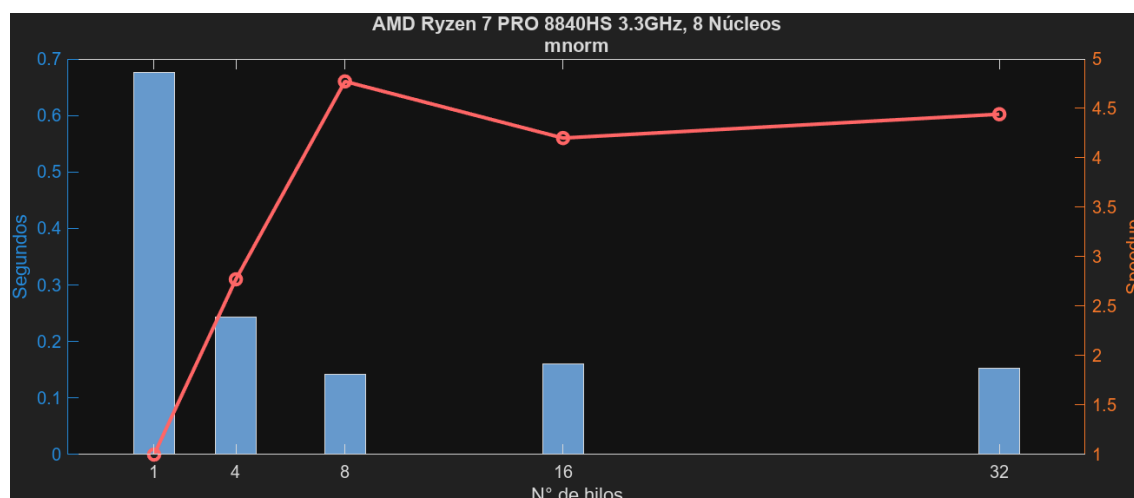


Figura 4.11: Tiempos y ganancias de `mnorm` en un AMD Ryzen 7 PRO 8840HS

Las ganancias que se muestran en la Figura 4.11, permiten afirmar que la ganancia aumenta progresivamente en la función `mnorm` hasta los 8 hilos; a partir de ese número, decrece con un ligero repunte en los 32 hilos.

Nº Hilos	SpeedUp	Eficiencia
4	2.77	0.69
8	4.77	0.60
16	4.20	0.26
32	4.44	0.14

Para concluir, se consigue la mayor eficiencia por hilo con únicamente cuatro hilos (0.69) y el mayor de los SpeedUps con 8, muy seguido de la versión con 32 hilos.

Función: `mmulc`

La Figura 4.12 muestra que en los tiempos de ejecución la función `mmul` se consigue una gran mejora respecto a la versión secuencial; sin embargo, las versiones paralelas tienen rendimientos muy similares.

En cuanto a las ganancias, la Figura 4.13 permite concluir que aumenta rápidamente la ganancia usando 4 hilos, pero luego aumenta gradualmente hasta el uso de 8 y continúa decreciendo de forma gradual hasta los 32.

Finalmente, la mayor eficiencia por hilo (0.77) es lograda de nuevo por la versión con menos hilos, la de cuatro. Mientras que el SpeedUp más alto lo tiene, como se ha mencionado antes, la versión de ocho hilos (3.20).

Capítulo 4. Experimentación y Resultados

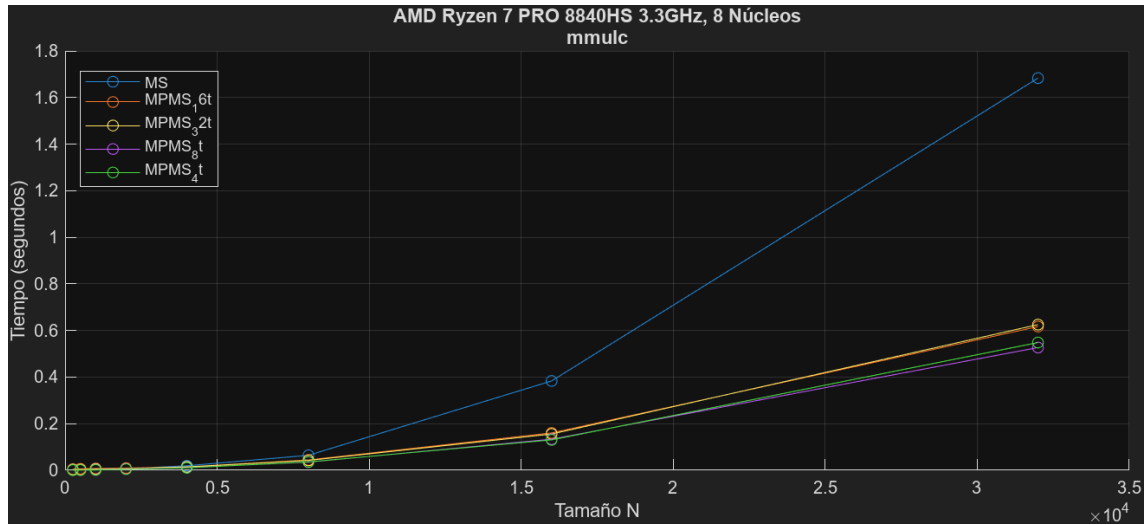


Figura 4.12: Tiempos de ejecución de `mmulc` en un AMD Ryzen 7 PRO 8840HS

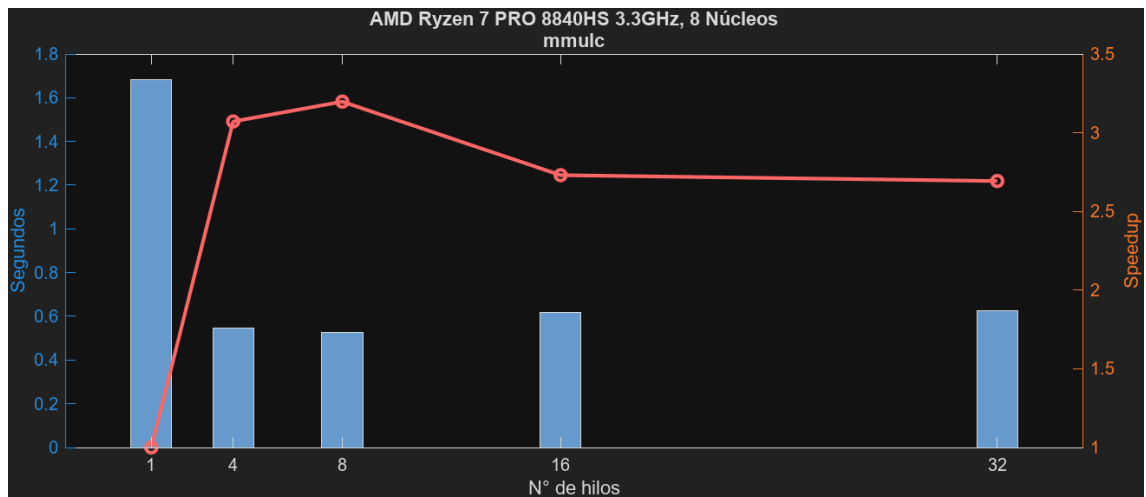


Figura 4.13: Tiempos y ganancias de `mmulc` en un AMD Ryzen 7 PRO 8840HS

Nº Hilos	SpeedUp	Eficiencia
4	3.07	0.77
8	3.20	0.40
16	2.73	0.17
32	2.69	0.08

Función: `mcpy`

La Figura 4.14 muestra los tiempos de ejecución de la función `mcpy`. En ella se puede ver de nuevo que se consigue una mejora, pero no hay mucha diferencia entre las versiones paralelas.

Se pueden observar sus respectivas ganancias en la Figura 4.15. Estas ganan-

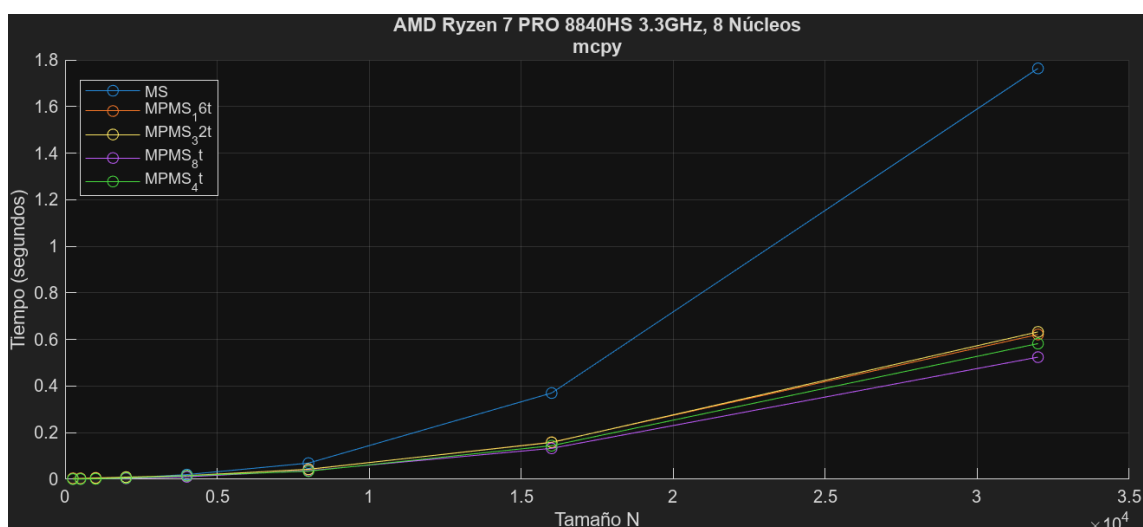


Figura 4.14: Tiempos de ejecución de `mcpy` en un AMD Ryzen 7 PRO 8840HS

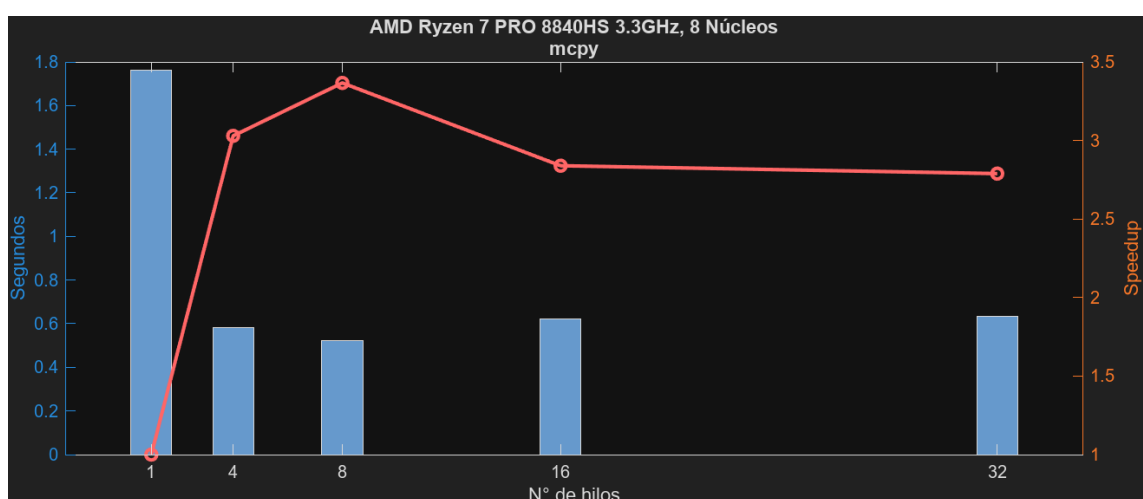


Figura 4.15: Tiempos y ganancias de `mcpy` en un AMD Ryzen 7 PRO 8840HS

cias aumentan de forma muy similar a las de la función `mmulc`. Aumentan de hasta el uso de ocho hilos para luego decrementarse de forma progresiva.

Nº Hilos	SpeedUp	Eficiencia
4	3.03	0.76
8	3.37	0.42
16	2.84	0.18
32	2.79	0.09

Se puede concluir que la mayor eficiencia por hilo se obtiene únicamente utilizando cuatro hilos (0.76), mientras que el mayor SpeedUP es con ocho hilos (3.37)

Capítulo 4. Experimentación y Resultados

Función: msub

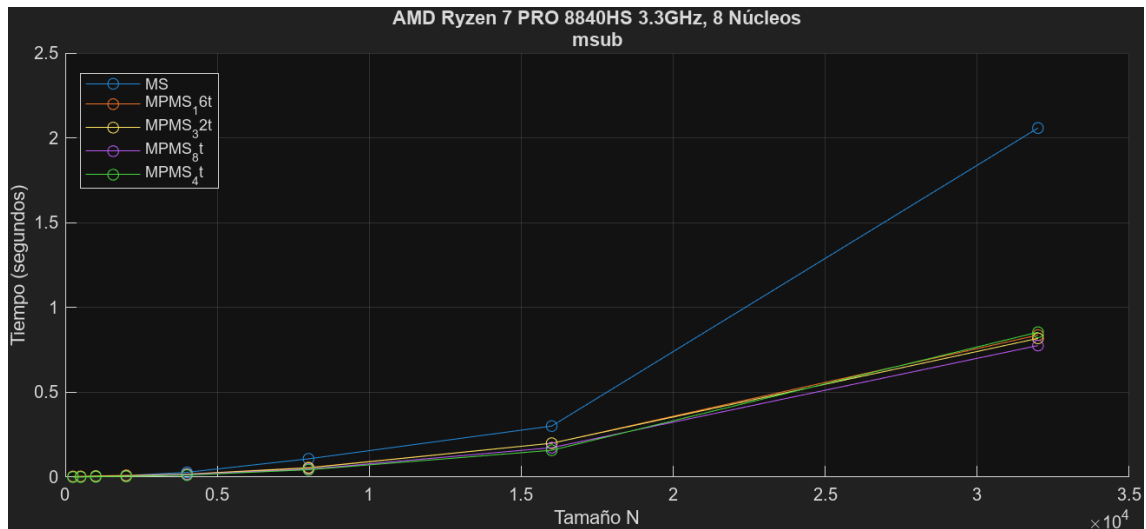


Figura 4.16: Tiempos de ejecución de `msub` en un AMD Ryzen 7 PRO 8840HS

En la Figura 4.16 se observan los tiempos de ejecución de la función `msub`, en los que se puede observar una mejora, pero nada remarcable, siendo muy similar en todas las versiones paralelas.

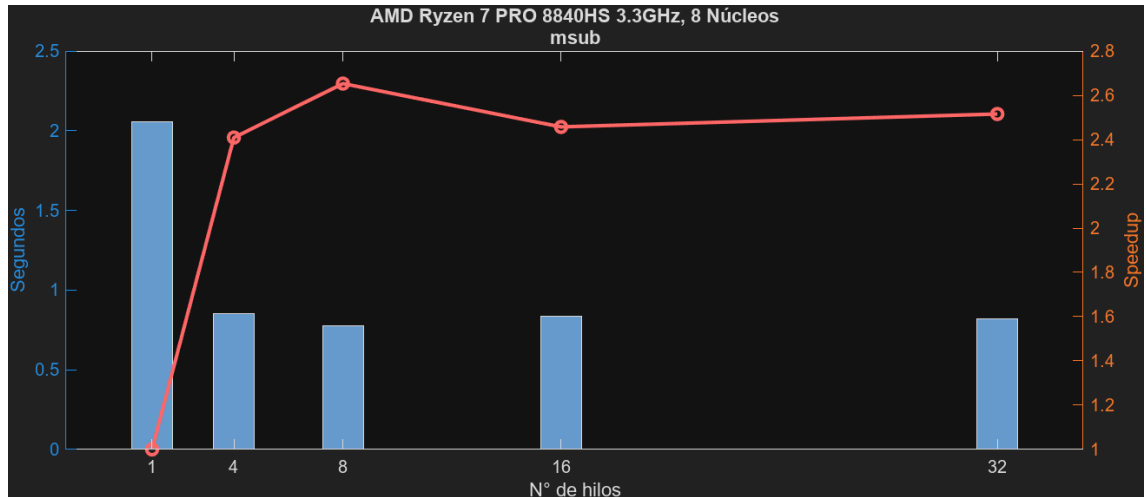


Figura 4.17: Tiempos y ganancias de `msub` en un AMD Ryzen 7 PRO 8840HS

En cuanto a la Figura 4.17, muestra sus respectivas ganancias, las cuales se estancan y decrecen a partir del uso de ocho hilos.

Para concluir, de nuevo la mayor eficiencia es de 0.60 perteneciendo a la versión de cuatro hilos, mientras que la versión de 32 consigue una eficiencia por hilo muy baja (0.08), ya que no mejora el tiempo de ejecución. Por otro lado, la mayor ganancia se consigue con ocho hilos, muy seguida de la conseguida con dieciséis.

Nº Hilos	SpeedUp	Eficiencia
4	2.41	0.60
8	2.65	0.33
16	2.46	0.15
32	2.52	0.08

Rendimiento del Algoritmo

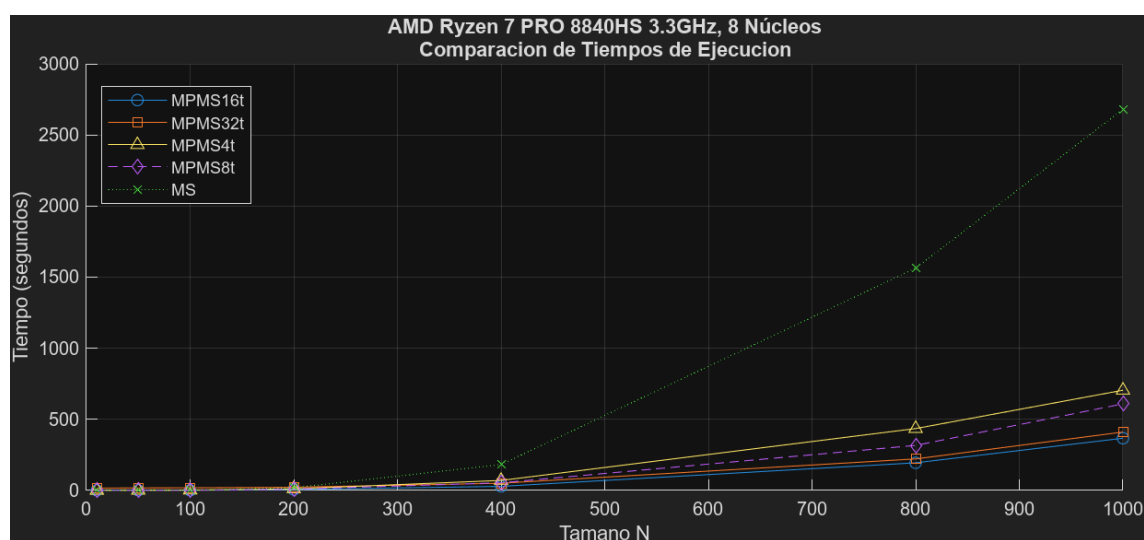


Figura 4.18: Tiempos de ejecución totales en un AMD Ryzen 7 PRO 8840HS

La Figura 4.18 muestra los tiempos de ejecución del algoritmo en conjunto utilizando las diferentes versiones de las librerías. Se puede apreciar en ella una gran mejoría en las versiones paralelas, siendo la que más la de dieciséis hilos y la que menos la de cuatro hilos.

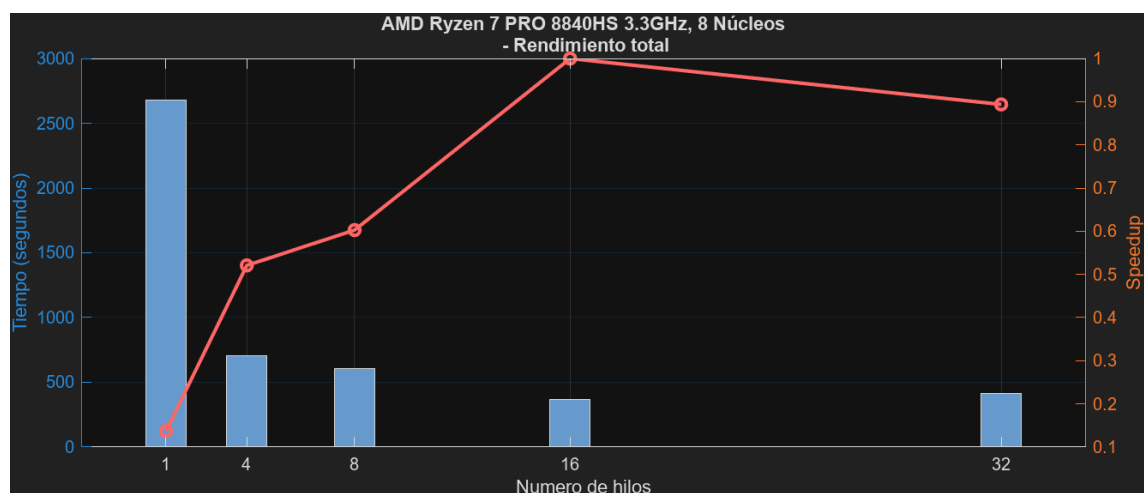


Figura 4.19: Tiempos y ganancias totales en un AMD Ryzen 7 PRO 8840HS

Capítulo 4. Experimentación y Resultados

Por otro lado, la Figura 4.19 muestra las ganancias obtenidas en el algoritmo en conjunto, pudiéndose apreciar un aumento semigradual de la ganancia hasta el uso de dieciséis hilos.

Nº Hilos	SpeedUp	Eficiencia
4	3.81	0.95
8	4.40	0.55
16	7.31	0.46
32	6.53	0.20

Finalmente, se puede afirmar que se consigue un gran SpeedUp en el algoritmo con este entorno. El mayor de ellos (7.31) se consigue con dieciséis hilos y, teniendo en cuenta que este entorno únicamente cuenta con 8 núcleos físicos, se podría decir que se consigue una alta paralelización, notablemente eficiente. Por otro lado, se consigue la mayor eficiencia por hilo utilizando únicamente cuatro hilos, 0.95 en concreto, una cifra notablemente grande.

Observaciones

Tras analizar los resultados, se puede concluir que el primer entorno ha conseguido un buen rendimiento en general, a excepción de las funciones `mset` y `mcpy`, que se ven lastradas por los constantes accesos a memoria. En funciones como `solvels` y `mmul` se obtienen las mejores ganancias utilizando 16 hilos, siendo `mmul` la función de la librería en la que mayor ganancia se consigue, 7.19 con una eficiencia de 0.45 por hilo. Por otro lado, el resto de funciones consiguen un mejor rendimiento utilizando solo 8 hilos, en vez de los 16 hilos con Simultaneous Multi-Threading. Este resultado se debe principalmente a que, con SMT, si dos hilos del mismo núcleo físico requieren los mismos recursos al mismo tiempo (por ejemplo, ambos necesitan la ALU para cálculos intensivos), el rendimiento puede ser similar o peor al de utilizar un solo hilo. En general, el primer entorno ha obtenido el resultado esperado; a pesar de que en algunas funciones sea mejor utilizar 8 hilos, el uso de los 16 hilos aprovecha el multi-threading y mejora el rendimiento total.

4.2. Entorno 2

El segundo entorno cuenta con un procesador AMD Ryzen 7 5800H, también de versión portátil con 8 núcleos a 3.2GHz. A continuación, se van a mostrar su rendimiento en las pruebas de las diferentes funciones de la librería y en el algoritmo. De nuevo, todas las pruebas las realizarán utilizando diferentes números de hilos, en concreto, 16, 32, 8 y 4 hilos. Este procesador tiene gran similitud con el primer entorno, por lo que se espera obtener unos resultados similares a los del anterior.

Rendimiento de las Funciones

Función: solvels

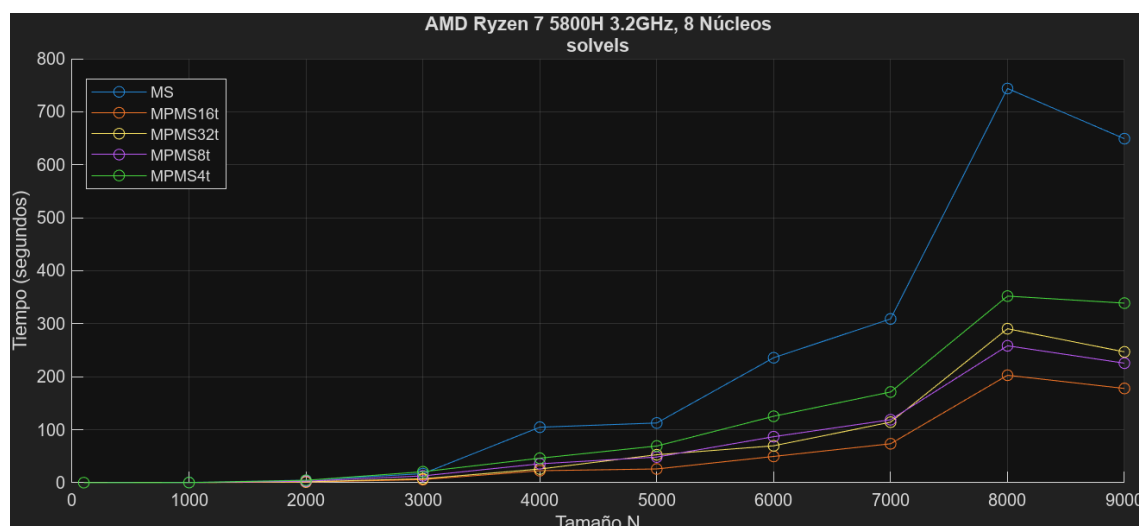


Figura 4.20: Tiempos de ejecución de `sodels` en un AMD Ryzen 7 5800H

Se puede observar en la Figura 4.20 los diferentes tiempos de ejecución obtenidos para la función `sodels` en este entorno. Es fácil apreciar que los tiempos de ejecución aumentan de manera uniforme, siendo ampliamente menores en las versiones paralelas. En concreto, se consiguen los mejores tiempos en la versión que utiliza dieciséis hilos.

En cuanto a sus ganancias, en la Figura 4.21 se muestra un aumento gradual de la ganancia hasta el uso de dieciséis hilos, número a partir del cual comienza a descender progresivamente.

Nº Hilos	SpeedUp	Eficiencia
4	1.92	0.48
8	2.88	0.36
16	3.65	0.23
32	2.63	0.08

Capítulo 4. Experimentación y Resultados

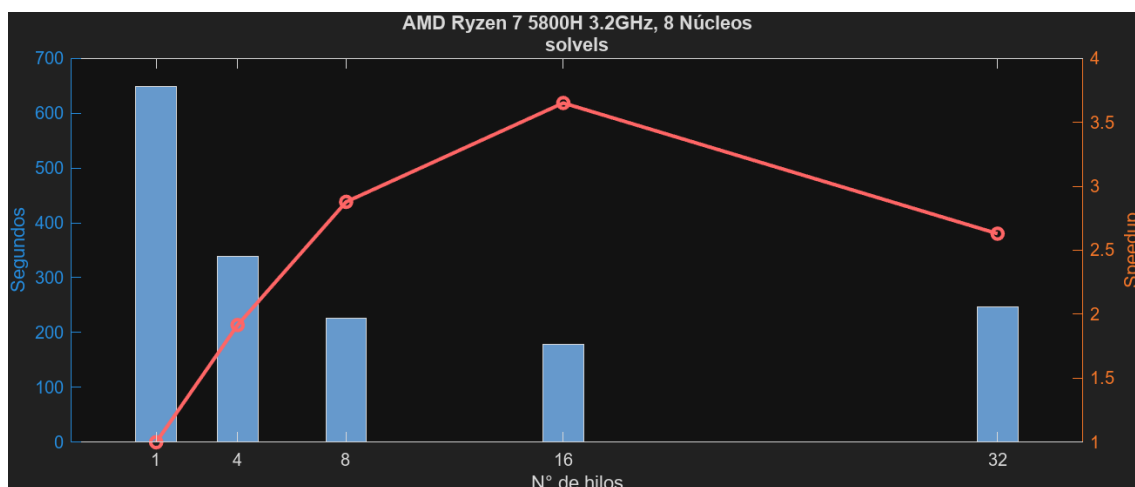


Figura 4.21: Tiempos y ganancias de `solvels` en un AMD Ryzen 7 5800H

Los resultados permiten concluir que la versión de dieciséis hilos consigue los mejores resultados y, por tanto, el mayor SpeedUp. Sin embargo, la mayor eficiencia por hilo se conseguiría utilizando únicamente cuatro de los hilos.

Función: `mmul`

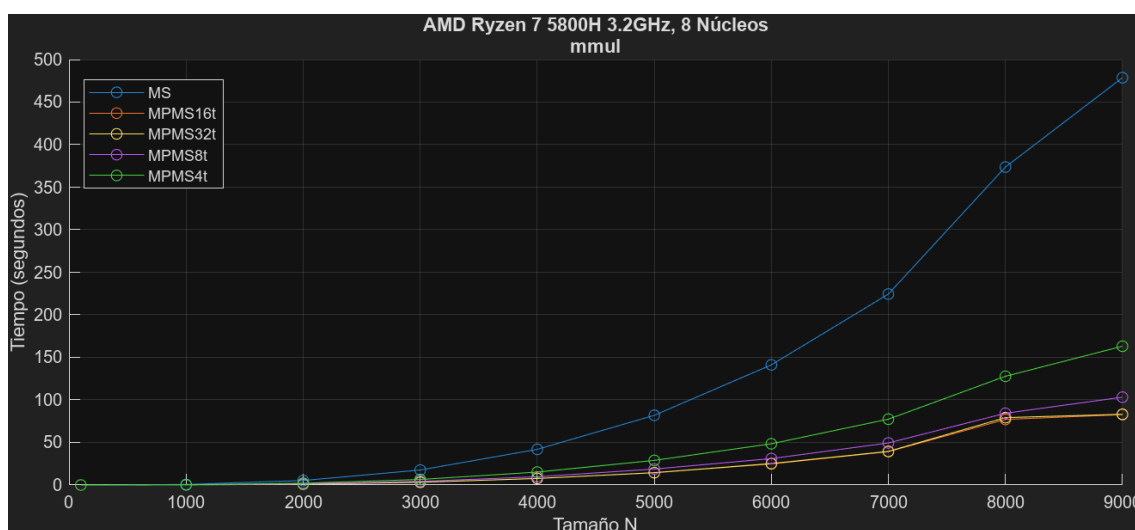


Figura 4.22: Tiempos de ejecución de `mmul` en un AMD Ryzen 7 5800H

Para analizar el rendimiento de la función `mmul`, se puede primero apreciar en la Figura 4.22 un aumento progresivo de los tiempos de ejecución en todas las versiones, siendo mucho más pronunciado en la versión secuencial. Destaca el rendimiento del uso de 16 y 32 hilos, ya que consiguen unos tiempos de ejecución mucho menores a los del resto.

Por otro lado, la Figura 4.23 representa las ganancias de cada versión, apreciándose de forma clara un aumento gradual hasta el uso de 16 hilos, a partir del

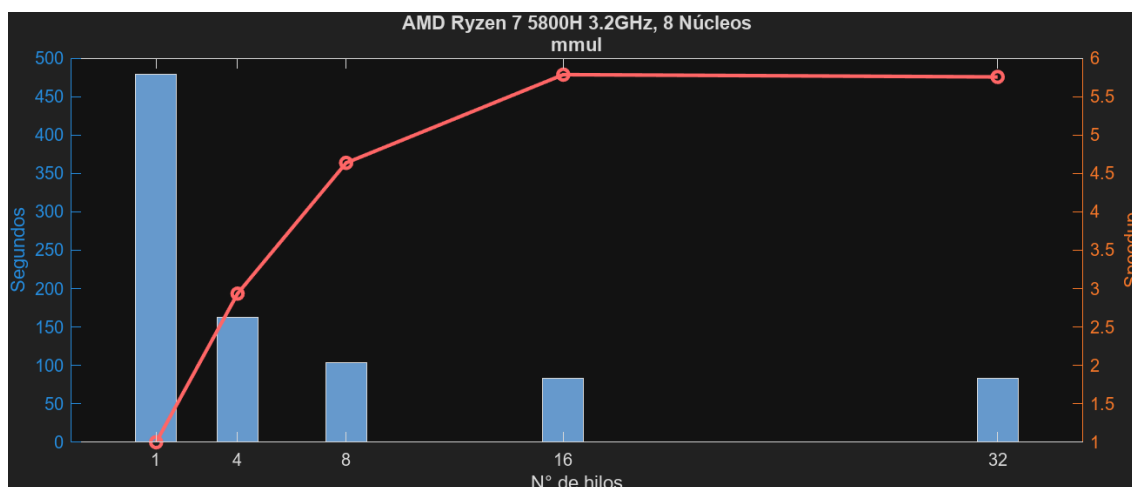


Figura 4.23: Tiempos y ganancias de `mmul` en un AMD Ryzen 7 5800H

cual se produce un estancamiento.

Nº Hilos	SpeedUp	Eficiencia
4	2.94	0.73
8	4.64	0.58
16	5.79	0.36
32	5.76	0.18

Finalmente, las versiones de 16 y 32 hilos consiguen los mayores SpeedUps con 5.79 y 5.76, respectivamente. Por otro lado, la mayor eficiencia por hilo vuelve a pertenecer al uso de menos hilos, en concreto, cuatro.

Función: `mset`

En cuanto a los tiempos de ejecución de la función `mset`, son representados por la Figura 4.24. Se puede apreciar en ellos muy poca mejora de las versiones paralelas respecto a la versión secuencial. Esto se debe principalmente a los accesos a memoria en esta función y a la arquitectura de la caché.

Las ganancias mostradas en la Figura 4.25 muestran un incremento rápido del SpeedUp con el uso de 4 hilos, pero que desaparece y se decrementa al utilizar más hilos.

Nº Hilos	SpeedUp	Eficiencia
4	1.45	0.36
8	1.20	0.15
16	1.09	0.07
32	1.07	0.03

Finalmente, los resultados arrojan unas ganancias muy poco notables en todas las versiones y, de la misma forma, se ven representadas sus eficiencias.

Capítulo 4. Experimentación y Resultados

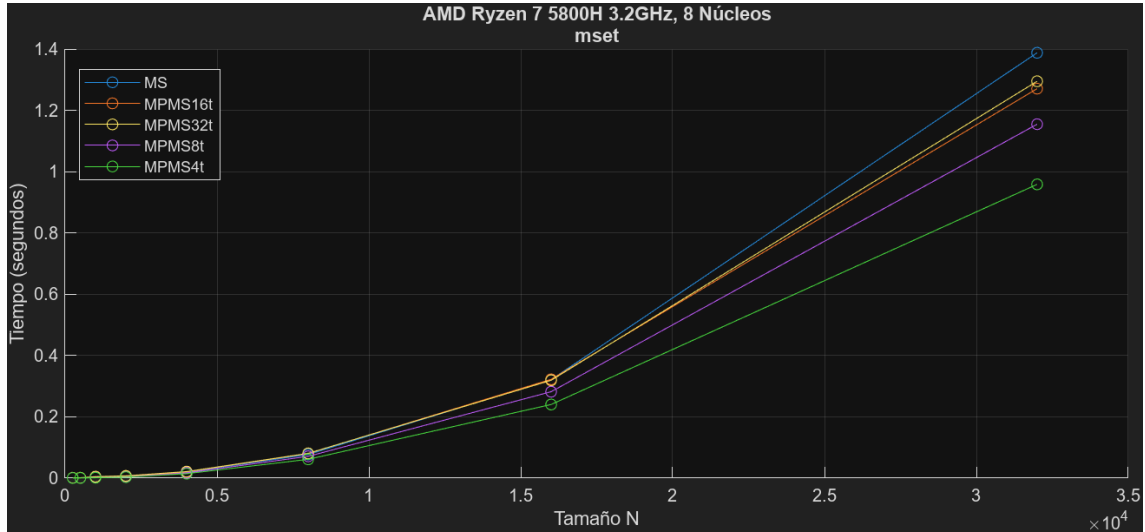


Figura 4.24: Tiempos de ejecución de `mset` en un AMD Ryzen 7 5800H

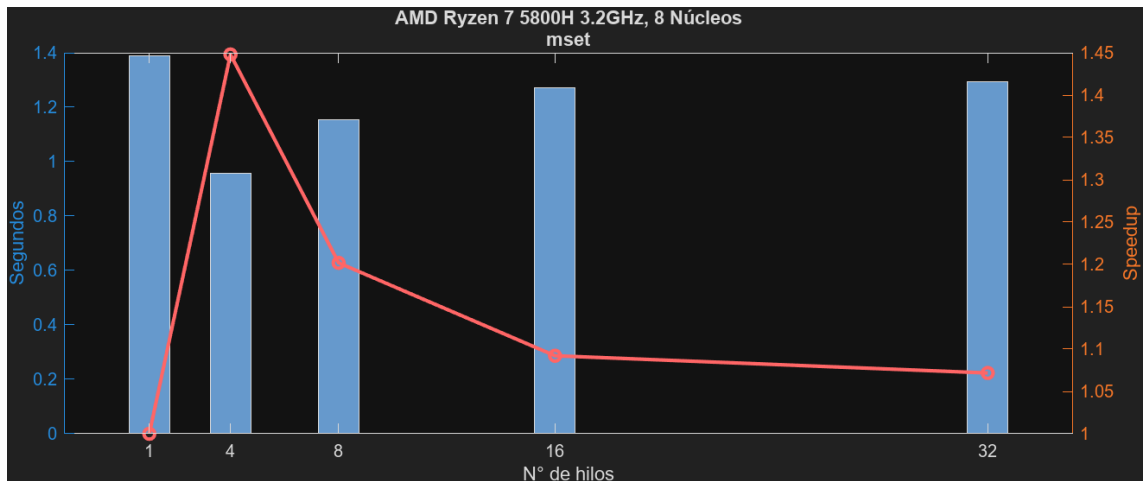


Figura 4.25: Tiempos y ganancias de `mset` en un AMD Ryzen 7 5800H

Función: `mnorm`

En la Figura 4.26 se ven graficados los tiempos de ejecución de la función `mnorm`. Estos tiempos son mucho menores en las versiones paralelas de la función, siendo el menor con el uso de 8 hilos.

En la Figura 4.27 se muestran las correspondientes ganancias, pudiéndose apreciar fácilmente un incremento de la ganancia que alcanza su punto máximo en ocho hilos, para después decrementarse con un ligero repunte en 32 hilos.

Se concluye la función `mnorm` consiguiendo el mayor SpeedUp con el uso de 8 hilos y la mayor eficiencia por hilo con solo 4.

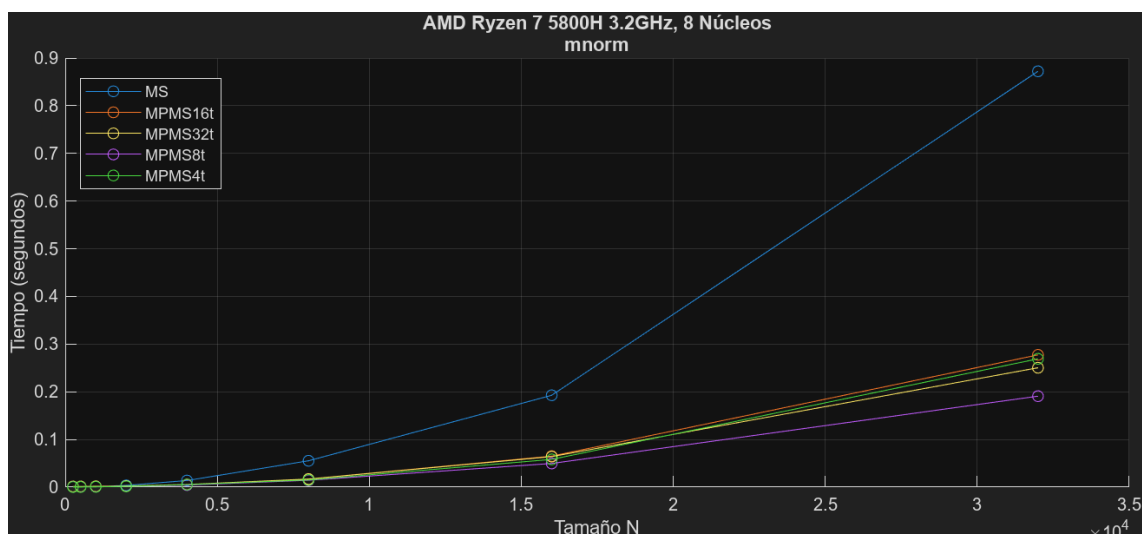


Figura 4.26: Tiempos de ejecución de `mnorm` en un AMD Ryzen 7 5800H

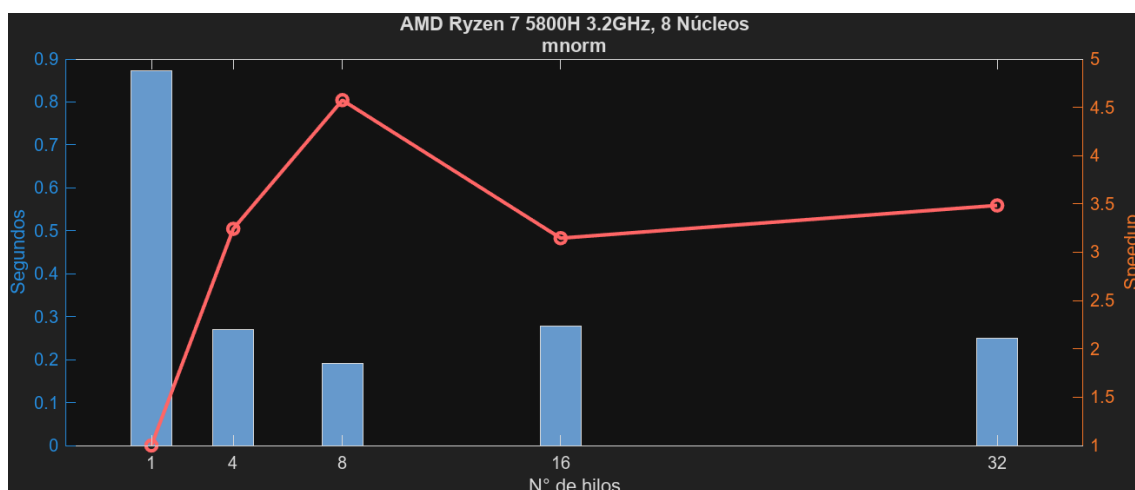


Figura 4.27: Tiempos y ganancias de `mnorm` en un AMD Ryzen 7 5800H

Nº Hilos	SpeedUp	Eficiencia
4	3.24	0.81
8	4.57	0.57
16	3.15	0.20
32	3.48	0.11

Función: `mmulc`

Como se puede apreciar en los tiempos de ejecución de la función `mmulc`, representados en la Figura 4.28, el menor tiempo de ejecución es conseguido por la versión que utiliza únicamente cuatro hilos, muy distante del tiempo de la versión con dieciséis.

Capítulo 4. Experimentación y Resultados

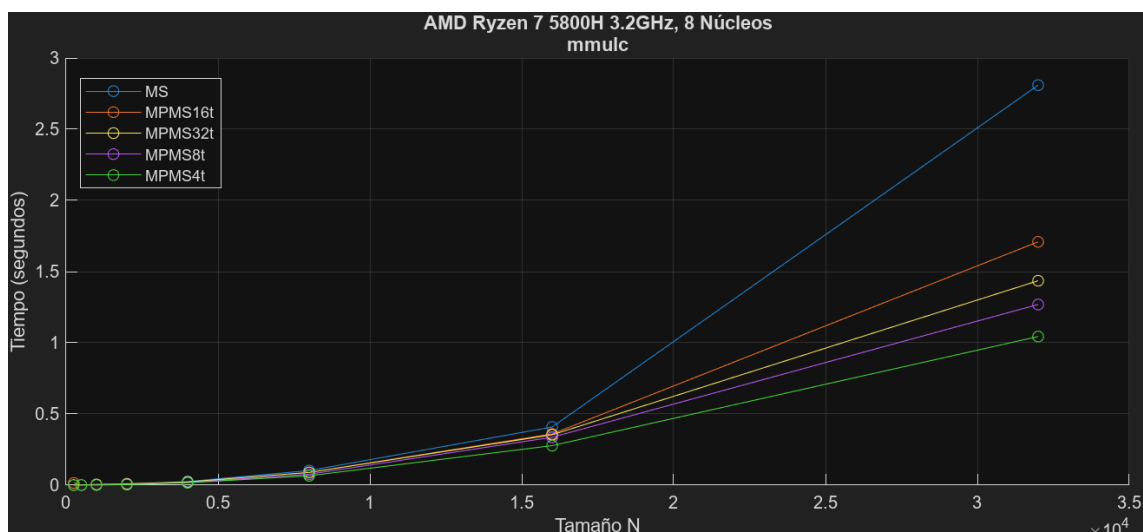


Figura 4.28: Tiempos de ejecución de `mmulc` en un AMD Ryzen 7 5800H

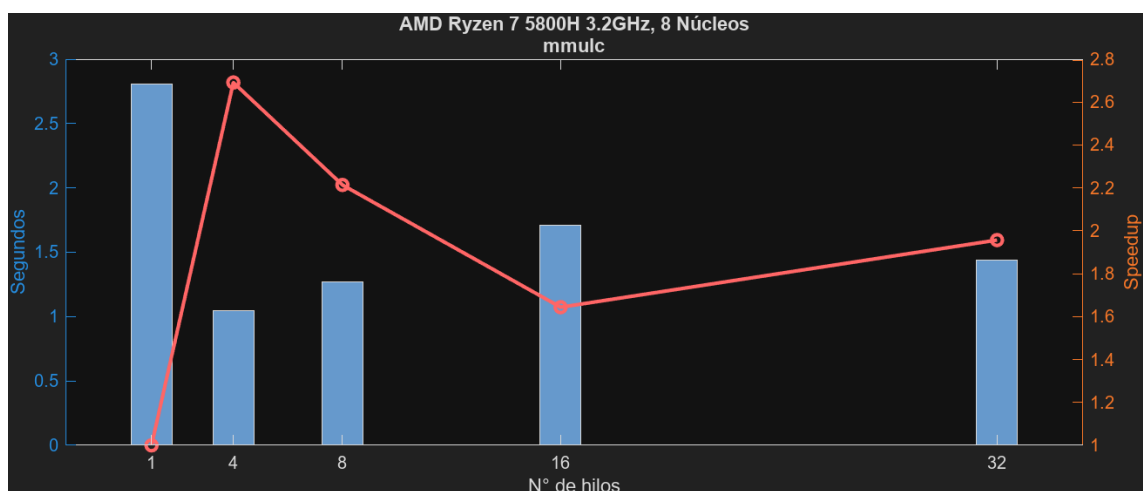
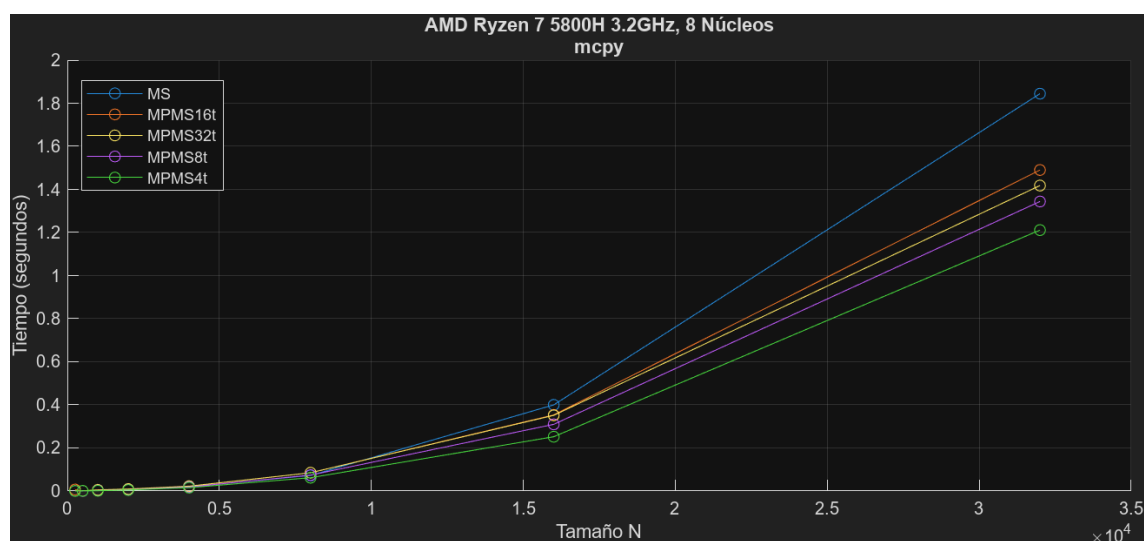


Figura 4.29: Tiempos y ganancias de `mmulc` en un AMD Ryzen 7 5800H

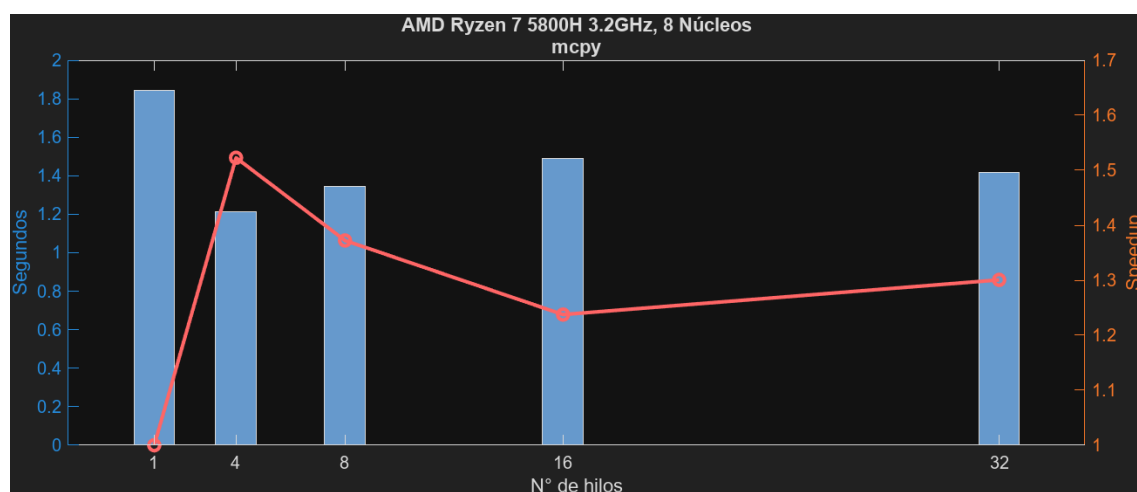
Las ganancias representadas en la Figura 4.29 son acordes a lo mencionado anteriormente. Se consigue la mayor de ellas utilizando cuatro hilos y, a partir de ahí, comienza a decrementarse, llegando a su punto más bajo con 16 hilos; no obstante, consigue un ligero incremento al utilizar 32.

Nº Hilos	SpeedUp	Eficiencia
4	2.69	0.67
8	2.21	0.28
16	1.64	0.10
32	1.96	0.06

Como resultados finales de la función `mmulc`, se puede afirmar que la mayor ganancia y eficiencia por hilo se consiguen ambas con cuatro hilos.

Función: mcpyFigura 4.30: Tiempos de ejecución de `mcpy` en un AMD Ryzen 7 5800H

Continuando con la función `mcpy`, la Figura 4.30 que representa sus tiempos de ejecución permite apreciar que no existe una gran mejoría de los tiempos de las versiones paralelas respecto a la secuencial. Esto se debe principalmente a la arquitectura de las memorias caché, en las que se produce *cache thrashing* entre los hilos.

Figura 4.31: Tiempos y ganancias de `mcpy` en un AMD Ryzen 7 5800H

Las ganancias mostradas en la Figura 4.31, incrementan rápidamente con el uso de cuatro hilos, pero decrecen progresivamente a partir de ese punto.

Se concluye de nuevo que el uso de cuatro hilos vuelve a tener el mayor SpeedUp (1.52) junto con la mayor eficiencia por hilo (0.38).

Capítulo 4. Experimentación y Resultados

Nº Hilos	SpeedUp	Eficiencia
4	1.52	0.38
8	1.37	0.17
16	1.24	0.08
32	1.30	0.04

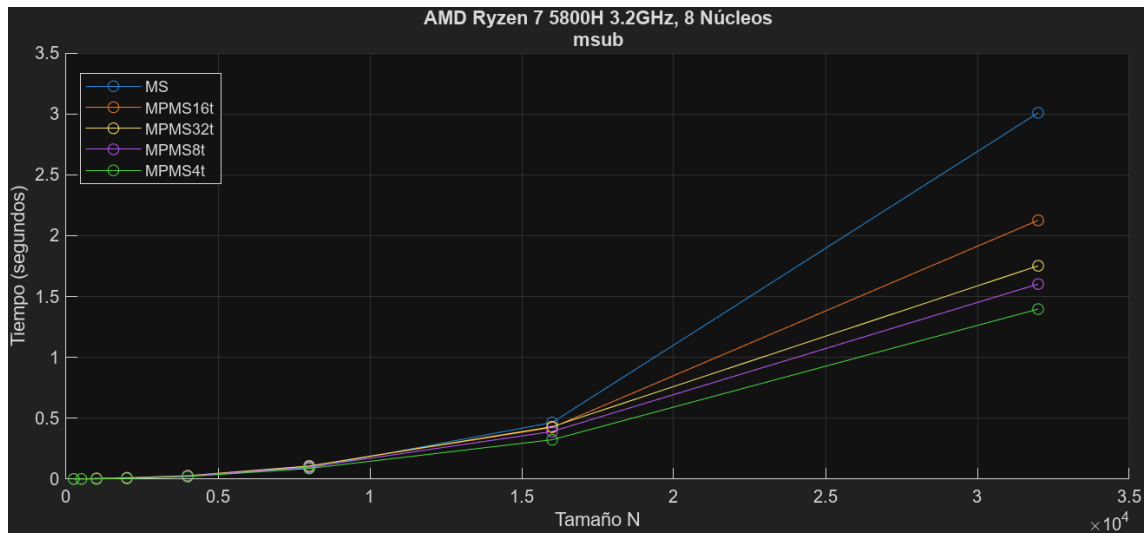


Figura 4.32: Tiempos de ejecución de `msub` en un AMD Ryzen 7 5800H

Función: `msub`

Los tiempos de ejecución de la función `msub` en la Figura 4.32 muestran un rendimiento similar al de la función anterior `mcpy`, pero notándose una mayor mejora.

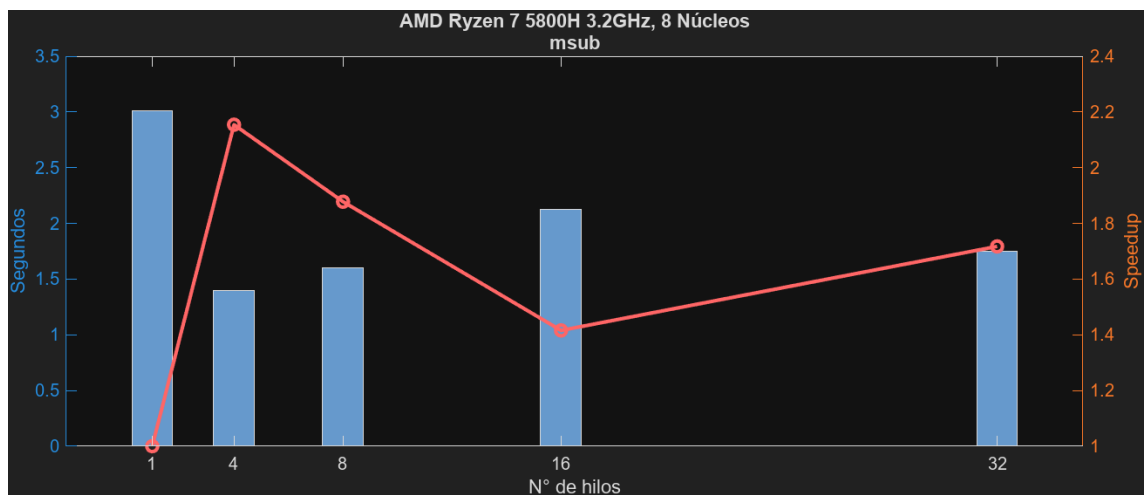


Figura 4.33: Tiempos y ganancias de `msub` en un AMD Ryzen 7 5800H

Pese a que la mejoría sea mayor, se puede observar en la Figura 4.33 que las ganancias han sido proporcionales, llegando a su punto máximo utilizando cuatro hilos.

Nº Hilos	SpeedUp	Eficiencia
4	2.15	0.54
8	1.88	0.23
16	1.42	0.09
32	1.72	0.05

Para concluir, se puede afirmar que de nuevo se consigue la mayor ganancia y eficiencia por hilo con el uso de únicamente cuatro hilos.

Rendimiento del Algoritmo

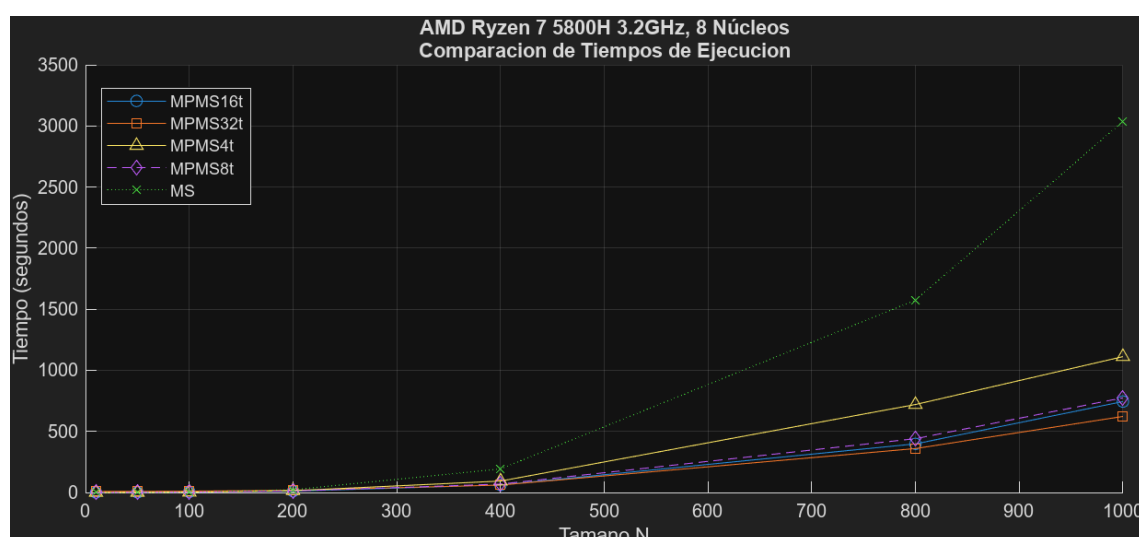


Figura 4.34: Tiempos de ejecución totales en un AMD Ryzen 7 5800H

La Figura 4.34 se encarga de representar los tiempos totales de ejecución del algoritmo utilizando las diferentes versiones de las librerías. En ella se puede observar una notable disminución del tiempo de ejecución en las versiones que utilizan 32 y 16 hilos.

Sus ganancias representadas en la Figura 4.35 muestran un aumento proporcional del SpeedUp a medida que aumenta el número de hilos, el cual se incrementa de manera más pausada al pasar los ocho hilos.

Finalmente, se puede concluir que en este segundo entorno se consigue la mayor ganancia utilizando 32 hilos (4.88), aunque con la peor de las eficiencias por hilo (0.15). Por otro lado, se ve que esta eficiencia por hilo aumenta a medida que se decrementa el número de hilos.

Capítulo 4. Experimentación y Resultados

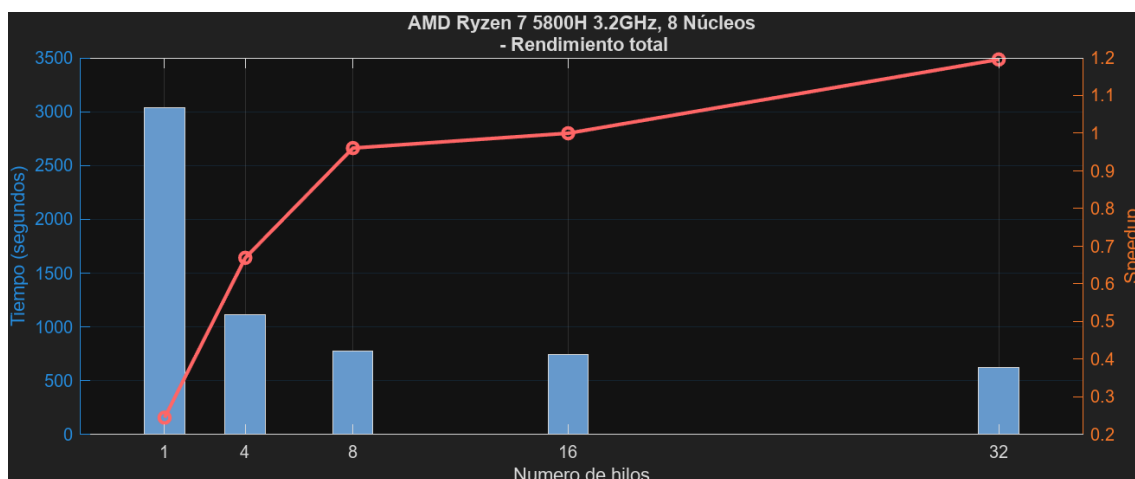


Figura 4.35: Tiempos y ganancias totales en un AMD Ryzen 7 PRO 8840HS

Nº Hilos	SpeedUp	Eficiencia
4	2.73	0.68
8	3.92	0.49
16	4.08	0.26
32	4.88	0.15

Observaciones

Como hemos mencionado antes, esperaríamos tener resultados muy similares a los del entorno anterior; sin embargo, cambian debido a la diferente arquitectura de la memoria caché. Nos ha llamado la atención que el uso de 8 y 4 hilos tiene buenos resultados, siendo los mejores resultados en funciones como `mset` y `mcpy`, junto con `mmulc` y `msub`, que se ven bastante lastradas por constantes accesos a memoria y los fallos en caché que estos conllevan. Podemos observar que el uso de SMT en las funciones no es eficiente, ya que si los hilos comparten datos que no caben en la caché L1 o L2 el rendimiento puede degradarse por los fallos que producen. Por otro lado, las funciones `solvefs` y `mmul` consiguen los mejores rendimientos usando 16 hilos, al igual que en el anterior entorno, pero esta vez seguidas muy de cerca por el rendimiento con 32 hilos. Finalmente, aunque no destaca en ninguna función específica, el uso de 32 hilos es la forma que mejores ganancias muestra en conjunto, consiguiendo un SpeedUp de 4.88 y una eficiencia por núcleo físico de 0.61.

4.3. Entorno 3

En cuanto al tercer entorno, es el primer procesador de la marca Intel en el que se harán pruebas. Se trata de un Intel Core i5-10600KF con 6 núcleos a 4.10GHz. Al igual que con los otros entornos, mostraremos a continuación su rendimiento en las pruebas de las diferentes funciones de la librería y en el algoritmo. Todas las pruebas se realizarán utilizando diferentes números de hilos, en concreto, 12, 24, 6 y 3 hilos.

Rendimiento de las Funciones

Función: solvels

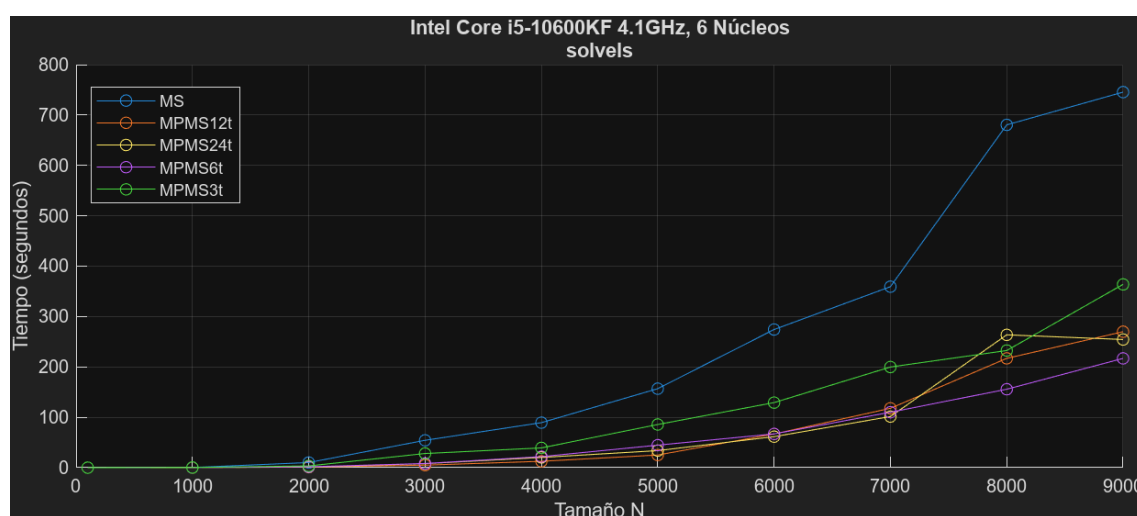


Figura 4.36: Tiempos de ejecución de `sodels` en un Intel Core i5-10600KF

Como se puede observar en la Figura 4.36, los tiempos de ejecución de la función `sodels` en este entorno incrementan de manera disforme para todas las versiones. En ella, los mejores tiempos de ejecución se consiguen utilizando seis hilos.

La Figura 4.36 muestra las ganancias de las diferentes versiones respecto a la versión secuencial. En ella se puede apreciar un aumento proporcional y progresivo hasta el uso de seis hilos, número a partir del cual comienzan a disminuir.

Nº Hilos	SpeedUp	Eficiencia
3	2.05	0.68
6	3.44	0.57
12	2.76	0.23
24	2.93	0.12

Se puede afirmar que en la función `sodels` se consigue el mayor SpeedUp (3.44) utilizando únicamente seis hilos; además, con una eficiencia muy cercana a la máxima.

Capítulo 4. Experimentación y Resultados

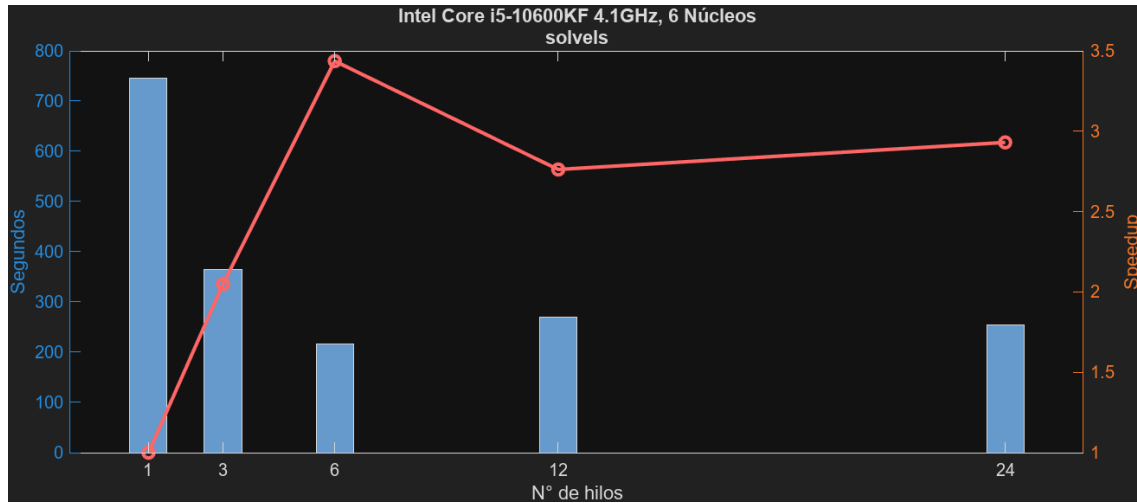


Figura 4.37: Tiempos y ganancias de `solvels` en un Intel Core i5-10600KF

Función: `mmul`

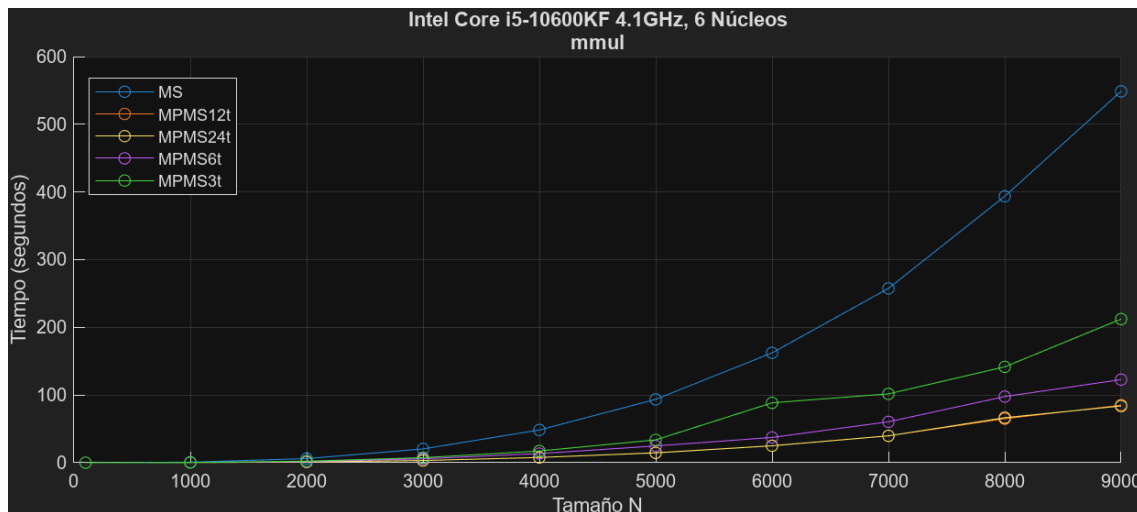


Figura 4.38: Tiempos de ejecución de `mmul` en un Intel Core i5-10600KF

Los tiempos de ejecución de la función `mmul` mostrados en la Figura 4.38, muestran una gran mejora de tiempos respecto al código secuencial. Destaca el uso de 12 y 24 hilos, ya que consiguen los menores tiempos de ejecución.

En cuanto a sus ganancias, la Figura 4.39 permite apreciar una clara curva que describe en la que cuantos más hilos, mayor es la ganancia. Claramente, no aumenta de manera proporcional la ganancia, sino que su incremento se reduce a partir del uso de doce hilos.

Finalmente se puede concluir que en la función `mmul`, cuanto mayor sea el número de hilos, mayor va a ser el SpeedUp; sin embargo, la eficiencia decrece de manera proporcional.

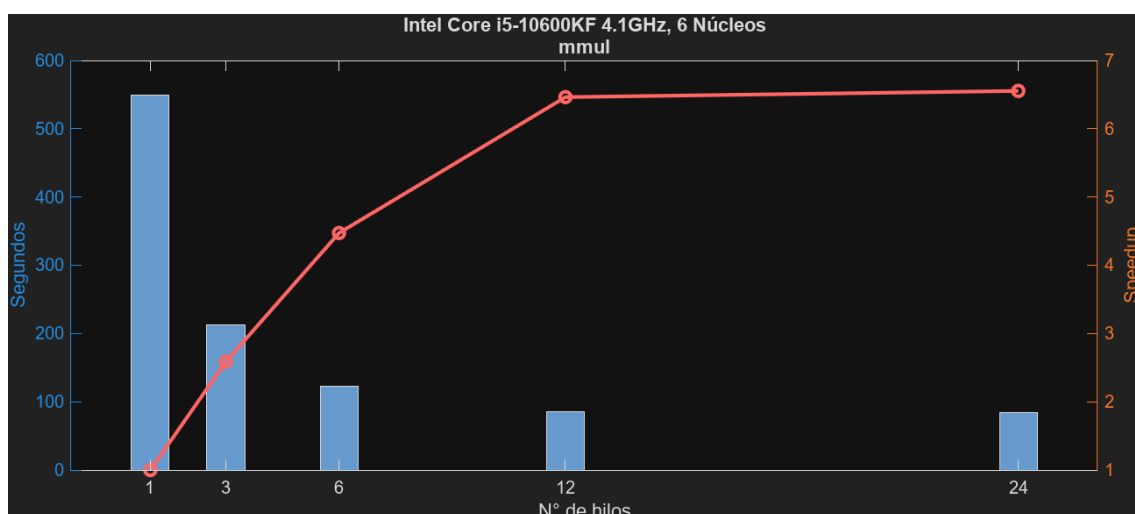


Figura 4.39: Tiempos y ganancias de `mmul` en un Intel Core i5-10600KF

Nº Hilos	SpeedUp	Eficiencia
3	2.59	0.86
6	4.47	0.75
12	6.46	0.54
24	6.55	0.27

Función: `mset`

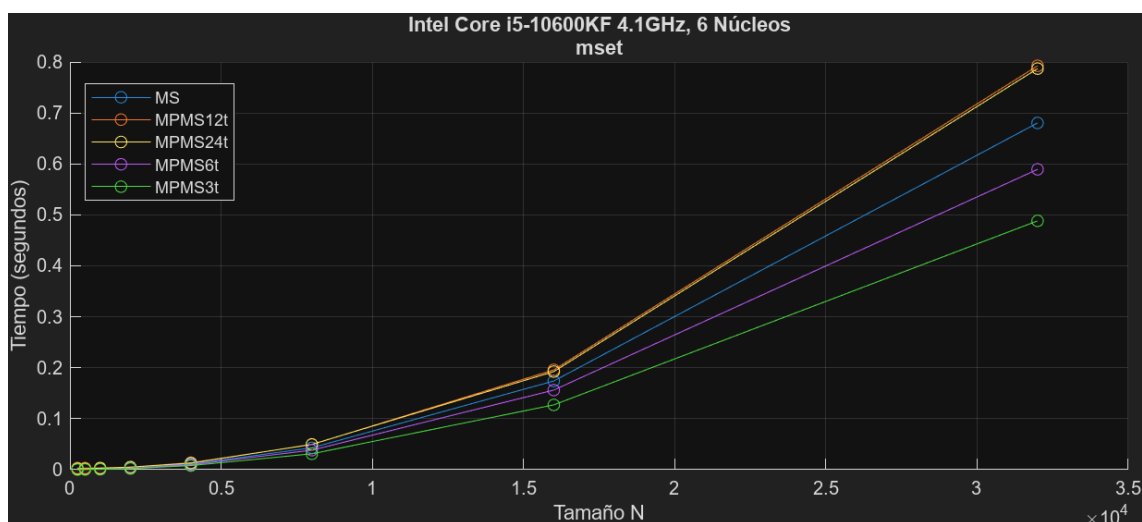


Figura 4.40: Tiempos de ejecución de `mset` en un Intel Core i5-10600KF

En la Figura 4.40 se pueden observar los diferentes tiempos de ejecución obtenidos en la función `mset`. En ella se puede apreciar que algunas versiones paralelas no solo no mejoran el rendimiento respecto a la versión secuencial, sino que lo empeoran. En concreto, utilizando 12 y 24 aumentan los tiempos de

Capítulo 4. Experimentación y Resultados

ejecución, lo cual puede deberse a la arquitectura de las cachés y que dos hilos que comparten núcleo físico se estén reemplazando sus datos mutuamente (*cache thrashing*). Por otro lado, en las versiones que utilizan menos hilos y no dependen del SMT tampoco hay mucha mejora.

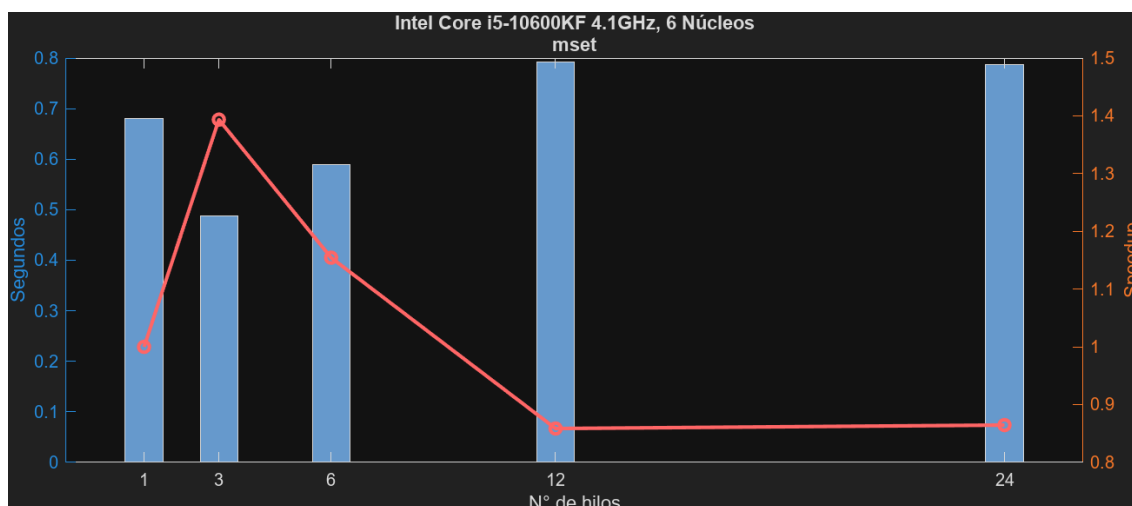


Figura 4.41: Tiempos y ganancias de `mset` en un Intel Core i5-10600KF

Como cabría esperar tras ver sus resultados, las ganancias que muestra la Figura 4.41 son muy leves, consiguiendo el mayor SpeedUp con únicamente tres hilos y decayendo a partir de este número.

Nº Hilos	SpeedUp	Eficiencia
3	1.39	0.46
6	1.15	0.19
12	0.86	0.07
24	0.86	0.04

Se puede concluir la función `mset` afirmando que la mayor ganancia (1.39) y eficiencia por hilo (0.46) se producen únicamente con tres hilos.

Función: `mnorm`

Los tiempos de ejecución de la función `mnorm` se ven reflejados en la Figura 4.42 y en ellos se puede ver una nueva disminución de tiempos al utilizar versiones paralelas, en concreto las de 12 y 24 hilos.

De igual forma, estos resultados se ven reflejados en la Figura 4.43 que grafica sus ganancias. La ganancia aumenta rápidamente con el uso de tres hilos y luego de forma más pausada hasta el uso de 24 hilos.

Como se ha mostrado anteriormente, el mayor SpeedUp (3.08) se logra mediante el uso de 24 hilos, sin embargo, se obtiene la peor eficiencia por hilo (0.13). Por otro lado, se obtiene una ganancia similar (3.05) utilizando 12 hilos y casi duplicando su eficiencia (0.25).

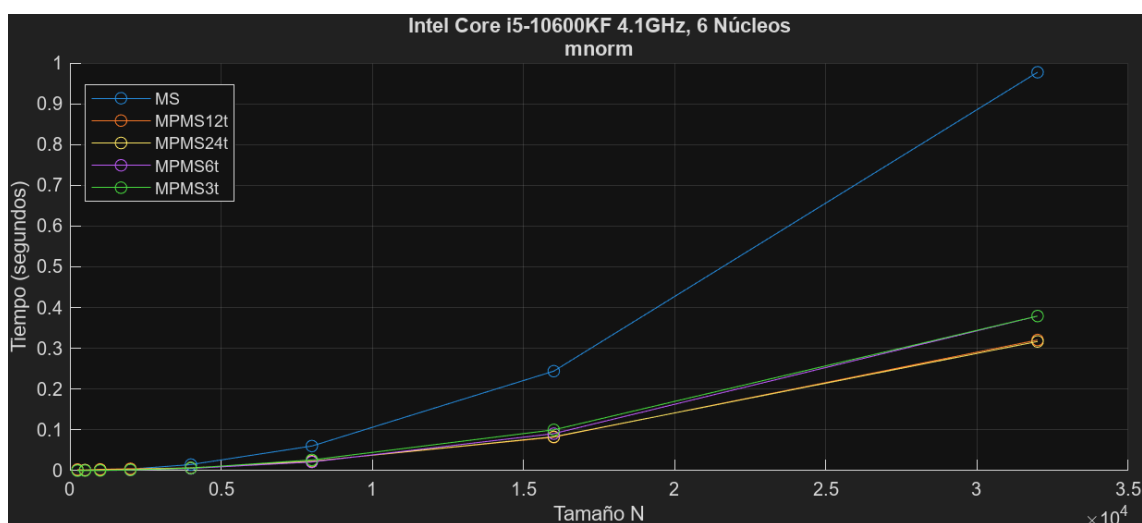


Figura 4.42: Tiempos de ejecución de `mnorm` en un Intel Core i5-10600KF

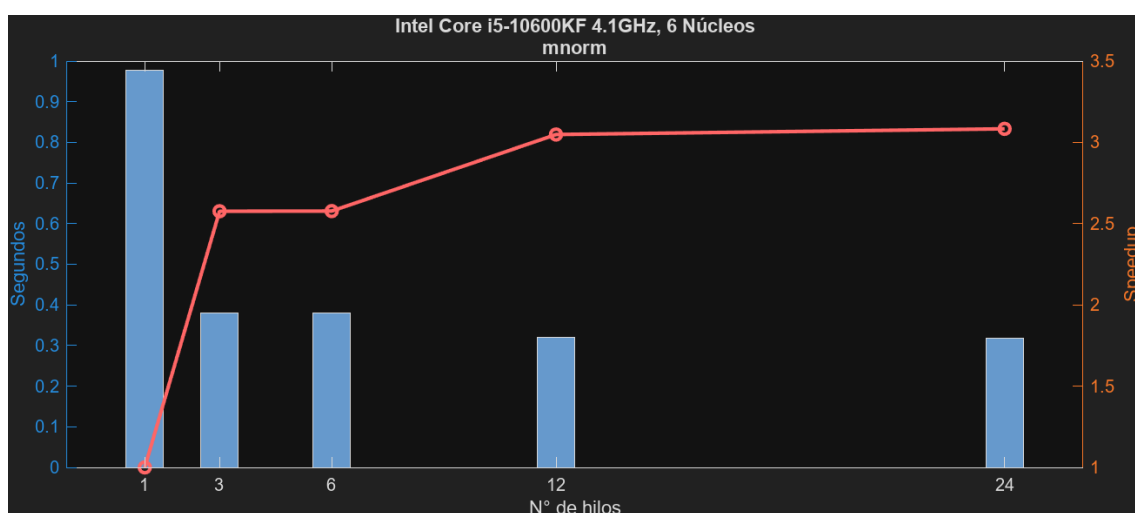


Figura 4.43: Tiempos y ganancias de `mnorm` en un Intel Core i5-10600KF

Nº Hilos	SpeedUp	Eficiencia
3	2.58	0.86
6	2.58	0.43
12	3.05	0.25
24	3.08	0.13

Función: `mmulc`

En cuanto a la función `mmulc`, sus tiempos de ejecución, que se muestran en la Figura 4.44, permiten apreciar una disminución del tiempo de ejecución mediante el uso de versiones paralelas; en concreto, destacan el uso de 3 y 6 hilos, ya que consiguen los menores tiempos de ejecución.

Capítulo 4. Experimentación y Resultados

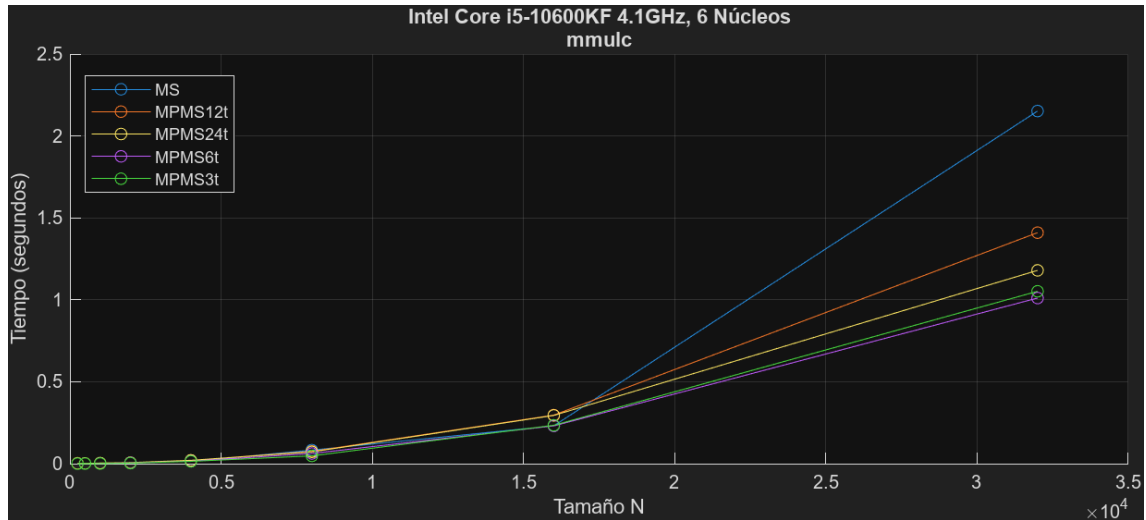


Figura 4.44: Tiempos de ejecución de `mmulc` en un Intel Core i5-10600KF

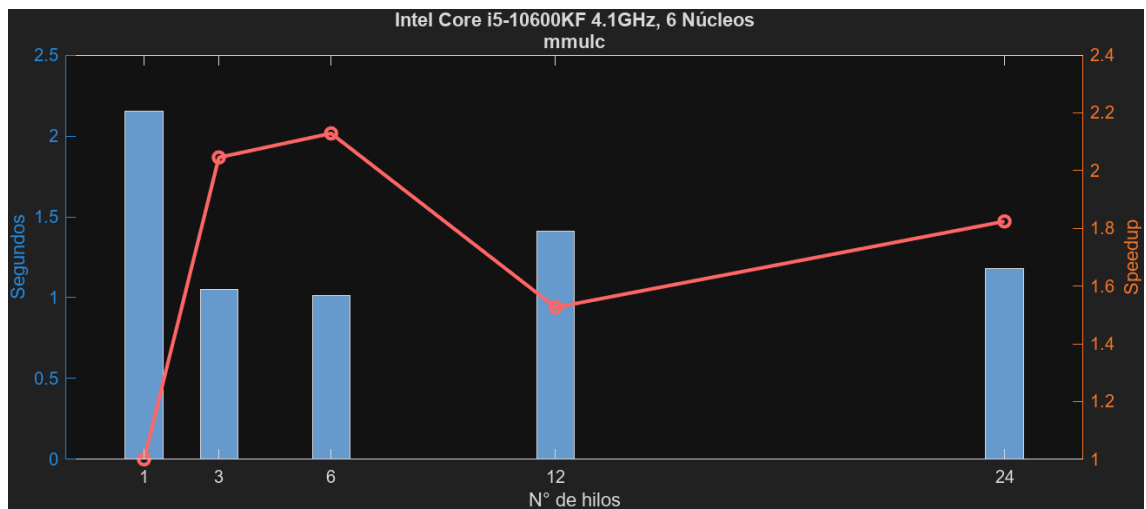


Figura 4.45: Tiempos y ganancias de `mmulc` en un Intel Core i5-10600KF

Las ganancias representadas en la Figura 4.45 muestran una clara curva de incremento hasta los seis hilos, para luego decrecer con el uso de doce y teniendo un ligero repunte con 24 hilos.

Nº Hilos	SpeedUp	Eficiencia
3	2.05	0.68
6	2.13	0.35
12	1.53	0.13
24	1.82	0.08

Como se puede observar, en la función `mmulc` se consigue la mayor ganancia (2.13) mediante el uso de seis hilos, junto con una eficiencia por hilo de (0.35). De nuevo, la eficiencia por hilo decrece a medida que aumenta el número de

hilos, siendo la mayor con solo tres hilos (0.68).

Función: `mcpy`

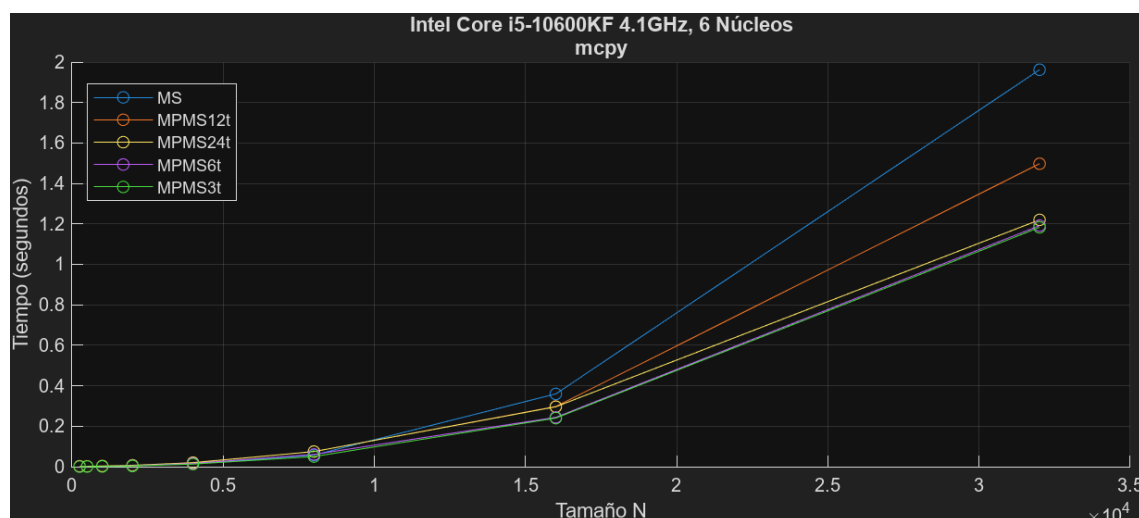


Figura 4.46: Tiempos de ejecución de `mcpy` en un Intel Core i5-10600KF

La Figura 4.46 muestra los tiempos de ejecución de la función `mcpy` con versiones paralelas y secuencial. En ellos se puede observar cierta similitud a los de la función `mmulc`; se consigue cierta mejora respecto al tiempo secuencial, pero muy leve.

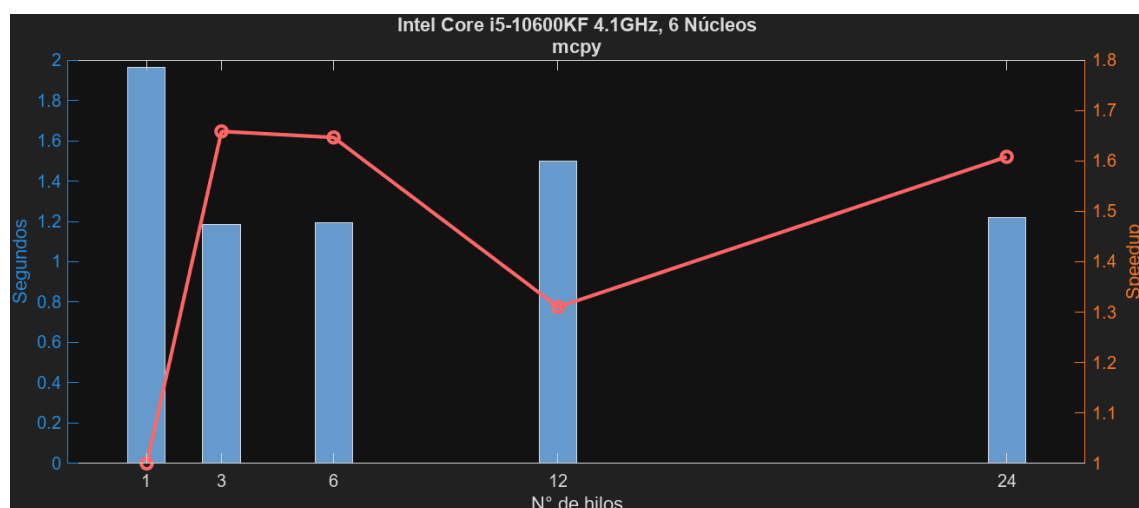


Figura 4.47: Tiempos y ganancias de `mcpy` en un Intel Core i5-10600KF

De la misma forma, las ganancias representadas en la Figura 4.47 siguen un patrón muy similar a las de la función `mmul`. Primero aumenta de forma rápida hasta los tres hilos, para luego decrecer gradualmente hasta los 12 y repuntar a los 24 hilos.

Capítulo 4. Experimentación y Resultados

Nº Hilos	SpeedUp	Eficiencia
3	1.66	0.55
6	1.65	0.27
12	1.31	0.11
24	1.61	0.07

Como se puede apreciar, los SpeedUps en la función `mcpy` son bastante leves y similares, siendo el mayor el de 3 hilos (1.66), muy seguido del resto de versiones.

Función: `msub`

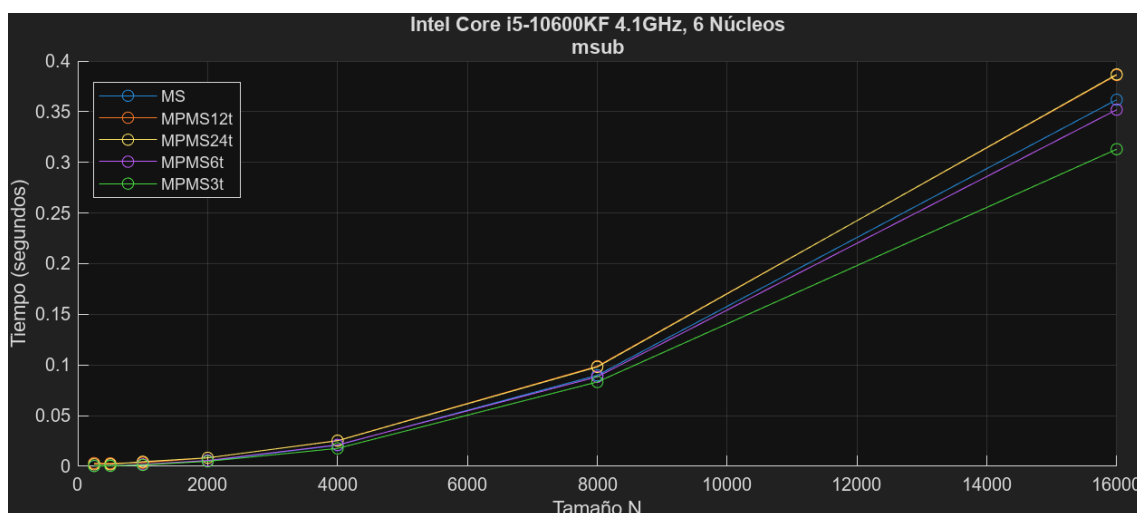


Figura 4.48: Tiempos de ejecución de `msub` en un Intel Core i5-10600KF

En cuanto a los tiempos de ejecución de la última función del tercer entorno, `msub`, representados en la Figura 4.48, es evidente que no consiguen a penas mejoría, incluso utilizando 24 hilos empeoran los resultados de la versión secuencial debido al *overhead*.

Sus ganancias, representadas en la Figura 4.49, llegan a ser incluso menores que 1 utilizando más de seis hilos. De la única forma que se consigue algo de mejora es mediante el uso de tres hilos.

Nº Hilos	SpeedUp	Eficiencia
3	1.16	0.39
6	1.03	0.17
12	0.94	0.08
24	0.94	0.04

Como se ha mencionado antes, se pueden apreciar SpeedUps menores a 1 debido al mal rendimiento de sus versiones.

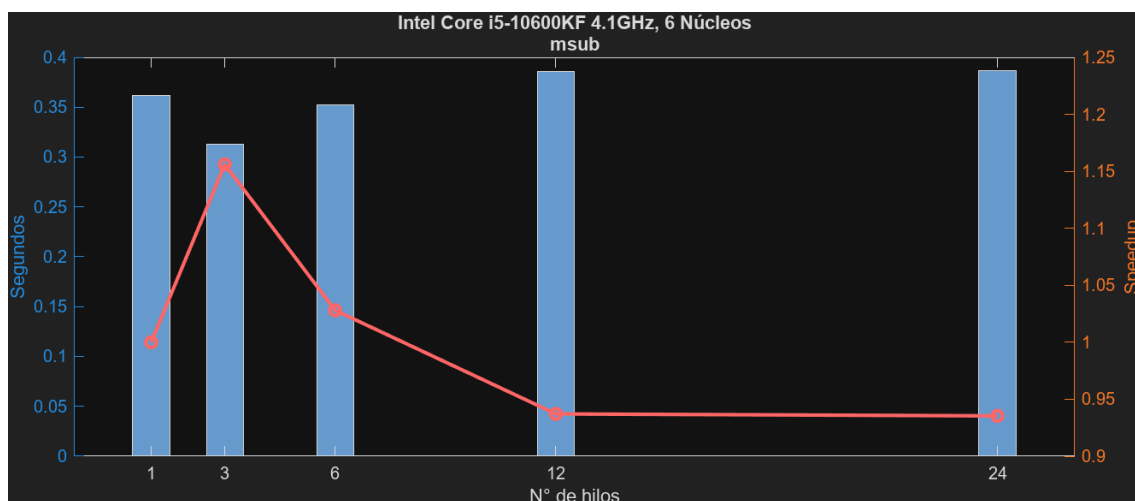


Figura 4.49: Tiempos y ganancias de m_{sub} en un Intel Core i5-10600KF

Rendimiento del Algoritmo

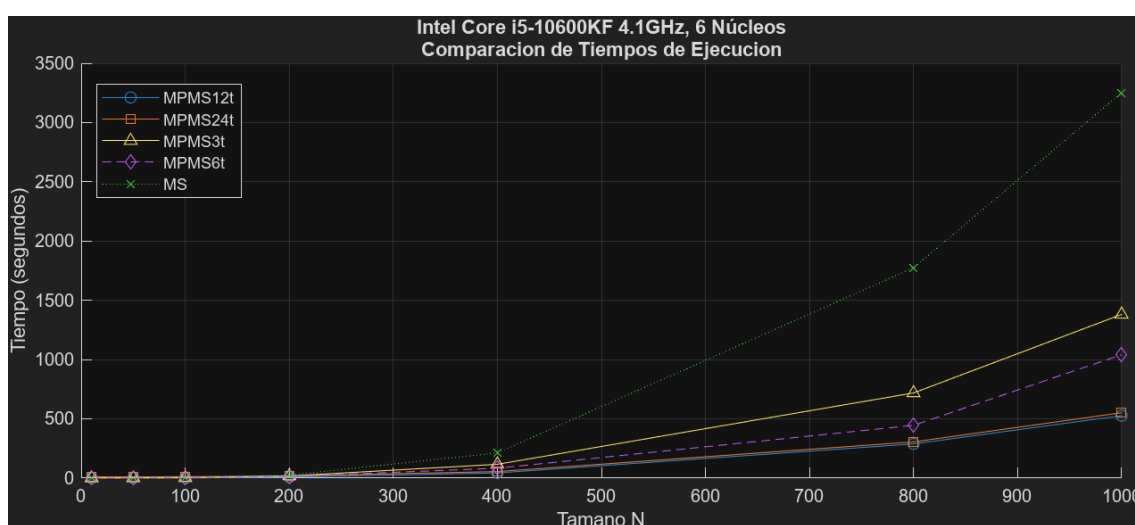


Figura 4.50: Tiempos de ejecución totales en un Intel Core i5-10600KF

En cuanto al rendimiento en conjunto en el algoritmo en este entorno, se puede observar en los tiempos de ejecución presentes en la Figura 4.50 que, a pesar del mal rendimiento de algunas funciones, se consigue mejorar el tiempo de forma notable respecto a la versión secuencial, destacando el uso de 12 y 24 hilos.

Por otro lado, la Figura 4.51 muestra las ganancias obtenidas por las diferentes versiones, en las que se logra un considerable incremento algo disforme hasta los 12 hilos, punto donde alcanza su máximo y comienza a decrecer.

Por último, destacar el SpeedUp obtenido con doce hilos, 6.01, una cifra notoria que va acompañada junto con la mayor eficiencia por hilo, 0.50.

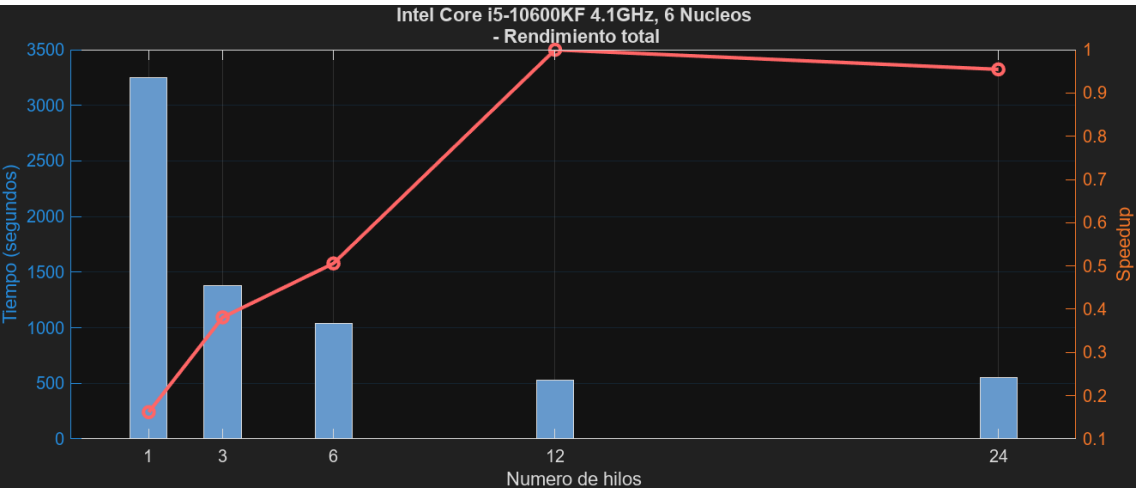


Figura 4.51: Tiempos y ganancias totales en un Intel Core i5-10600KF

Nº Hilos	SpeedUp	Eficiencia
12	6.01	0.50
24	2.64	0.11
6	2.34	0.39
3	1.91	0.64

Observaciones

En este tercer entorno, los resultados obtenidos muestran un comportamiento que confirma varios aspectos esperados del algoritmo paralelizado. Principalmente, se observa una mejora significativa en los tiempos de ejecución en comparación con la implementación secuencial, poniendo de manifiesto la eficacia de la paralelización en reducir los tiempos totales de cómputo.

Por otro lado, la relación entre el número de hilos utilizados y el speedup obtenido no es completamente lineal. Esto puede atribuirse a factores como la sobrecarga de creación y sincronización de hilos, así como al efecto de la jerarquía de la memoria caché, que tiene un impacto más notorio a medida que aumenta la cantidad de datos procesados simultáneamente.

Un aspecto interesante que merece ser destacado es el balance entre eficiencia y escalabilidad. Aunque el speedup alcanza valores elevados con un número moderado de hilos, la eficiencia tiende a decrecer con configuraciones que emplean muchos hilos, lo que sugiere que el algoritmo se ve limitado por factores como la saturación de la memoria compartida o la competencia por los recursos del sistema.

En resumen, este entorno destaca cómo el rendimiento del algoritmo se ve afectado por la arquitectura subyacente y cómo se beneficia de un número óptimo de hilos, proporcionando información valiosa para optimizar configuraciones futuras; además, subraya la importancia de un diseño cuidadoso al implementar algoritmos paralelos.

4.4. Limonero DATSI

Por último, se probará el algoritmo en un servidor perteneciente al Departamento de Arquitectura Tecnología de Sistemas Informáticos (DATSI) de la universidad llamado Limonero. Su arquitectura se basa en Intel Xeon E5-2620 v3 con dos sockets de 6 núcleos cada uno a 2.4GHz.

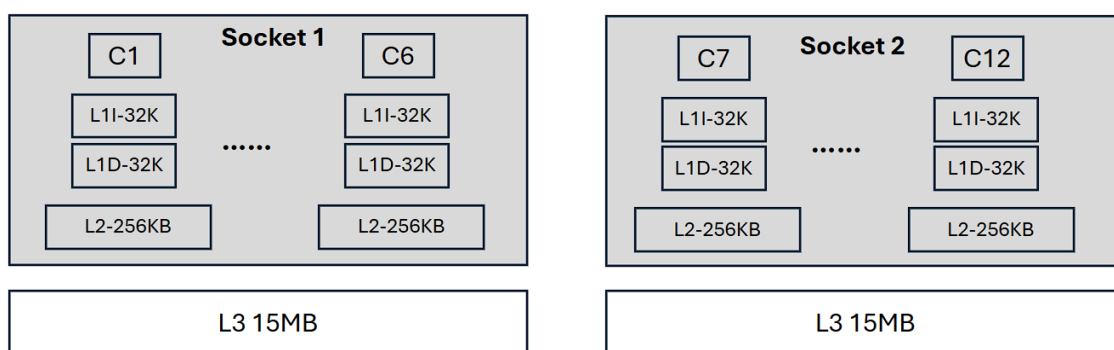


Figura 4.52: Arquitectura de Limonero

A continuación, se va a mostrar su rendimiento en las pruebas de las diferentes funciones de la librería. En este caso, las pruebas se realizarán con 24, 48, 12 y 6 hilos.

Rendimiento de las Funciones

Función: solvels

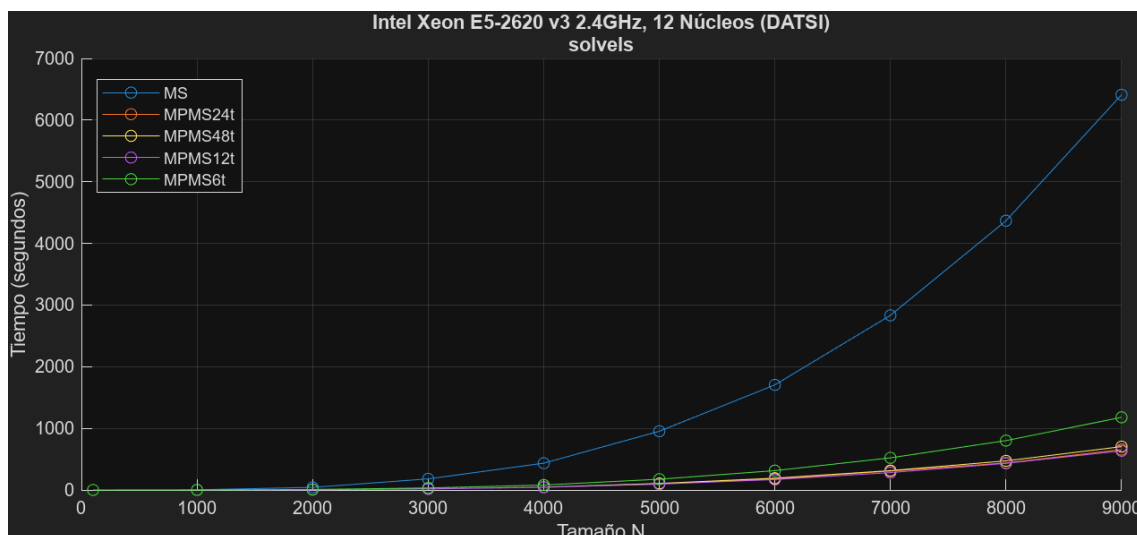


Figura 4.53: Tiempos de ejecución de `sodels` en un Intel Xeon E5-2620 v3

La Figura 4.53 muestra los tiempos de ejecución de las diferentes versiones de la función `sodels` en el tercer entorno. Se puede observar que en este entorno los tiempos siguen una trayectoria uniforme a diferencia de otros entornos. Además,

Capítulo 4. Experimentación y Resultados

en ella se puede apreciar una notable diferencia entre los tiempos de ejecución de la versión secuencial y las versiones paralelas. Obteniendo muy buenos resultados en todas estas versiones, siendo quizás la más alejada la que utiliza únicamente seis hilos.

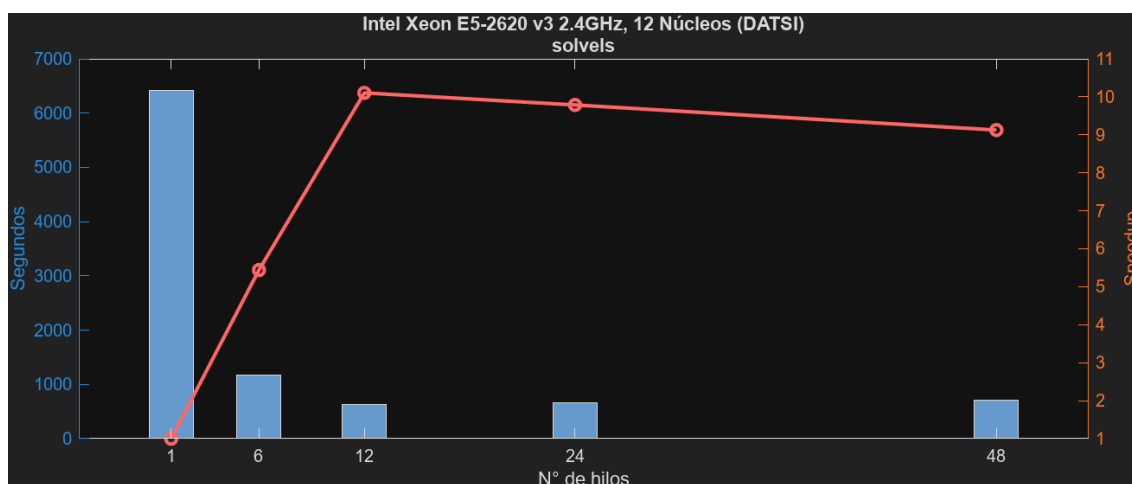


Figura 4.54: Tiempos y ganancias de solvels en un Intel Xeon E5-2620 v3

Se ven representadas sus respectivas ganancias en la Figura 4.54. En ella se muestra un aumento proporcional al número de hilos, el cual se ve estancado a partir de los doce hilos.

Nº Hilos	SpeedUp	Eficiencia
6	5.44	0.91
12	10.10	0.84
24	9.79	0.41
48	9.13	0.19

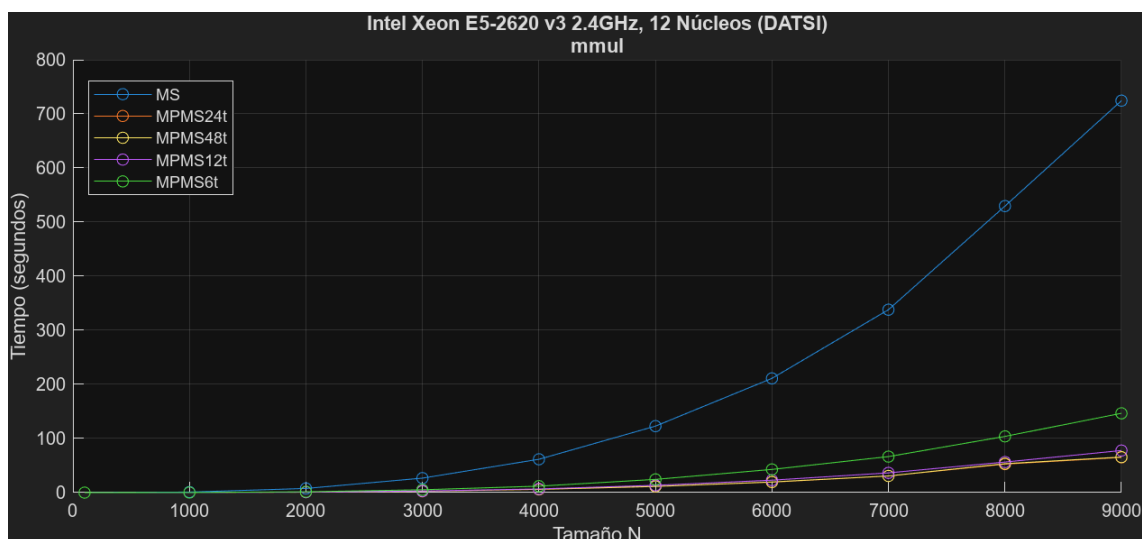
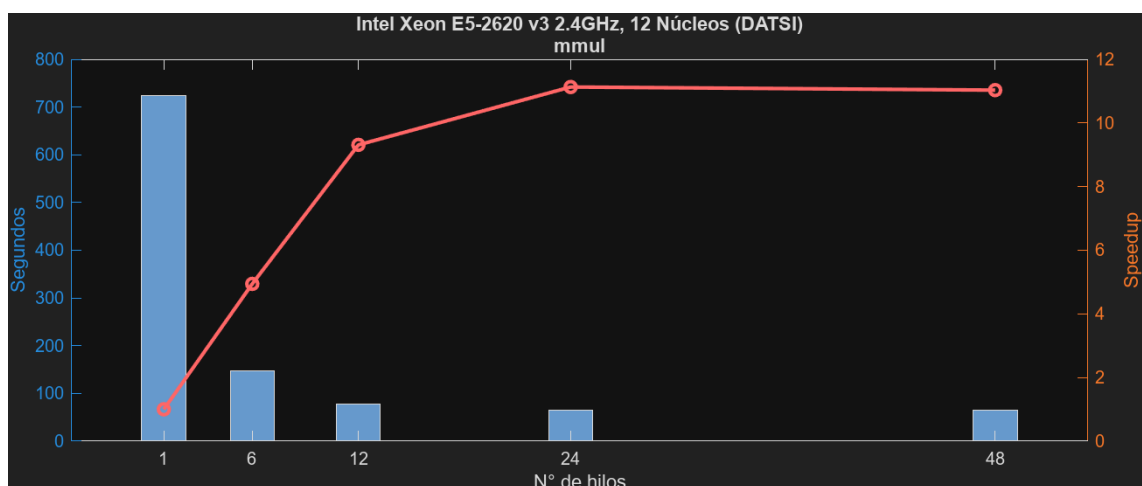
Finalmente, se concluye que el uso de doce hilos consigue el mayor de los SpeedUps (10.10) junto con una muy buena eficiencia por hilo (0.84), únicamente superada por la versión que utiliza seis hilos (0.91).

Función: mmul

En cuanto a la función `mmul` se obtienen unos tiempos de ejecución muy similares a los de la función `solvels`, los cuales se ven representados en la Figura 4.55.

Sus ganancias, las cuales se muestran en la Figura 4.56, siguen una trayectoria progresiva y proporcional que se estanca a partir del uso de 24 hilos.

Para finalizar, se logran las mayores ganancias con el uso de 24 y 48 hilos, 11.13 y 11.03, con unas eficiencias por hilo de 0.46 y 0.23.

Figura 4.55: Tiempos de ejecución de `mmul` en un Intel Xeon E5-2620 v3Figura 4.56: Tiempos y ganancias de `mmul` en un Intel Xeon E5-2620 v3

Nº Hilos	SpeedUp	Eficiencia
6	4.94	0.82
12	9.31	0.78
24	11.13	0.46
48	11.03	0.23

Función: `mset`

Por otro lado, los tiempos de ejecución de la función `mset`, representados en la Figura 4.57, muestran de nuevo una notable mejora tras la paralelización.

En la Figura 4.58, se puede observar fácilmente la curva que generan las ganancias que aumentan rápida y proporcionalmente, viéndose decrementadas

Capítulo 4. Experimentación y Resultados

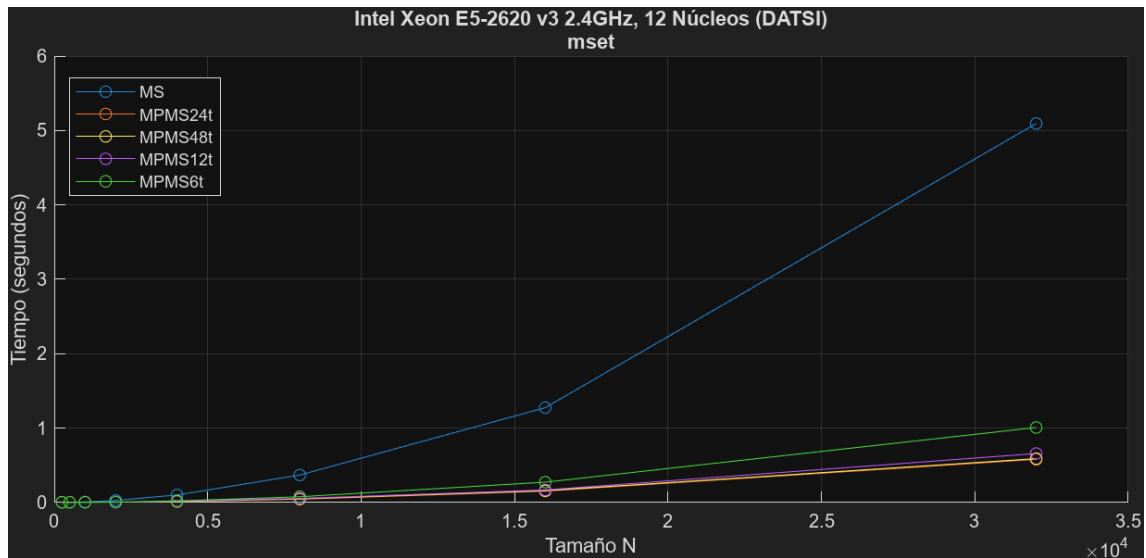


Figura 4.57: Tiempos de ejecución de `mset` en un Intel Xeon E5-2620 v3

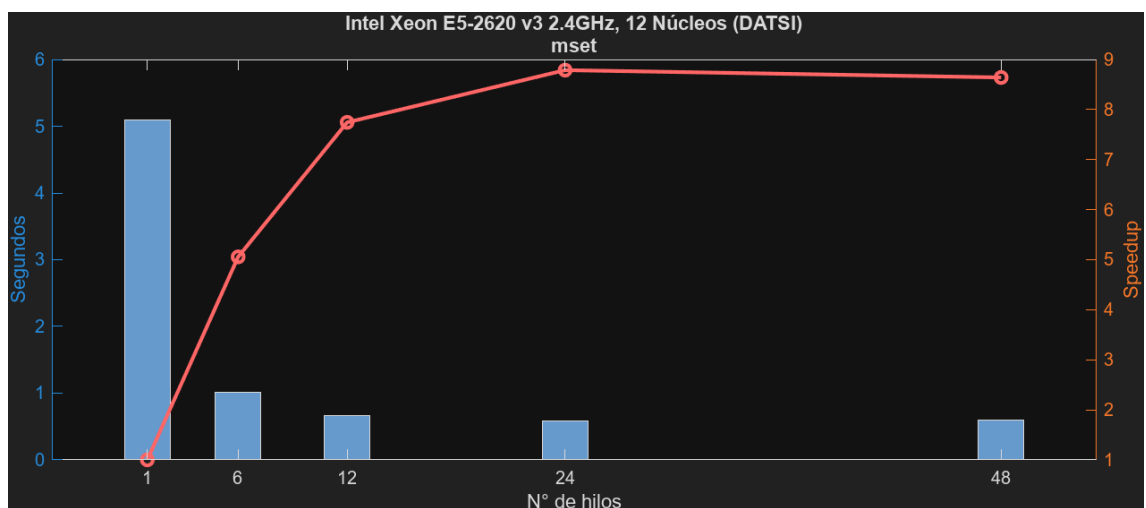


Figura 4.58: Tiempos y ganancias de `mset` en un Intel Xeon E5-2620 v3

levemente a partir del uso de 24 hilos.

Nº Hilos	SpeedUp	Eficiencia
6	5.06	0.84
12	7.75	0.65
24	8.79	0.37
48	8.64	0.18

Se puede concluir que la función `mset` consigue notables ganancias, pero quizás la más destacada sea usando 24 hilos (8.79) muy seguida de su versión con 48 (8.64). Por otro lado, las eficiencias por hilo disminuyen a medida que aumenta el número de hilos.

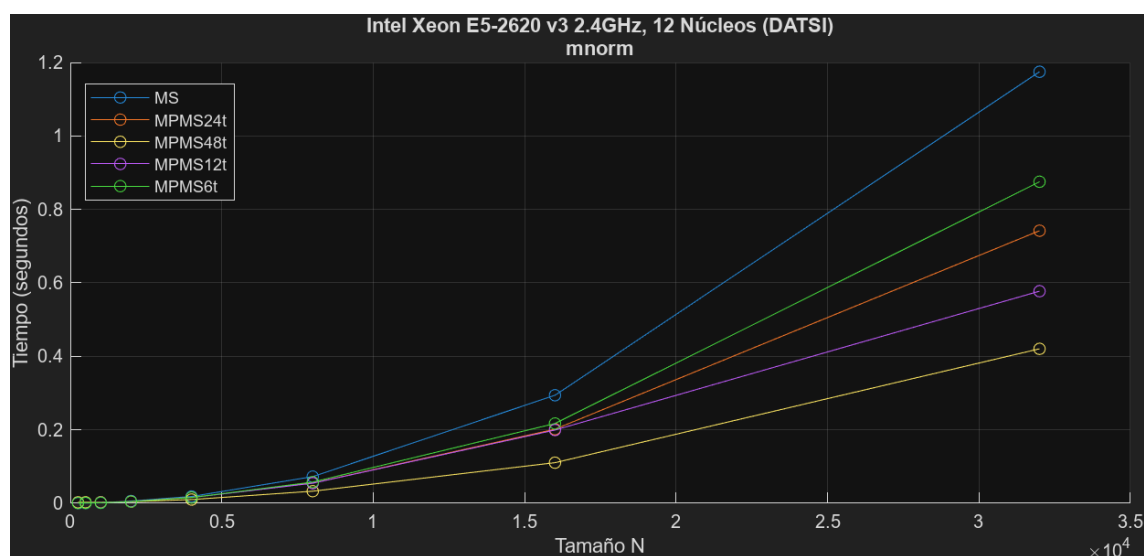
Función: mnorm

Figura 4.59: Tiempos de ejecución de `mnorm` en un Intel Xeon E5-2620 v3

Los tiempos de ejecución de la función `mnorm` mostrados en la Figura 4.59, enseñan un rendimiento bastante inferior al de funciones anteriores; sin embargo, se sigue logrando mejora, sobre todo con el uso de 48 hilos.

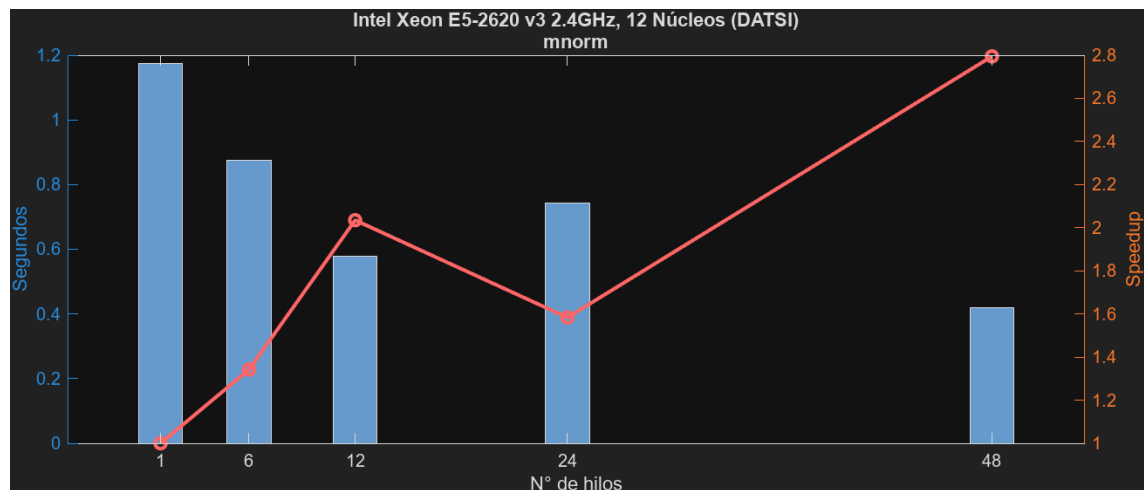


Figura 4.60: Tiempos y ganancias de `mnorm` en un Intel Xeon E5-2620 v3

EN la Figura 4.60, se pueden apreciar unas ganancias un tanto anómalas a las vistas hasta el momento. El SpeedUp aumenta hasta los 12 hilos, momento en el que decremanta con el uso de 24, para finalmente incrementar hasta su punto máximo con 48 hilos. Esto puede deberse principalmente a la arquitectura de las cachés y la diferencia en los datos que utilizan los hilos que comparten núcleo físico. Con 24 hilos podría estar produciéndose cache thrashing y verse solventado utilizando 48 hilos al tener otros datos los hilos que comparten núcleo físico.

Capítulo 4. Experimentación y Resultados

Nº Hilos	SpeedUp	Eficiencia
6	1.34	0.22
12	2.03	0.17
24	1.58	0.07
48	2.79	0.06

Podemos afirmar que la función `mnorm` obtiene unas ganancias un tanto reducidas en comparación al resto de funciones en este entorno, siendo la mayor de ellas la de la versión que utiliza doce hilos (2.03).

Función: `mmulc`

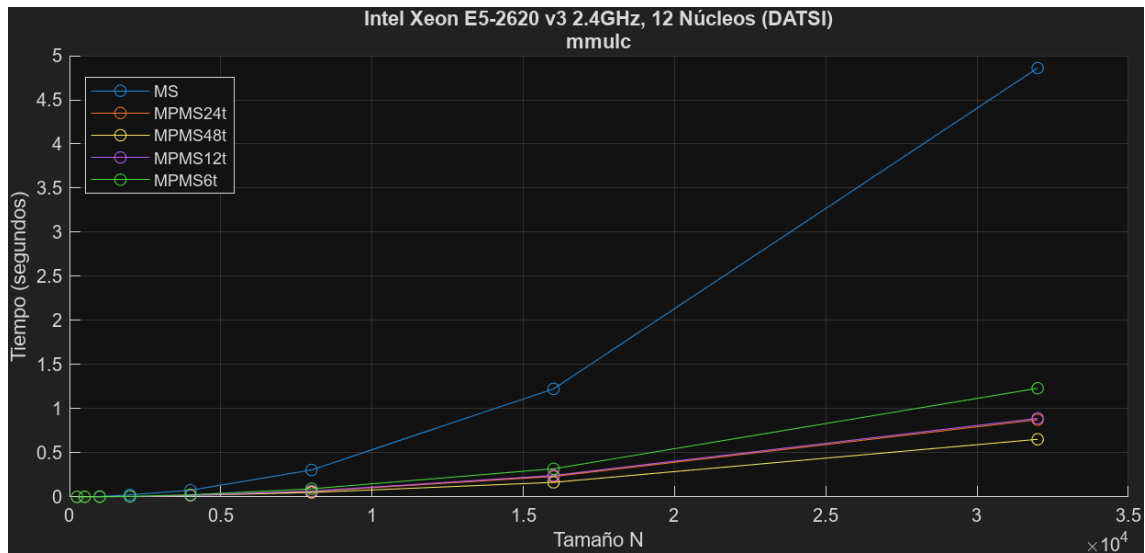


Figura 4.61: Tiempos de ejecución de `mmulc` en un Intel Xeon E5-2620 v3

La `mmulc` vuelve a obtener unos tiempos de ejecución más similares a los que estaba arrojando este entorno en sus otras funciones. Estos tiempos se ven representados en la Figura 4.61, en ella se aprecia una notable mejora de los tiempos de ejecución, especialmente utilizando 48 hilos.

En cuanto a sus ganancias, representadas en la Figura 4.62, únicamente aumentan hasta los 48 hilos, en algunos tramos de forma más rápida (hasta los 12 hilos) y en otros de forma más gradual (a partir de los doce hasta los 48 hilos).

Nº Hilos	SpeedUp	Eficiencia
6	3.95	0.66
12	5.45	0.45
24	5.55	0.23
48	7.43	0.15

Finalmente, se puede afirmar que el mayor de los SpeedUps se consigue me-

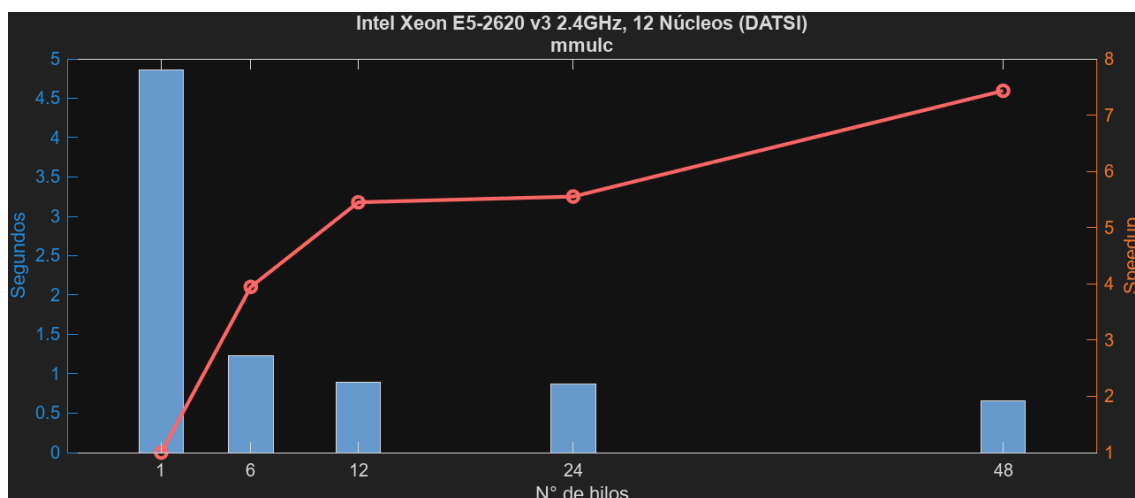


Figura 4.62: Tiempos y ganancias de `mmulc` en un Intel Xeon E5-2620 v3

diante 48 hilos (7.53), aunque de nuevo, con la peor eficiencia por hilo (0.15).

Función: `mcpy`

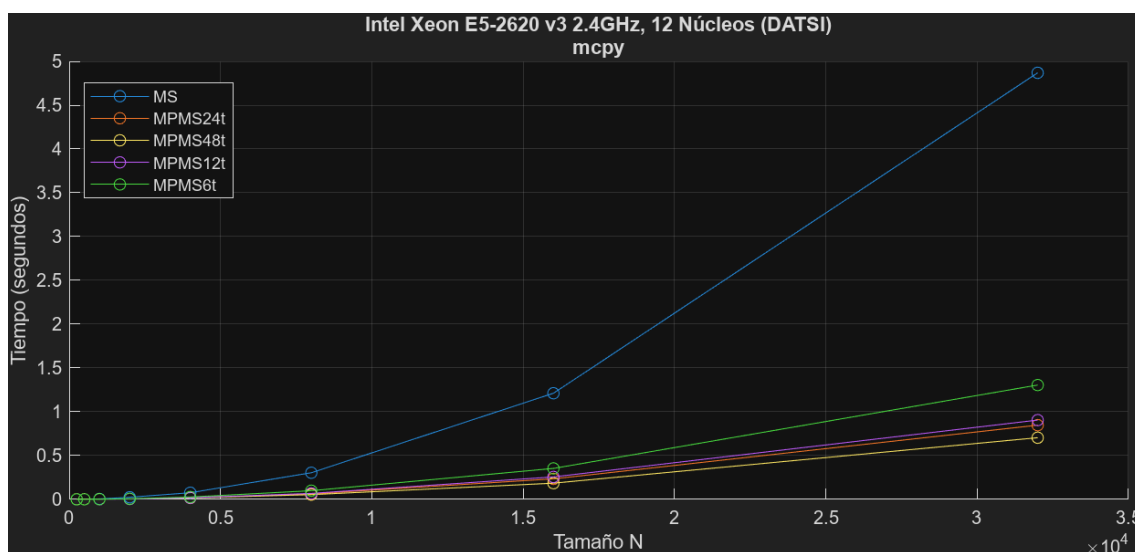


Figura 4.63: Tiempos de ejecución de `mcpy` en un Intel Xeon E5-2620 v3

La Figura 4.63 muestra los tiempos de ejecución de la función `mcpy` tanto en su versión secuencial como en sus versiones paralelas. Se puede observar una notable mejora en las versiones paralelas, destacando el uso de 48 hilos de nuevo.

Por otro lado, se pueden ver sus ganancias en la Figura 4.64, aumentando estas de forma difforme hasta el uso de 48 hilos.

Para concluir, se puede afirmar que, de nuevo, se consigue el mayor rendimiento

Capítulo 4. Experimentación y Resultados

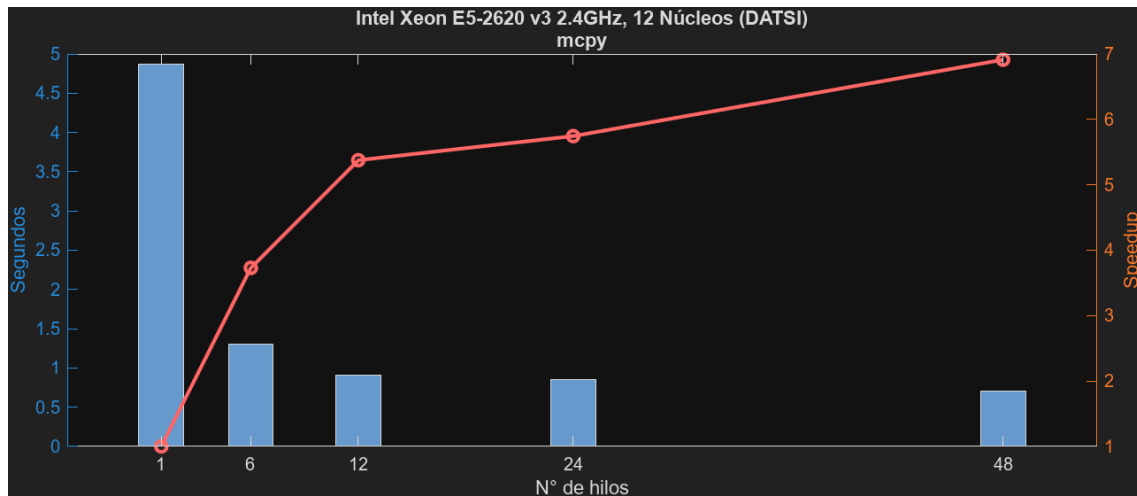


Figura 4.64: Tiempos y ganancias de `mcpy` en un Intel Xeon E5-2620 v3

Nº Hilos	SpeedUp	Eficiencia
6	3.73	0.62
12	5.38	0.45
24	5.75	0.24
48	6.91	0.14

mediante el uso de 48 hilos (6.91); sin embargo, este también consigue la peor de las eficiencias (0.14).

Función: `msub`

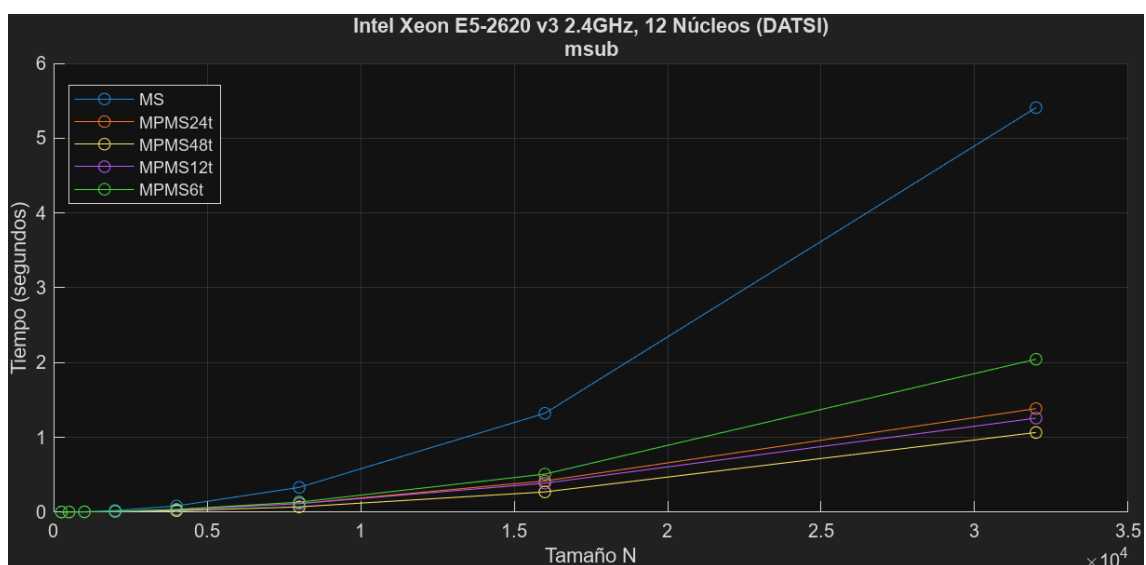


Figura 4.65: Tiempos de ejecución de `msub` en un Intel Xeon E5-2620 v3

Los tiempos de ejecución de la última de las funciones de este entorno, `msub`, quedan representados en la Figura 4.65. En esta se puede apreciar una gran mejora de las versiones paralelas respecto a la secuencial, aunque quizás no tan notable como en otras funciones.

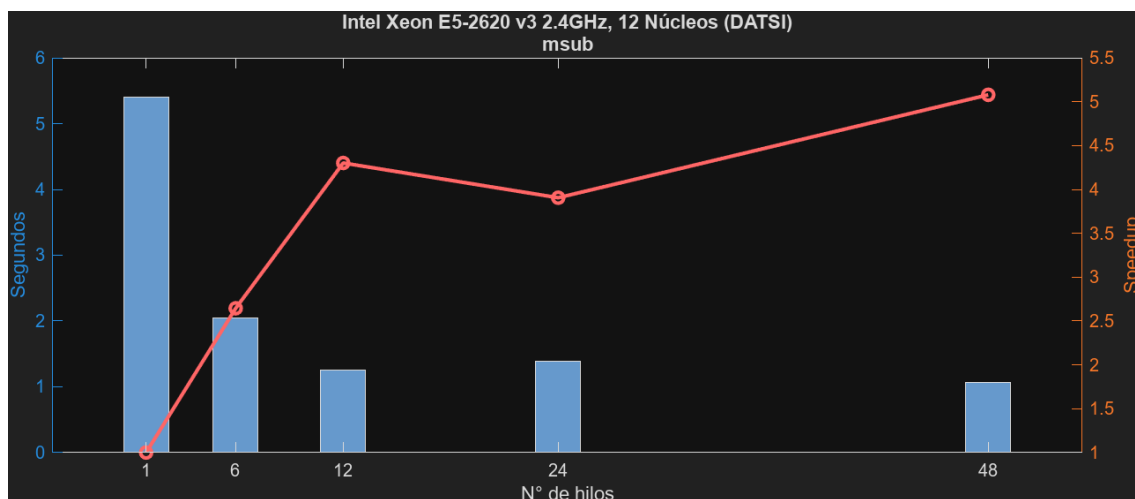


Figura 4.66: Tiempos y ganancias de `msub` en un Intel Xeon E5-2620 v3

Sus respectivas ganancias, presentes en la Figura 4.66, muestran un incremento proporcional al número de hilos, que comienza a decrecer a partir de los doce hilos, repuntando en los 48.

Nº Hilos	SpeedUp	Eficiencia
6	2.65	0.44
12	4.30	0.36
24	3.91	0.16
48	5.08	0.11

Se puede afirmar que la mayor ganancia resulta a partir del uso de 48 hilos (5.08) y la mayor de las eficiencias mediante seis hilos (0.44)

Rendimiento del Algoritmo

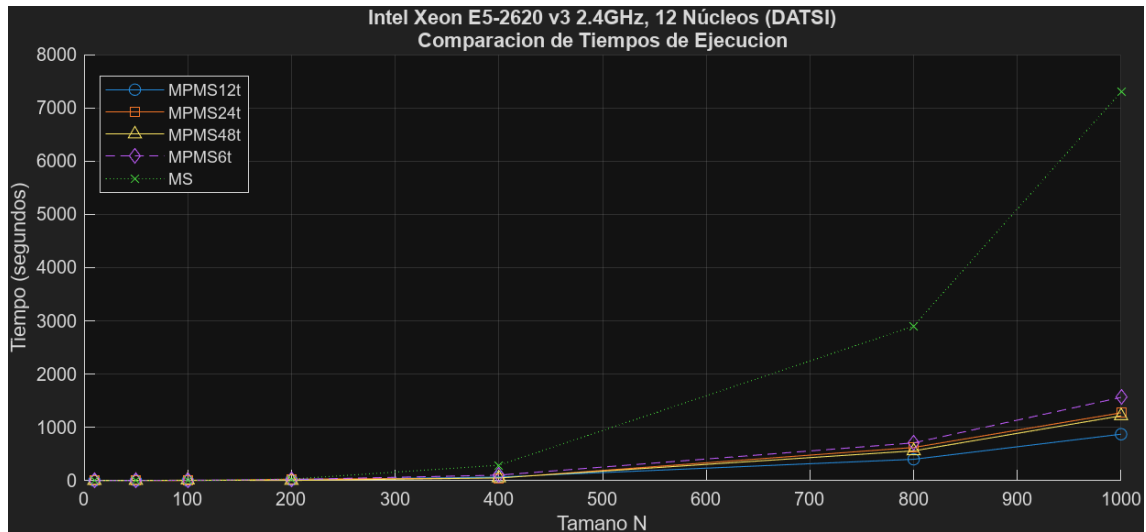


Figura 4.67: Tiempos de ejecución totales en un Intel Xeon E5-2620 v3

En cuanto al rendimiento total del algoritmo utilizando las diferentes versiones de las librerías, se pueden observar sus tiempos de ejecución en la Figura 4.67. En ella se puede destacar una notable diferencia entre los tiempos de la versión secuencial y las paralelas, siendo especialmente inferior el tiempo de ejecución del algoritmo utilizando 12 hilos, muy seguido del resto de versiones paralelas.

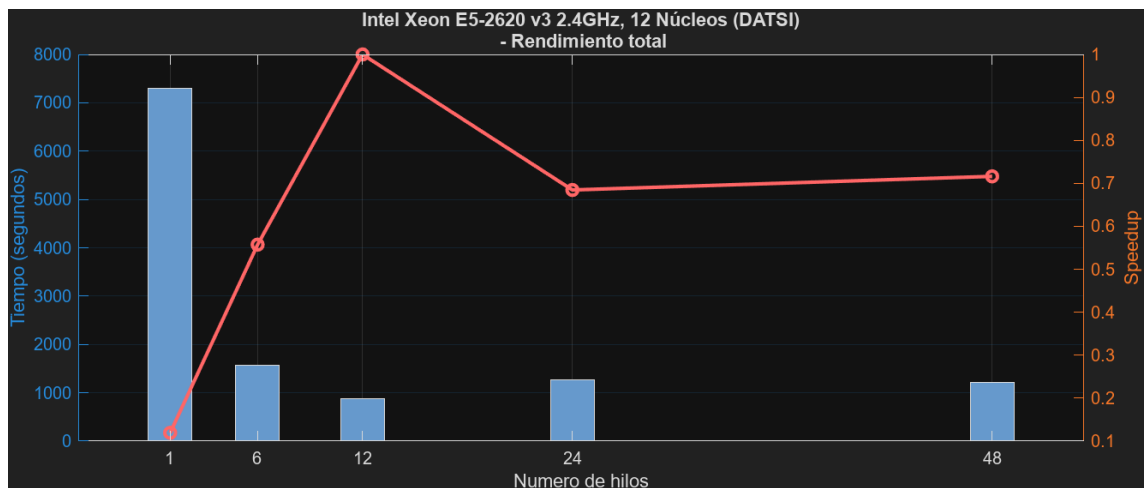


Figura 4.68: Tiempos y ganancias totales en un Intel Xeon E5-2620 v3

La gráfica de ganancias (Figura 4.68) muestra un aumento uniforme de la ganancia, llegando a su punto máximo mediante el uso de doce hilos, decreciendo poco después.

Finalmente, se concluye que el algoritmo obtiene el mayor rendimiento utilizando 12 hilos (SpeedUp de 8.38), con una eficiencia muy buena (0.70) cercana a la máxima (0.78)

Nº Hilos	SpeedUp	Eficiencia
6	4.67	0.78
12	8.38	0.70
24	5.73	0.24
48	6.00	0.13

Observaciones

Tras un breve vistazo, se puede afirmar que esta es la arquitectura que mejores resultados arroja. Las funciones que se veían lastradas por constantes accesos a memoria han visto una notable mejoría, la cual se cree que se debe principalmente a la arquitectura basada en dos sockets que tiene este sistema. Por otro lado, el SMT aquí no solo no consigue ninguna mejoría, sino que consigue empeorar el rendimiento. Esto puede deberse a que, si ambos hilos del mismo núcleo físico requieren los mismos recursos al mismo tiempo, pueden ocasionar bloqueos y no lograr su objetivo de ser más eficientes. En cuanto al rendimiento total, se consigue una ganancia de 8.38 con 12 hilos, teniendo así una eficiencia por hilo de 0.7.

Capítulo 5

Conclusiones

En este trabajo se ha explorado la implementación de un algoritmo para resolver la ecuación diferencial matricial de Riccati mediante procesamiento paralelo, destacando el uso de OpenMP como herramienta de paralelización en sistemas de memoria compartida. Se han desarrollado dos librerías, una secuencial y otra paralela, y se ha evaluado su rendimiento en diversos escenarios experimentales.

Los resultados del trabajo realizado demuestran que la implementación paralela logra un aumento significativo en el rendimiento, aumentando de forma proporcional a los núcleos utilizados, como es de esperar. Sin embargo, se observó que a partir de cierto número de hilos, el speedup tiende a estabilizarse debido a factores como el overhead de sincronización y el uso compartido de memoria. Este número varía en gran medida dependiendo del hardware utilizado, ya que el número de núcleos con los que cuente es esencial y también será muy importante si consigue ser eficiente su SMT o el diseño de las memorias, sobre todo las cachés. En los casos estudiados se ha podido apreciar la importancia de estos factores; en los entornos 1 y 2, los cuales cuentan con procesadores de AMD, el SMT logra mejorar el rendimiento de forma notable, mientras que en los entornos 3 y Limonero el Hyper-Threading (implementación propia de Intel de SMT) no solo no conlleva una mejora, sino que empeora los resultados. Por otro lado, el tercer entorno se ve enormemente lastrado por los accesos a memoria y los fallos en la caché, mientras que Limonero no sufría ninguno de estos problemas.

Este proyecto contribuye al conocimiento del uso de OpenMP en la resolución de ecuaciones matriciales, ofreciendo un análisis detallado de cómo las directivas de paralelización pueden mejorar el rendimiento en problemas rígidos y de gran escala. Aunque no fuese una solución viable paralelizar el bucle principal del algoritmo debido a que sus iteraciones tenían cálculos dependientes entre sí, las optimizaciones aplicadas en las constantes operaciones matriciales que realiza han permitido maximizar el uso de los recursos de hardware disponibles. Con ello se ha conseguido maximizar el rendimiento del algoritmo, a la vez que se estudia la eficiencia de su paralelización en distintos entornos con diferente número de hilos.

En un futuro, sería interesante explorar el uso de técnicas como la de paso de mensajes con MPI, y quizás plantear el desarrollo de enfoques híbridos con OpenMP. También sería atractivo continuar investigando uso de GPUs mediante CUDA para operaciones matriciales aún más eficientes, o incluso aplicar el algoritmo a otras ecuaciones diferenciales rígidas, evaluando su rendimiento en arquitecturas híbridas que combinen CPUs y GPUs.

Capítulo 6

Análisis de impacto

La implementación del algoritmo paralelo para resolver la ecuación diferencial de Riccati tiene un impacto potencial en múltiples áreas relevantes para el desarrollo sostenible. Su capacidad para abordar problemas complejos de optimización lo convierte en una herramienta clave para resolver desafíos técnicos en diversos contextos. A continuación, se analiza su impacto en diferentes ámbitos.

Impacto Médico

El uso de algoritmos optimizados para resolver problemas como la administración precisa de medicamentos permite mejorar tratamientos médicos personalizados. Por ejemplo, en la dosificación de medicamentos para pacientes con condiciones complejas, como diabetes o enfermedades cardíacas, la ecuación diferencial de Riccati puede optimizar el suministro de fármacos en tiempo real, reduciendo efectos secundarios y mejorando la calidad de vida de las personas.

Impacto Empresarial

En el sector empresarial, este trabajo tiene aplicaciones directas en industrias como la automotriz y la aeroespacial. Por ejemplo:

- En **coches autónomos**, la gestión eficiente de las baterías mediante modelos basados en Riccati puede extender su vida útil, mejorar el rendimiento y reducir costes de operación.
- En la **industria aeroespacial**, el algoritmo puede optimizar trayectorias de cohetes y satélites, reduciendo el consumo de combustible y maximizando la eficiencia de las misiones espaciales.

Impacto Social

Desde el punto de vista social, las aplicaciones de la ecuación de Riccati pueden contribuir a resolver problemas globales como:

- **La gestión eficiente de recursos energéticos** en sistemas renovables, lo que garantiza un acceso más equitativo a la energía.

-
- **La mejora de la movilidad urbana** mediante algoritmos que optimicen el uso de flotas de transporte autónomo, reduciendo los tiempos de viaje y mejorando la calidad del aire en las ciudades.

Estas soluciones benefician directamente a las comunidades, promoviendo la equidad y la sostenibilidad.

Impacto Medioambiental

La contribución más significativa de este trabajo al desarrollo sostenible está en su potencial para mitigar el impacto ambiental. Entre sus aplicaciones destacadas se incluyen:

- **Gestión de baterías** en coches eléctricos, reduciendo el consumo de recursos no renovables y las emisiones contaminantes.
- **Optimización de trayectorias de cohetes y drones**, disminuyendo el uso de combustibles fósiles en el sector aeroespacial.
- **Control eficiente de sistemas energéticos renovables**, promoviendo un uso sostenible de la energía solar y eólica.

Al reducir las emisiones y fomentar la eficiencia energética, este trabajo contribuye directamente a los Objetivos de Desarrollo Sostenible (ODS) relacionados con la acción climática y la sostenibilidad energética.

Bibliografía

- [1] Wikipedia contributors. «Ecuación diferencial ordinaria de Riccati — Wikipedia, la enciclopedia libre.» Última modificación: 13 de enero de 2025. (2025), dirección: https://es.wikipedia.org/wiki/Ecuaci%C3%B3n_diferencial_ordinaria_de_Riccati (visitado 13-01-2025).
- [2] H. M. Peter Benner. «BDF Methods for Large-Scale Differential Riccati Equations.» (2009), dirección: https://csc.mpi-magdeburg.mpg.de/mpcsc/benner/pub/BM_mtns04.pdf (visitado 14-10-2024).
- [3] E. A. Jesus Peinado Jacinto J. Ibañez, «Speeding up solving of differential matrix Riccati equations using GPGPU computing and MATLAB,» *Concurrency and Computation: Practice Experience*, vol. 24, págs. 1334-1348, ago. de 2012. DOI: 10.1002/cpe.1835.
- [4] J. Lovón, E. Arias y M. Castillo-Cara, «When performance met energy consumption. Broyden's method: Case study,» vol. 14, págs. 33-38, dic. de 2017. dirección: https://csc.mpi-magdeburg.mpg.de/mpcsc/benner/pub/BM_mtns04.pdf (visitado 09-11-2024).
- [5] R. T. Boštjan Slivnik Borut Robič, «Introduction to Parallel Computing: From Algorithms to Programming on State-of-the-Art Platforms,» 2018. dirección: https://csc.mpi-magdeburg.mpg.de/mpcsc/benner/pub/BM_mtns04.pdf (visitado 14-10-2024).
- [6] *Slides Arquitectura de Computadores, Tema 3. Procesadores ILP*, Presentación en formato de diapositivas, Imágenes extraídas de las diapositivas del tema 3, 2025. dirección: https://www.datsi.fi.upm.es/docencia/Arquitectura_09/privado/ILPS-Abr18B.pdf (visitado 13-01-2025).
- [7] *Slides Arquitectura de Computadores, Tema 2: Sistema de Memoria*, Presentación en formato de diapositivas, Imágenes extraídas de las diapositivas del tema 2, 2025. dirección: https://www.datsi.fi.upm.es/docencia/Arquitectura_09/privado/AC-SM-Feb2018.pdf (visitado 13-01-2025).

Anexos

Apéndice A

Diseño de Pruebas

Para la realización de las pruebas de las librerías MS y MPMS, se ha desarrollado un programa "test.c".^{en} C al que se introducen como parámetros el nombre de la función que se quiere probar y un tamaño N; con esto calculará el tiempo de ejecución de una operación con la respectiva función utilizando matrices de tamaño N.

Programa: test.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <time.h>
5  #include "mpmos.h"
6  #include <omp.h>
7
8  int main(int argc, char *argv[]) {
9      if (argc != 3) {
10         fprintf(stderr, "Número de argumentos erróneo.\n");
11         return 1;
12     }
13
14     // Asignar operación
15     int op = -1;
16     if (strcmp(argv[1], "solves") == 0) op = 0;
17     if (strcmp(argv[1], "mnorm") == 0) op = 1;
18     if (strcmp(argv[1], "mset") == 0) op = 2;
19     if (strcmp(argv[1], "mmul") == 0) op = 3;
20     if (strcmp(argv[1], "mcpy") == 0) op = 4;
21     if (strcmp(argv[1], "mmulc") == 0) op = 5;
22     if (strcmp(argv[1], "msub") == 0) op = 6;
23
24     if (op == -1) {
25         fprintf(stderr, "Operación inválida.\n");
26         return 2;
27     }
28
29     // Leer el tamaño de la matriz desde los argumentos
30     int n = atoi(argv[2]);
31     if (n <= 0) {
```

```

32     fprintf(stderr, "El tamaño de la matriz debe ser un entero positivo.\n"
33     );
34     return 1;
35 }
36
37 // Variables comunes
38 int lda = n, ldb = n, ldc = n, nrhs = 1, info;
39 int *ipiv = NULL;
40 double *A = NULL, *B = NULL, *C = NULL, *WORK = NULL, *D = NULL;
41 char norm, uplo, transa, transb;
42 double alpha, beta, result, constant;
43
44 // Inicializar matrices y parámetros específicos
45 switch (op) {
46     case 0: // solves
47         ipiv = malloc(n * sizeof(int));
48         A = malloc(n * n * sizeof(double));
49         B = malloc(n * n * sizeof(double));
50         if (!A || !B || !ipiv) {
51             fprintf(stderr, "Error al asignar memoria para solves.\n");
52             free(A); free(B); free(ipiv);
53             return 1;
54         }
55         for (int i = 0; i < n * n; i++) A[i] = rand() % 10 + 1;
56         for (int i = 0; i < n * n; i++) B[i] = rand() % 10 + 1;
57         break;
58
59     case 1: // mnrm
60         A = malloc(n * n * sizeof(double));
61         WORK = malloc(n * n * sizeof(double));
62         if (!A || !WORK) {
63             fprintf(stderr, "Error al asignar memoria para mnrm.\n");
64             free(A); free(WORK);
65             return 1;
66         }
67         for (int i = 0; i < n * n; i++) A[i] = rand() % 10 + 1;
68         norm = 'F';
69         break;
70
71     case 2: // mset
72         C = malloc(n * n * sizeof(double));
73         if (!C) {
74             fprintf(stderr, "Error al asignar memoria para mset.\n");
75             free(C);
76             return 1;
77         }
78         uplo = 'A';
79         alpha = 2.0;
80         beta = 5.0;
81         break;
82
83     case 3: // mmul
84         A = malloc(n * n * sizeof(double));
85         B = malloc(n * n * sizeof(double));
86         C = malloc(n * n * sizeof(double));
87         if (!A || !B || !C) {
88             fprintf(stderr, "Error al asignar memoria para mmul.\n");
89             free(A); free(B); free(C);
90         }
91     }
92 }

```

```

89         return 1;
90     }
91     for (int i = 0; i < n * n; i++) {
92         A[i] = rand() % 10 + 1;
93         B[i] = rand() % 10 + 1;
94         C[i] = 0.0;
95     }
96     transa = 'N';
97     transb = 'N';
98     alpha = 1.0;
99     beta = 0.0;
100    break;
101
102    case 4: // mcpy
103        A = malloc(n * n * sizeof(double));
104        C = malloc(n * n * sizeof(double));
105        if (!A || !C) {
106            fprintf(stderr, "Error al asignar memoria para mcpy.\n");
107            free(A); free(C);
108            return 1;
109        }
110        for (int i = 0; i < n * n; i++) A[i] = rand() % 10 + 1;
111        uplo = 'A';
112        break;
113
114    case 5: // mmulc
115        A = malloc(n * n * sizeof(double));
116        D = malloc(n * n * sizeof(double));
117        if (!A || !D) {
118            fprintf(stderr, "Error al asignar memoria para mmulc.\n");
119            free(A); free(D);
120            return 1;
121        }
122        for (int i = 0; i < n * n; i++) A[i] = rand() % 10 + 1;
123        constant = 2.0; // Ejemplo: multiplicar por 2
124        break;
125
126    case 6: // msub
127        A = malloc(n * n * sizeof(double));
128        B = malloc(n * n * sizeof(double));
129        D = malloc(n * n * sizeof(double));
130        if (!A || !B || !D) {
131            fprintf(stderr, "Error al asignar memoria para msub.\n");
132            free(A); free(B); free(D);
133            return 1;
134        }
135        for (int i = 0; i < n * n; i++) {
136            A[i] = rand() % 10 + 1;
137            B[i] = rand() % 10 + 1;
138        }
139        break;
140    }
141
142    // Medir el tiempo de ejecución de la operación seleccionada
143    int nthreads=omp_get_max_threads();
144    // int nthreads=omp_get_max_threads()*2; Doble Threads
145    // int nthreads=omp_get_max_threads()/2; Mitad de Threads
146    // int nthreads=omp_get_max_threads()/4; Un cuarto de Threads

```

Capítulo A. Diseño de Pruebas

```
147     omp_set_num_threads(nthreads);
148     double start = omp_get_wtime();
149
150     switch (op) {
151         case 0: solvels(&n, &nrhs, A, &lدا, ipiv, B, &lدا, &info); break;
152         case 1: result = mnorm(&norm, &n, &n, A, &lدا, WORK); break;
153         case 2: mset(&uplo, &n, &n, &alpha, &beta, C, &lدا); break;
154         case 3: mmul(&transa, &transb, &n, &n, &n, &alpha, A, &lدا, B, &lدا, &
            beta, C, &lدا); break;
155         case 4: mcpy(&uplo, &n, &n, A, &lدا, C, &lدا); break;
156         case 5: mmulc(D, n, A, constant); break;
157         case 6: msub(D, n, A, B); break;
158     }
159
160     double end = omp_get_wtime();
161     double elapsed_time = end - start;
162
163     // Imprimir el tiempo de ejecución
164     fprintf(stdout, "%.6f\n", elapsed_time);
165
166     // Liberar memoria
167     free(A); free(B); free(C); free(WORK); free(ipiv); free(D);
168     return 0;
169 }
```

Listing A.1: Programa para la prueba de funciones

Posteriormente, el script `runTest.sh` se encargará de correr el programa llamando a las diferentes funciones de nuestra librería con múltiples tamaños N y guardando sus resultados.

Script: `runTest.sh`

```
1  !/bin/bash
2  # Archivo de resultados
3  OUTPUT_FILE="res_mos.txt"
4
5  # Crear (o truncar) el archivo de resultados
6  : > "$OUTPUT_FILE"
7
8  # Escribir cabecera
9  echo "Resultados de las pruebas de test.o" >> "$OUTPUT_FILE"
10 echo "===== " >> "$OUTPUT_FILE"
11
12 # Función para obtener los tamanos específicos según la función
13 get_sizes() {
14     case $1 in
15         "solvels") echo "100 1000 2000 3000 4000 5000 6000 7000 8000 9000";;
16         "mmul") echo "100 1000 2000 3000 4000 5000 6000 7000 8000 9000";;
17         "mcpy") echo "250 500 1000 2000 4000 8000 16000 32000";;
18         "mset") echo "250 500 1000 2000 4000 8000 16000 32000";;
19         "mnorm") echo "250 500 1000 2000 4000 8000 16000 32000";;
20         "mmulc") echo "250 500 1000 2000 4000 8000 16000 32000";;
21         "msub") echo "250 500 1000 2000 4000 8000 16000 32000";;
22         *) echo ""; return 1;;
23     esac
24     return 0
25 }
```

```

25 }
26
27 # Lista de funciones a probar
28 FUNCTIONS=("solvels" "mmul" "mcpy" "mset" "mnorm" "mmulc" "msub")
29
30 # Iterar sobre las funciones
31 for FUNC in "${FUNCTIONS[@]}; do
32     echo -e "\nFunción: $FUNC" >> "$OUTPUT_FILE"
33     echo "N          Tiempo (segundos)" >> "$OUTPUT_FILE"
34     echo "-----" >> "$OUTPUT_FILE"
35
36     # Obtener tamanos específicos para la función
37     SIZES=$(get_sizes "$FUNC")
38     if [ $? -ne 0 ]; then
39         echo "Error al obtener tamanos para la función $FUNC."
40         continue
41     fi
42
43     # Iterar sobre los tamanos
44     for N in $SIZES; do
45         # Ejecutar el programa y capturar el tiempo
46         TIME=$(./test.o "$FUNC" "$N")
47
48         # Verificar si la ejecución fue exitosa
49         if [ $? -eq 0 ]; then
50             echo "$N      $TIME" >> "$OUTPUT_FILE"
51         else
52             echo "$N      ERROR" >> "$OUTPUT_FILE"
53         fi
54     done
55 done
56
57 # Mensaje final
58 echo "Pruebas completadas. Resultados guardados en $OUTPUT_FILE."

```

Listing A.2: Script para correr pruebas con test.o

Por último, el script `correrVersiones.sh` se encargará de ejecutar y calcular el tiempo de ejecución del algoritmo por completo utilizando ambas librerías y múltiples tamaños de problema.

Script: `correrVersiones.sh`

```

1  #!/bin/bash
2  # Directorios de las versiones del proyecto
3  declare -A versiones=(
4      ["MS"]="/home/borja/tfg/mos"
5      ["MSMP16t"]="/home/borja/tfg/mpmos"
6      ["MSMP32t"]="/home/borja/tfg/mpmos"
7      ["MSMP8t"]="/home/borja/tfg/mpmos"
8      ["MSMP4t"]="/home/borja/tfg/mpmos"
9  )
10
11 # Valores de N
12 valores_N=(10 50 100 200 400 800 1000)
13
14 # Archivo de salida

```

Capítulo A. Diseño de Pruebas

```
15 archivo_salida="tiempos.txt"
16
17 # Limpiar archivo de salida
18 > "$archivo_salida"
19
20 # Escribir cabecera
21 echo -e "Versión\tN\tTiempo (s)" >> "$archivo_salida"
22
23 # Recorrer las versiones
24 for version in "${!versiones[@]}"; do
25     directorio="${versiones[$version]}"
26     ejecuta_script="$directorio/ejecuta"
27
28     # Configurar el número de threads según la versión
29     case "$version" in
30         "MS")
31             export OMP_NUM_THREADS=1
32             ;;
33         "MSMP16t")
34             export OMP_NUM_THREADS=16
35             ;;
36         "MSMP32t")
37             export OMP_NUM_THREADS=32
38             ;;
39         "MSMP8t")
40             export OMP_NUM_THREADS=8
41             ;;
42         "MSMP4t")
43             export OMP_NUM_THREADS=4
44             ;;
45         *)
46             export OMP_NUM_THREADS=16
47             ;;
48     esac
49
50     # Verificar que el script ejecuta existe y es ejecutable
51     if [[ -x "$ejecuta_script" ]]; then
52         for N in "${valores_N[@]}"; do
53             # Cambiar al directorio del script
54             pushd "$directorio" > /dev/null
55
56             # Ejecutar el script con el valor de N
57             tiempo=$( "$ejecuta_script" "$N" 2>/dev/null)
58
59             # Volver al directorio original
60             popd > /dev/null
61
62             # Verificar si se obtuvo un tiempo válido
63             if [[ $? -eq 0 && ! -z "$tiempo" ]]; then
64                 echo -e "${version}\t${N}\t${tiempo}" >> "$archivo_salida"
65             else
66                 echo -e "${version}\t${N}\tError" >> "$archivo_salida"
67             fi
68         done
69     else
70         echo "El script 'ejecuta' de la versión '$version' no se encuentra o no es
71         ejecutable." >> "$archivo_salida"
72     fi
```

```
72  
73 done  
74  
75 echo "Resultados guardados en $archivo_salida."
```

Listing A.3: Script para la prueba de versiones del arlgoritmo

Apéndice B

Procesado de Resultados MATLAB

Este anexo se va a centrar en cómo se han pasado los resultados obtenidos en las pruebas a una representación más visual. Como habéis podido observar, todos los scripts y programas que se han mostrado generan ficheros de texto que almacenan los resultados. Los datos de este modo son engorrosos y difíciles de entender, por lo que se han desarrollado varios scripts para graficarlos visualmente en MATLAB. De este modo se busca tanto una mejor forma para mostrarlos, como para poder entenderlos y estudiarlos. A continuación, se van a mostrar estos scripts, contando su uso, pero sin entrar en cómo es su funcionamiento.

Script: procesarResultados.m

El primero de los scripts procesa los ficheros de texto correspondientes a los resultados de las librerías y genera una gráfica por cada función de la librería en la que dibuja sus diferentes tiempos de ejecución.

```
1 %maquina="Intel Xeon E5-2620 v3 2.4GHz, 12 Núcleos (DATSI)";
2 maquina="AMD Ryzen 7 PRO 8840HS 3.3GHz, 8 Núcleos";
3 %maquina="AMD Ryzen 7 5800H 3.2GHz, 8 Núcleos";
4 %maquina="Intel Core i5-10600KF 4.1GHz, 6 Núcleos"
5 % Archivos de entrada
6 archivos = {
7     'res_mos.txt', 'MS';
8     'res_mpmos.txt', 'MPMS_16t';
9     'res_mpmos_dt.txt', 'MPMS_32t';
10    'res_mpmos_ht.txt', 'MPMS_8t';
11    'res_mpmos_qt.txt', 'MPMS_4t'
12 };
13 };
14
15 % Funciones a analizar
16 funciones = {'solvels', 'mmul', 'mcpy', 'mset', 'mnorm', 'mmulc', 'msub',};
17
18 datos = struct();
19
```

```

20 % Leer datos
21 for i = 1:size(archivos, 1)
22     archivo = archivos{i, 1};
23     etiqueta = archivos{i, 2};
24     fid = fopen(archivo, 'r');
25     contenido = textscan(fid, '%s', 'Delimiter', '\n');
26     fclose(fid);
27
28     contenido = contenido{1};
29
30     for j = 1: numel(funciones)
31         funcion = funciones{j};
32         idx = find(contains(contenido, ['Función: ' funcion]));
33         if ~isempty(idx)
34             % Extraer N y tiempos
35             inicio = idx + 2;
36             fin = inicio + find(cellfun(@isempty, contenido(inicio:end)) | ...
37                                 contains(contenido(inicio:end), 'Función:'), 1,
38                                     'first') - 2;
39
40             % Última función
41             if isempty(fin)
42                 fin = numel(contenido);
43             end
44
45             datos_temp = contenido(inicio:fin);
46
47             % Convertir líneas a matriz
48             matriz = [];
49             for k = 1: numel(datos_temp)
50                 linea = datos_temp{k};
51
52                 % Ignorar separadores
53                 if contains(linea, '-----')
54                     continue;
55                 end
56
57                 % Procesar líneas válidas
58                 valores = sscanf(linea, '%f %f');
59                 if numel(valores) == 2
60                     matriz = [matriz; valores'];
61                 else
62                     fprintf('Línea malformada en %s para %s: %s\n', etiqueta,
63                             funcion, linea);
64                 end
65             end
66
67             % Guardar
68             if ~isfield(datos, funcion)
69                 datos.(funcion) = struct();
70             end
71             datos.(funcion).(etiqueta) = matriz;
72         end
73     end
74
75 % Generar gráficas
76 for i = 1: numel(funciones)

```

Capítulo B. Procesado de Resultados MATLAB

```
76     funcion = funciones{i};
77     figure('Name', funcion);
78     hold on;
79     etiquetas = fieldnames(datos.(funcion));
80
81     for j = 1:numel(etiquetas)
82         etiqueta = etiquetas{j};
83         valores = datos.(funcion).(etiqueta);
84         if size(valores, 2) == 2
85             plot(valores(:, 1), valores(:, 2), '-o', 'DisplayName', etiqueta);
86         else
87             warning('Los datos para %s en %s no tienen el formato esperado.',
88                     etiqueta, funcion);
89         end
90     end
91
92     title([maquina funcion]);
93     xlabel('Tamano N');
94     ylabel('Tiempo (segundos)');
95     legend('Location', 'northwest');
96     grid on;
97     hold off;
98 end
```

Listing B.1: Script de MATLAB para procesar resultados de librerías

Script: graficarTiempos.m

El segundo script recoge los resultados de la ejecución del algoritmo utilizando ambas librerías y los dibuja en una gráfica.

```
1  %maquina="Intel Xeon E5-2620 v3 2.4GHz, 12 Núcleos (DATSI)";
2  maquina="AMD Ryzen 7 PRO 8840HS 3.3GHz, 8 Núcleos";
3  %maquina="AMD Ryzen 7 5800H 3.2GHz, 8 Núcleos";
4  %maquina="Intel Core i5-10600KF 4.1GHz, 6 Núcleos"
5  % Leer el archivo linea por linea
6  fid = fopen('tiempos.txt', 'r');
7  if fid == -1
8      error('No se pudo abrir el archivo tiempos.txt.');
```

```
9  end
10
11  % Leer encabezado
12  header = fgetl(fid); % Leer la primera linea como encabezado
13  header = strrep(header, 'Versión', 'Version'); % Sustituir caracteres
    especiales
14  header = strrep(header, 'Tiempo (s)', 'Tiempo'); % Eliminar parentesis y
    caracteres especiales
15
16  % Leer datos
17  data = textscan(fid, '%s %d %f', 'Delimiter', '\t'); % Leer columnas: Version,
    N, Tiempo
18  fclose(fid);
19
20  % Crear tabla
21  tabla = table(data{1}, data{2}, data{3}, 'VariableNames', {'Version', 'N', '
    Tiempo'});
22
```

```

23 versiones_unicas = unique(tabla.Version);
24
25 figure;
26 hold on;
27
28 colores = {'-o', '-s', '-^', '--d', ':x'}; % Ajusta estilos segun las versiones
29
30 for i = 1:length(versiones_unicas)
31     version_actual = versiones_unicas{i};
32
33     % Filtrar datos para la version actual
34     filtro = strcmp(tabla.Version, version_actual);
35     tamanos_filtrados = tabla.N(filtro);
36     tiempos_filtrados = tabla.Tiempo(filtro);
37
38     % Graficar
39     plot(tamanos_filtrados, tiempos_filtrados, colores{i}, 'DisplayName',
40          version_actual);
41
42 end
43
44 % Configuracion grafica
45 title([maquina 'Comparacion de Tiempos de Ejecucion']); % Sin tildes para
46     evitar errores
47 xlabel('Tamano N'); % Sin tildes para evitar errores
48 ylabel('Tiempo (segundos)');
49 legend('Location', 'northwest');
50 grid on;

```

Listing B.2: Script de MATLAB para graficar tiempos de ejecucion desde "tiempos.txt"

Script: calcularGanancias.m

Utilizamos este script para calcular la ganancia y eficiencia en cada una de las pruebas de las funciones utilizando distinto número de hilos.

```

1 files = {
2     'res_mos.txt',
3     'res_mpmos_gt.txt',
4     'res_mpmos_ht.txt',
5     'res_mpmos.txt',
6     'res_mpmos_dt.txt'
7 };
8
9 % Número de hilos utilizados en cada configuración
10 %threads = [1, 4, 8, 16, 32];
11 threads = [1, 6, 12, 24, 48];
12 %threads = [1, 3, 6, 12, 24];
13
14 % Archivo de salida
15 output_file = 'ganancias.txt';
16 fid_out = fopen(output_file, 'w');
17 if fid_out == -1
18     error('No se pudo abrir el archivo de salida: %s', output_file);
19 end
20
21 % Leer los datos

```

Capítulo B. Procesado de Resultados MATLAB

```
22 data = cell(size(files));
23 for i = 1:length(files)
24     data{i} = leer_resultados(files{i});
25 end
26
27 % Obtener las funciones
28 funciones = fieldnames(data{1}); % Basado en el archivo secuencial
29
30 % Crear tablas de resultados
31 for f = 1:length(funciones)
32     funcion = funciones{f};
33
34     fprintf(fid_out, 'Resultados para la función: %s\n', funcion);
35     fprintf(fid_out, '%-10s %-15s %-15s\n', 'Hilos', 'Speedup', 'Eficiencia');
36
37     tiempo_base = data{1}.(funcion).tiempos(end);
38
39     % Comparar con cada archivo
40     for i = 2:length(files)
41         tiempos_paralelos = data{i}.(funcion).tiempos;
42         tiempo_paralelo = tiempos_paralelos(end); % Último tamaño
43
44         % Calcular speedup y eficiencia
45         speedup = tiempo_base / tiempo_paralelo;
46         eficiencia = speedup / threads(i);
47
48         fprintf(fid_out, '%-10d %-15.2f %-15.2f\n', threads(i), speedup,
49             eficiencia);
50     end
51     fprintf(fid_out, '\n');
52 end
53
54 fclose(fid_out);
55 fprintf('Resultados almacenados en %s\n', output_file);
56
57 % Leer resultados
58 function resultados = leer_resultados(filename)
59     fid = fopen(filename, 'r');
60     if fid == -1
61         error('No se pudo abrir el archivo: %s', filename);
62     end
63
64     lines = textscan(fid, '%s', 'Delimiter', '\n');
65     lines = lines{1};
66     fclose(fid);
67
68     resultados = struct();
69
70     current_function = '';
71     for i = 1:length(lines)
72         line = strtrim(lines{i});
73         if startsWith(line, 'Función:')
74             current_function = strtrim(strrep(line, 'Función:', ''));
75             resultados.(current_function) = struct('N', [], 'tiempos', []);
76         elseif ~isempty(current_function) && ~isempty(line) && ~startsWith(line, '-')
77             data = sscanf(line, '%d %f');
```

```

78         if ~isempty(data)
79             resultados.(current_function).N(end+1) = data(1);
80             resultados.(current_function).tiempos(end+1) = data(2);
81         end
82     end
83 end
84 end

```

Listing B.3: Script de MATLAB para calcular las ganancias y eficiencias

Script: graficarGanancias.m

Por último, *graficarGanancias.m* como su nombre indica, se encargará de graficar los tiempos de ejecución de cada función junto con el SpeedUp al utilizar diferentes números de hilos.

```

1  maquina="Intel Xeon E5-2620 v3 2.4GHz, 12 Núcleos (DATSI)"; threads = [1, 24,
2     48, 12, 6];
3  %maquina="AMD Ryzen 7 PRO 8840HS 3.3GHz, 8 Núcleos"; threads = [1, 16, 32, 8,
4     4];
5  %maquina="AMD Ryzen 7 5800H 3.2GHz, 8 Núcleos"; threads = [1, 16, 32, 8, 4];
6  %maquina="Intel Core i5-10600KF 4.1GHz, 6 Núcleos"; threads = [1, 12, 24, 6,
7     3];
8
9  % Archivos de resultados
10 files = {
11     'res_mos.txt', ...
12     'res_mpmos.txt', ...
13     'res_mpmos_dt.txt', ...
14     'res_mpmos_ht.txt', ...
15     'res_mpmos_qt.txt'
16 };
17
18 % Leer datoss
19 data = cell(size(files));
20 for i = 1:length(files)
21     data{i} = leer_resultados(files{i});
22 end
23
24 funciones = fieldnames(data{1}); % Basado en el archivo secuencial
25
26 % Crear gráficas
27 for f = 1:length(funciones)
28     funcion = funciones{f};
29     tiempos = zeros(length(files), 1);
30
31     % Obtener el tiempo del último tamaño para cada configuración
32     for i = 1:length(files)
33         tiempos(i) = data{i}.(funcion).tiempos(end);
34     end
35
36     speedup = tiempos(1) ./ tiempos;
37
38     % Ordenar threads y speedup
39     [threads_sorted, sort_idx] = sort(threads);
40     tiempos_sorted = tiempos(sort_idx);

```

```
39     speedup_sorted = speedup(sort_idx);
40
41     % Gráfica
42     figure;
43     yyaxis left;
44     bar(threads_sorted, tiempos_sorted, 'FaceColor', [0.4 0.6 0.8], 'BarWidth',
45         0.5);
46     ylabel('Segundos');
47     xlabel('N de hilos');
48     title([maquina funcion]);
49
50     yyaxis right;
51     plot(threads_sorted, speedup_sorted, '-o', 'LineWidth', 2, 'Color', [1 0.4
52         0.4]);
53     ylabel('Speedup');
54
55     % Guardar la gráfica
56     %saveas(gcf, [funcion, '_speedup.png']);
57 end
58
59 % Leer resultados desde un archivo
60 function resultados = leer_resultados(filename)
61     fid = fopen(filename, 'r');
62     if fid == -1
63         error('No se pudo abrir el archivo: %s', filename);
64     end
65
66     lines = textscan(fid, '%s', 'Delimiter', '\n');
67     lines = lines{1};
68     fclose(fid);
69
70     resultados = struct();
71
72     current_function = '';
73     for i = 1:length(lines)
74         line = strtrim(lines{i});
75         if startsWith(line, 'Función:')
76             current_function = strtrim(strrep(line, 'Función:', ''));
77             resultados.(current_function) = struct('N', [], 'tiempos', []);
78         elseif ~isempty(current_function) && ~isempty(line) && ~startsWith(line, '-')
79             data = sscanf(line, '%d %f');
80             if ~isempty(data)
81                 resultados.(current_function).N(end+1) = data(1);
82                 resultados.(current_function).tiempos(end+1) = data(2);
83             end
84         end
85     end
86 end
87 end
```

Listing B.4: Script de MATLAB para generar gráficas de tiempo y speedup para cada función